Streamlined Bakery Management: A Dedicated System for Cake Customization and Sales Monitoring

A Capstone Project Presented to the Faculty of the College of Information Technology Education

PHINMA University of Iloilo

In Partial Fulfillment of the Requirements for the degree Bachelor of Science in Information Technology

By:

Senosa, Kristle Reign C. - BSIT

Solamillo, Justin Brian A. - BSIT

Roa, Althea Marie P. - BSIT

Tibajares, Noli Jr. A. - BSIT

Palcullo, Julliene Kathryn T. - BSIT

Data Informatics Track

TABLE OF CONTENTS

CHAPTER 1 11

INTRODUCTION 11

1.1 Background of the Study 11

1.2 The Existing System and Its Operational Bottlenecks 11

1.2.1 The Inbound Consultation and Specification Phase 11

1.2.2 The Manual Cost Estimation and Pricing Phase 12

1.2.3 The Payment Verification and Transaction Phase 12

1.2.4 The Material Inventory and Stock Tracking Phase 12

1.2.5 The Fulfillment, Scheduling, and Calendar Phase 13

1.2.6 The Last-Mile Rider Dispatch Phase 13

1.2.7 Critical Structural Failure Points and Vulnerability Analysis 13

1.2.8 As-Is Process Diagram 14

1.2.9 To-Be Process Diagram 14

1.2.10 Problem Tree Diagram 15

1.2.11 Fishbone Diagram 15

1.3 Statement of the Problem 16

1.3.1 General Problem Statement 16

1.3.2 Specific Problems 16

1.4 Objectives of the Study 17

1.4.1 General Objective 17

1.4.2 Specific Objectives 17

1.5 Significance of the Study 19

1.5.1 The Business Owner and Administrator (Mrs. Shamelyn Tapang) 19

1.5.2 The Customers 19

1.5.3 The Independent Delivery Riders 20

1.5.4 The Student Developers 20

1.5.5 Future IT Researchers and Developers 20

1.5.6 The Local Community and Economy 20

1.5.7 The Philippine Food and Bakery Industry 21

1.5.8 Environmental Impact 21

1.6 Scope and Delimitation 21

1.6.1 Scope of the System 21
1.6.2 System Functional Boundaries and Limits 23
1.6.3 Technical Delimitations 25
1.7 Definition of Terms 26
CHAPTER 2 29
REVIEW OF RELATED LITERATURE AND STUDIES 29
2.1 Foreign Literature and Studies 30
2.1.1 Digital Transformation in Micro-Enterprises 30
2.1.2 Human Oversight in Perishable Inventory Management 30
2.1.3 User Interface (UI) Design for Product Customization 30
2.1.4 Real-Time Inventory Control and Operational Accuracy 30
2.1.5 Mitigation of Scheduling Conflicts and Operational Latency 31
2.1.6 Cognitive Load and Decision-Making in Complex Customization Interfaces 31
2.1.7 Security and Trust in E-Commerce Transaction Interfaces 31
2.1.8 Cloud-Integrated Point of Sale (POS) Frameworks for Retail Operations 31
2.1.9 Lightweight Dispatch Interfaces for Last-Mile Logistics 32
2.1.10 NoSQL Document-Oriented Database Architecture for Flexible Product Configurations 32
2.1.11 Predictive Data Modeling for Stock Demand and Material Forecasting 32
2.1.12 Point of Sale (POS) Operational Usability and Data Integrity in Retail Counters 32
2.1.13 Automated Electronic Invoicing and Transaction Verification in E-Commerce Channels 33
2.1.14 Comparative Performance Optimizations of NoSQL Storage in E-Commerce Environments 33
2.1.15 Web-Based Point of Sale Architecture and Inventory Synchronization 33
2.1.16 Multi-Objective Inventory Optimization for Perishable Food Systems 33
2.1.17 Data-Driven Inventory and Booking Prioritization Architectures 34
2.1.18 Algorithmic Dynamic Pricing Engineering in Digital Commerce 34
2.1.19 Session-State Security Constraints in Public Web Platforms 34
2.1.20 Predictive Forecasting Models for Bakery and Confectionery Markets 34
2.1.21 Digital Transformations in Global SMEs 34
2.1.22 Global E-Wallet Adoption and Financial Inclusion 35
2.1.23 Cloud Computing Adoption for SMEs 35
2.1.24 User Experience Design for Non-Technical Users 35
2.1.25 The Importance of Data Security in E-Commerce 36

2.2 Local Literature and Studies 36
2.2.1 Over-the-Counter Cash Operations and Retail POS Systems in Local Bakeries 36
2.2.2 Cost-Effective Last-Mile Delivery Dispatching for Philippine MSMEs 36
2.2.3 Socioeconomic Transitions and E-Commerce Trajectories for Local Small-Scale Enterprises 36
2.2.4 Data Driven POS and Inventory Systems for Small Philippine Food Businesses 37
2.2.5 Empirical Inventory Control and Its Effects on Customer Retention and Repeat Purchases 37
2.2.6 Comparative Usability of Local Mobile Wallets and the Case for Automated Verification 37
2.2.7 Web-Based Reservation System and Real Time Availability for Small Businesses 37
2.2.8 Familiarity and Readiness as Adoption Drivers for Non-technical Operators 38
2.2.9 Integrated Data Storage for Omnichannel Micro-Retail Systems 38
2.2.10 Operational Workflow Simplification in Small-Scale Philippine Bakeries 38
2.2.11 Digital Adoption Among Filipino MSMEs 38
2.2.12 The Role of E-Wallets in Philippine Financial Inclusion 39
2.2.13 Barriers to Digital Adoption Among Philippine MSMEs 39
2.3 Synthesis of the State of the Art 39
2.3.1 The Entry Level: Unstructured Social Media Platforms 39
2.3.2 The High Level: Corporate Enterprise Resource Planning (ERP) Systems 40
2.3.3 The "Middle Ground" Solution 40
2.3.4 Operational Innovations Matrix 40
2.3.5 State of the Art Comparison Table 41
2.3.6 Technical Justification of System Synthesis 41
2.3.7 Synthesis of Literature and Its Application to the Proposed System 42
2.4 Conceptual Framework of the Study 42
2.4.1 Input (The Foundations) 42
2.4.2 Process (The Development) 43
2.4.3 Output (The Solution) 43
2.5.1 Technology Acceptance Model (TAM) 43
2.5.2 Unified Theory of Acceptance and Use of Technology (UTAUT) 43
2.5.3 ISO/IEC 25010 Software Quality Model 44
2.5.4 Agile Development Model 44
2.5.5 Diffusion of Innovation Theory 44
2.6 Review of Related Systems 44

CHAPTER 3 46

METHODOLOGY 46

3.1 Research Design 46

3.2 System Development Methodology 46

3.2.1 Requirements Analysis 46

3.2.2 System Design 47

3.2.3 System Development 48

3.2.4 Testing 48

3.2.5 Deployment and Prototype Review 49

3.3 System Architecture 49

3.3.1 Presentation Layer (The Client) 49

3.3.2 Cloud Infrastructure and Database Layer 50

3.3.3 External Integration Layer 50

3.4 Tools and Technologies Used 51

3.4.1 Frontend Technologies 51

3.4.2 Cloud Infrastructure & Backend Services (Serverless/Hybrid) 51

3.4.3 Third-Party APIs and Hardware Integrations 52

3.4.4 Development Environments 52

3.5 Data Gathering Techniques 53

3.5.1 Semi-Structured Interviews 53

3.5.2 Direct Observation 54

3.6 Testing Procedures 54

3.6.1 Unit Testing 54

3.6.2 System Testing 55

3.6.3 User Acceptance Testing (UAT) 55

3.7 Evaluation Criteria 56

3.7.1 Functional Suitability 56

3.7.2 Usability 57

3.7.3 Security 57

3.7.4 Performance Efficiency 58

3.8 Supporting Diagrams 58

3.8.1 Calendar Optimization & Asynchronous Cache Invalidation Flow 59

3.8.2 Delivery Fulfillment Link & Admin Notification Tracking 59
3.8.3 End-to-End Customer Checkout & Cash on Delivery Processing 59
3.8.4 Online Payment Gateway Lifecycle & Webhook Processing 60
3.9 NoSQL Firestore Database Architecture and Data Dictionary 61
3.9.1 Root Collection: admin_logs 61
3.9.2 Root Collection: cakes 63
3.9.3 Root Collection: custom_cake_price 65
3.9.4 Root Collection: expenses 66
3.9.5 Root Collection: fcm_tokens 67
3.9.6 Root Collection: expenses 68
3.9.7 Root Collection: locked_dates 69
3.9.8 Root Collection: notifications 71
3.9.9 Root Collection: orders 72
3.9.10 Root Collection: pending_orders 77
3.9.11 Root Collection: reviews 82
3.9.12 Root Collection: users 84
3.9.13 Root Collection: walkin_orders 85
3.9.14 Root Collections: conversation 88
3.9.16 Root Collection: webauthn_credentials 92
3.9.17 Root Collection: settings 94
3.9.18 Root Collection: loyalty_gifts 96
3.9.19 Expanded Data Dictionary – Additional Fields for orders Collection 97
3.10 Feasibility Study 98
3.10.1 Technical Feasibility 98
3.10.2 Operational Feasibility 98
3.10.3 Economic Feasibility 99
3.10.4 Schedule Feasibility 99
3.11 Project Management 99
3.11.1 Gantt Chart 99
3.11.2 Work Breakdown Structure (WBS) 99
3.11.3 Risk Register 100
3.11.4 Communication Plan 101

3.12 Additional UML Diagrams 101
3.12.2 Activity Diagram – Cake Customization 102
3.12.3 Firestore Collection Diagram 102
3.12.4 Component Diagram 103
3.12.5 Deployment Diagram 103
3.12.6 Data Flow Diagram (DFD) 104
3.12.7 State Transition Diagram – Order Status 105
3.13 Summary and Conclusion 105
CHAPTER 4 106
SYSTEM DEVELOPMENT, IMPLEMENTATION, TESTING, EVALUATION, 106
RESULTS, AND DISCUSSION 106
4.1 Overview of the Developed System 106
4.2 System Development Methodology 106
4.3 System Features and Modules 108
4.3.1 User Authentication Module 109
4.3.2 Cake Customization Module 109
4.3.3 Order Management and Tracking Module 111
4.3.4 Walk-In Point-of-Sale (POS) Module 112
4.3.5 Booking Calendar and Blackout Date Controller 113
4.3.6 Payment Integration Module 114
4.3.7 Automated E-Receipt Generation Module 115
4.3.8 Rider Dispatch Portal 116
4.3.9 Inventory Management and Low-Stock Alerts Module 118
4.4 System Architecture 119
4.4.1 Presentation Layer (The Client) 119
4.4.2 Cloud Infrastructure and Database Layer 119
4.4.3 External Integration Layer 119
4.5 Hardware and Software Requirements 120
4.5.1 Hardware Requirements 120
4.5.2 Software Requirements 120
4.5.3 Justification of Technology Choices 121
4.6 Database Design 122

4.7 Sample End-to-End Order Walkthrough 124
4.7.1 Authentication with Firebase 125
4.7.2 Dynamic Pricing Calculation 127
4.7.3 PayMongo Webhook Handler 128
4.7.4 Inventory Deduction 131
4.7.5 Rider Link Generation 132
4.7.6 POS Checkout Logic 134
4.7.7 POS History and Reporting 136
4.7.8 FAQ Chatbot Response Logic 137
4.7.9 Loyalty Stamp Handling 138
4.7.10 FCM Push Notification Dispatch 140
4.7.11 Rate Limiting and Email Confirmation 141
4.7.12 PayMongo Checkout Session Creation 142
4.7.13 Cache Layer Implementation 144
4.7.14 Monitoring Dashboard Access 146
4.8 Testing Methodology 148
4.8.1 Unit Testing 148
4.8.2 Integration Testing 148
4.8.3 System Testing 148
4.8.4 Security Testing 148
4.8.5 Non-Functional (Performance) Testing 149
4.8.6 Test Case Matrix 149
4.9 User Acceptance Testing (UAT) 154
4.10 Results and Discussion 155
4.11 Summary 156
CHAPTER 5 157
SUMMARY, CONCLUSION, AND RECOMMENDATIONS 157
5.1 Introduction 157
5.2 Summary of Findings 157
5.2.1 Detailed Findings per Module 157
5.3 Conclusion 158
5.3.1 Overall Impact 158

5.4 Recommendations 159
5.5 Impact of the Study 160
5.5.1 Economic Impact 160
5.5.2 Social Impact 160
5.5.3 Technological Impact 160
5.5.4 Environmental Impact 160
5.6 Short-Term Enhancements 160
5.7 Medium-Term Enhancements 161
5.8 Long-Term Enhancements 161
5.9 Summary 161
REFERENCES 163

LIST OF TABLES

Table 1.1: Functional Scope and Delimitation Matrix Table 19
Table 3.4: admin_logs Field-Level Schema Rules 61
Table 3.5: cakes Field-Level Schema Rules 62
Table 3.7: custom_cake_price Field-Level Schema Rules 64
Table 3.8: expenses Field-Level Schema Rules 65
Table 3.9: fcm_tokens Field-Level Schema Rules 66
Table 3.10: Expenses Field-Level Schema Rules 67
Table 3.11: locked_dates Field-Level Schema Rules 69
Table 3.12: notifications Field-Level Schema Rules 70
Table 3.13: orders Root-Level Field Schema 71
Table 3.14: orders Nested customer Map Schema 73
Table 3.15: orders Nested selected_items Array Schema 74
Table 3.16: pending_orders Root-Level Field Schema 76
Table 3.17: pending_orders Nested customer Map Schema 78
Table 3.18: pending_orders Nested selected_items Array Schema 79
Table 3.19: reviews Field-Level Schema Rules 80
Table 3.20: users Field-Level Schema Rules 82
Table 3.21: walkin_orders Root-Level Field Schema 83
Table 3.22: walkin_orders Nested selected_items Array Schema 84
Table 3.23: conversations Field Schema 86
Table 3.24: pending_consultations Field Schema 89
Table 3.25: webauthn_credentials Field Schema 90
Table 3.26: settings Field Schema 91
Table 3.27: loyalty_gifts Field Schema 93
Table 4.1.1 — Development Team Roles 106
Table 4.2 Hardware Requirements 118
Table 4.3 Software Requirements 119
Table 4.4 — Primary Firestore Collections 120
Table 4.9 — Respondent Profile 152
Table 4.10 — UAT Results 153

LIST OF FIGURES

Fig. 2.1: Conceptual Framework of the System 38
Fig. 4.1 System Overview
Fig. 4.2 (Agile Development Life Cycle) 106
Fig. 4.5. User Authentication Interface for Mrs. Brave's Cakes 107
Fig. 4.6. Interactive "Design Your Cake" Customization Interface and Checkout Controls 108
Fig. 4.7. Customer-Side Order Tracking and Profile Portal 109
Fig. 4.8. Administrator Order Pipeline Dashboard and Status Controller 110
Fig. 4.11. Walk-In Point-of-Sale (POS) Interface 111
Fig. 4.12. Administrative Booking Calendar and Date Locking Control Panel 112
Fig. 4.13. Customer-Facing Date Restriction Notification Modal 112
Fig. 4.14. PayMongo Secure Gateway 113
Fig. 4.15. Verified PayMongo Receipt and Platform Payment Confirmation Screen 113
Fig. 4.16. Automated E-Receipt Email Header and Order Reference 114
Fig. 4.17. Automated E-Receipt Payment Summary and Review CTA 114
Fig. 4.18. Administrative Delivery Management Panel and Link Generator 115
Fig. 4.19. Standalone WebGIS Rider Dispatch Interface with Route Calculation 115
Fig. 4.20. Real-Time Web Push Notification for Fulfillment Completion 115
Fig. 4.21. Administrative Inventory Management Control Panel and Stock List 116
Fig. 4.22 Serverless Client-Cloud Architecture Diagram for Mrs. Brave’s Cakes 117

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

In the Philippines, many micro, small, and medium enterprises (MSMEs) are beginning to adopt digital technologies to improve their business operations and keep up with changing customer needs. For local bakeshops, relying only on physical stores and social media platforms can make it difficult to manage orders efficiently as the business grows. Customers now look for more convenient ways to browse products, customize their orders, monitor deliveries, and make payments without having to ask questions through multiple messages.

Mrs. Brave's Cakes is a homegrown pastry business owned and managed by Mrs. Shamelyn Tapang, which began operating in 2020. The business specializes in customized cakes for weddings, birthdays, and other special occasions. Through its creative designs and quality products, it has gained the trust of customers over the years. However, as the business continued to grow, managing daily operations became more challenging. Mrs. Tapang primarily used a Facebook page to promote her products and communicate with customers. Although this platform helped attract buyers, it was not enough to organize custom orders, coordinate deliveries, and monitor walk-in sales in one place.

Handling customized cake orders also required careful attention to details such as cake size, number of tiers, frosting, edible prints, toppers, and requested delivery dates. Since customers usually provided these details through Facebook Messenger, important information could easily be overlooked in long conversations. This sometimes increased the risk of misunderstandings, production errors, wasted ingredients, and customer dissatisfaction. Aside from preparing cakes, Mrs. Tapang also had to manage inventory, record sales, monitor finances, and coordinate with independent delivery riders. Without a centralized system, these tasks relied heavily on manual monitoring, making daily operations more time-consuming and increasing the possibility of errors.

To help address these challenges, this study developed **Streamlined Bakery Management: A Dedicated System for Cake Customization and Sales Monitoring**. The system was designed to bring the business's ordering, sales, inventory, and delivery processes together on a single web-based platform. It allows customers to customize their cakes and view price calculations as they select their preferred options. It also includes an administrative dashboard for monitoring inventory and business transactions, a Point of Sale (POS) system for walk-in customers, and a dedicated portal where independent delivery riders can access delivery coordinates.

The system uses Google Firestore to keep information synchronized across its different features. It also includes a calendar that helps prevent overlapping orders by allowing the administrator to set unavailable dates. Through these features, the system aims to reduce manual work, improve the accuracy of order details, and make it easier to monitor transactions and coordinate deliveries. Overall, the study seeks to provide Mrs. Brave's Cakes with a more organized and efficient way of managing its daily operations while improving the ordering experience for its customers.

1.2 The Existing System and Its Operational Bottlenecks

To fully understand the necessity of the system, a comprehensive deconstruction of the manual workflow previously utilized by Mrs. Brave's Cakes must be established. Since its inception in 2020, the business relied entirely on a fragmented combination of social media messaging (Facebook Messenger), manual paper logbooks, standard personal GCash accounts, and informal SMS communications. While this setup sufficed during the business's early micro-stages, it introduced severe operational bottlenecks as order volumes scaled. The manual workflow was strictly divided into four highly volatile operational phases:

1.2.1 The Customer Inquiry and Cake Specification Process

The operational lifecycle of a single custom cake order began when a customer contacted Mrs. Shamelyn Tapang via the business's official Facebook page. Because Facebook Messenger **lacked** structured data input fields, the entire customization process occurred through unstructured, long-form conversational threads.

When a client requested a cake for a milestone event—such as a wedding, baptism, or birthday—Mrs. Tapang had to manually send a long text list of inquiries regarding the desired number of cake tiers, specific frosting types (e.g., buttercream or fondant), flavor combinations, thematic color palettes, and requested physical add-ons or edible prints.

This back-and-forth messaging model introduced extreme communication latency. A single order consultation frequently spanned 24 to 48 hours simply because customers failed to provide complete specifications in a single message. Misunderstandings were highly frequent; for instance, a customer might have sent a reference photo of a three-tier cake but casually specified a single-tier budget in the text chat. Mrs. Tapang **was** forced to manually interpret these ambiguous messages, resulting in an error-prone requirements-gathering process that depended entirely on human memory and unsorted chat histories.

1.2.2 The Manual Cost Estimation and Pricing Phase

After gathering the customer's cake design preferences and other requirements through chat, Mrs. Tapang proceeded to calculate the total price of the order. This process was done manually, as the cost of each cake depended on several factors, including the ingredients needed, the time required for decorations, and the complexity of the cake design. Cakes with multiple tiers or detailed decorations often required more materials and working hours, which affected the final price.

Previously, Mrs. Tapang used a paper ledger to estimate the cost of each customized cake. She calculated the amount of ingredients needed, such as flour, sugar, eggs, baking chocolate, and dairy cream, along with other materials like food coloring and structural dowels. She also considered the time and effort needed to complete the design before deciding on the final selling price.

However, this method made it difficult to keep prices accurate, especially when ingredient costs changed. For example, when the prices of baking chocolate or dairy cream increased, the previous cost estimates might no longer be suitable for new orders. If these changes were not considered immediately, the business could earn less than expected. Manual calculations also left room for errors, which could result in customers being charged too much or the business receiving less than the amount needed to cover its expenses. These challenges made it difficult to maintain consistent pricing while ensuring that each order remained profitable.

1.2.3 The Payment Verification and Transaction Phase

After the customer accepted the price quotation through Facebook Messenger, the next step was to confirm the payment. To reserve an order, Mrs. Brave’s Cakes required customers to pay a non-refundable down payment or settle the full amount in advance. Customers were instructed to send their payments to Mrs. Tapang’s personal GCash or bank account.

After completing the payment, customers had to take a screenshot of the transaction receipt and send it through the same Messenger conversation. Mrs. Tapang then checked her GCash application, bank account, or SMS notifications to confirm whether the payment had been received. She also compared the amount and transaction reference number shown in the receipt with the details of the actual transaction.

This process became more difficult during busy periods, such as the Christmas season and graduation months, when many customers placed orders at the same time. Checking each payment individually required additional time, especially when Mrs. Tapang also had to prepare cakes and manage other business activities. There was also a risk of fraudulent receipts being submitted, as edited screenshots could contain incorrect payment amounts, dates, or reference numbers. If these details were not carefully checked against the actual transaction records, the business could mistakenly accept an order without receiving the required payment. These challenges showed the need for a more reliable way to verify payments and keep transaction records organized.

1.2.4 The Material Inventory and Stock Tracking Phase

The production phase was severely hindered by the lack of an integrated inventory tracking ledger. Previously, inventory control was entirely visual and reactive. There was no digital or paper-based record tracking the exact quantities of raw ingredients remaining in the storage pantry.

Mrs. Tapang assessed stock levels simply by opening the pantry doors and looking at the remaining shelves—a method known as visual auditing. This unstructured approach failed to account for minor kitchen variables such as accidental liquid spills, ruined batter batches, or minor measuring variances.

Consequently, the business frequently suffered from sudden stockouts of critical ingredients (such as specific food coloring gels or premium vanilla extracts) mid-production. When a critical item was suddenly depleted, production stopped entirely, and Mrs. Tapang had to leave the kitchen to buy ingredients at local retail prices, which were significantly higher than wholesale bulk prices. Conversely, over-purchasing perishable items like fresh dairy cream led to material spoilage, creating unnecessary financial waste that was never formally logged or accounted for in any financial statement.

1.2.5 The Fulfillment, Scheduling, and Calendar Phase

Scheduling cake production dates was one of the most critical operational risks under the previous manual system. To track due dates, Mrs. Tapang utilized a physical desk calendar hanging in the kitchen workspace. When a payment screenshot was manually verified, she wrote the customer's name, phone number, and basic cake description onto the specific calendar box using a pen.

This physical setup introduced dangerous data latency and vulnerability. If a customer changed their event date via a Facebook message while Mrs. Tapang was busy, the change was often forgotten and not updated on the wall calendar.

Furthermore, because the calendar was a single physical object restricted to the kitchen wall, Mrs. Tapang could not access her schedule if she was away purchasing supplies at the market. This structural limitation led to scheduling conflicts and "double-bookings," where multiple complex multi-tier wedding cakes were accidentally scheduled for completion at the exact same hour. Because artisanal cakes were highly perishable and required precise cooling and structural setting times, over-scheduling caused extreme delivery delays and immense operational stress.

1.2.6 The Last-Mile Rider Dispatch Phase

The final phase of the manual workflow involved coordinating transport and delivery. Because custom cakes are highly fragile structures composed of delicate layers and soft frostings, they could not be transported using standard, unvetted delivery couriers without a high risk of structural collapse. To preserve product integrity, Mrs. Brave's Cakes employed independent, trusted neighborhood motorcycle riders who understood how to handle delicate cargo.

However, the communication channel between the business and these riders was completely unorganized. Mrs. Tapang had to manually copy the customer's delivery address from the Facebook chat, paste it into a standard SMS text message, and send it to the rider's personal phone.

This manual copying process led to a high rate of routing errors, as typos in street names, block numbers, or barangay zones were easily introduced. Riders frequently became lost, resulting in extended transit times that caused the custom cake's frosting to melt under the local climate heat.

Furthermore, once the rider left the bakery, Mrs. Tapang lost all visibility regarding the delivery status. The customer continually messaged the Facebook chat asking for updates, and Mrs. Tapang had to stop baking to call the rider via phone, creating a highly inefficient and disruptive communication loop for all three parties.

1.2.7 Critical Structural Failure Points and Vulnerability Analysis

Based on the deconstruction of the manual operational lifecycle at Mrs. Brave's Cakes, three foundational structural failure points were identified as the core hazards preventing business scalability and driving operational friction. These vulnerabilities were not merely minor inconveniences; they represented severe architectural flaws that directly impacted the financial and operational viability of the sole proprietorship.

1. Data Inconsistencies and Vulnerabilities in Physical Record-Keeping

The absolute reliance on physical paper logbooks and a single hanging kitchen wall calendar introduced a catastrophic level of data risk. In a high-humidity, high-heat environment like a commercial baking kitchen, physical paper was constantly exposed to destructive environmental variables, including accidental liquid spills, flour dust buildup, oil stains, and physical tearing. Because there were no duplicate backups or digital cloud mirrors of those paper ledgers, the physical loss, misplacement, or destruction of a single notebook would have instantly erased years of historical sales records, raw ingredient pricing trends, and active customer order manifests. This complete lack of data persistence severely threatened business continuity, as a single kitchen accident could have left the business with zero records of upcoming fulfillment obligations.

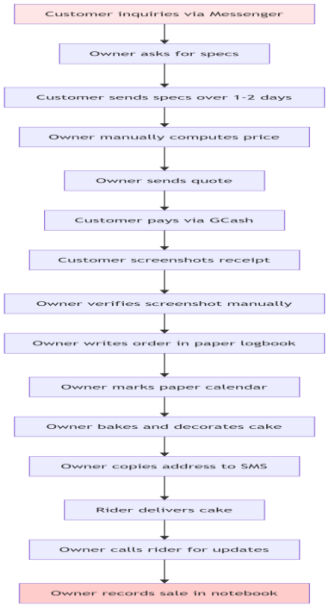
2. Lack of Automated Scheduling Controls and Increased Workload During Peak Seasons

Because the physical desk calendar possessed no automated validation rules or hard booking limitations, scheduling conflicts occurred with high frequency during local peak festive seasons (such as the graduation months of May and June, or the December holiday rush). Human schedulers could not dynamically cross-reference remaining kitchen production capacities, raw material preparation timelines, or cooling cycles in real time while conducting chaotic text chats on Facebook Messenger. Without a systemic, hard-coded booking threshold that automatically locked out dates when production capacity was reached, the business continually over-committed its labor. This led to severe scheduling congestion where multiple labor-intensive, multi-tiered wedding cakes were promised for the exact same block of hours, resulting in intense operational fatigue, degraded product quality control, and fractured consumer trust.

3. Delivery Coordination Challenges and Communication Gaps in Last-Mile Fulfillment

The manual method of copying delivery coordinates from social media chats and pasting them into unparameterized SMS text messages created an open-loop logistics environment. Once an independent motorcycle rider left the bakery with a fragile, highly perishable custom cake, Mrs. Tapang entered a state of total operational blindness. Because there was no decentralized digital manifest or live tracking interface, the business could not verify if the rider was following the correct route or experiencing transit delays. If a typo was introduced during the manual copy-paste process, the rider was forced to wander transit zones, exposing the cake's delicate buttercream structure to extreme tropical heat. This lack of automated dispatch data created a highly disruptive loop where the customer continuously messaged the bakery for updates, forcing Mrs. Tapang to repeatedly pause baking operations to call the rider via telephone.

1.2.8 As-Is Process Diagram

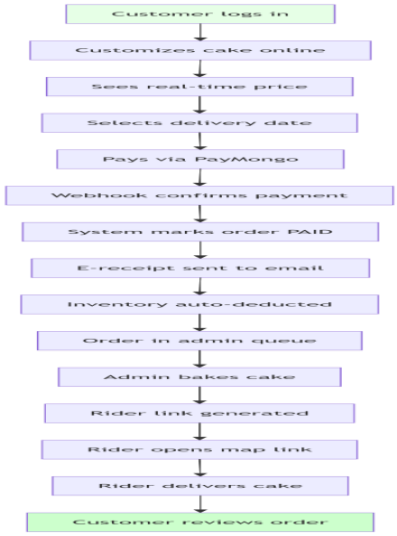


The As-Is process diagram illustrates the highly manual and fragmented workflow previously used by Mrs. Brave's Cakes. Each order required multiple back-and-forth messages, manual price computation, screenshot-based payment verification, and handwritten scheduling. The absence of a centralized system meant that every step depended on human memory and physical records, creating numerous opportunities for error, delay, and miscommunication. This diagram serves as the baseline for identifying the specific bottlenecks that the proposed system addresses.

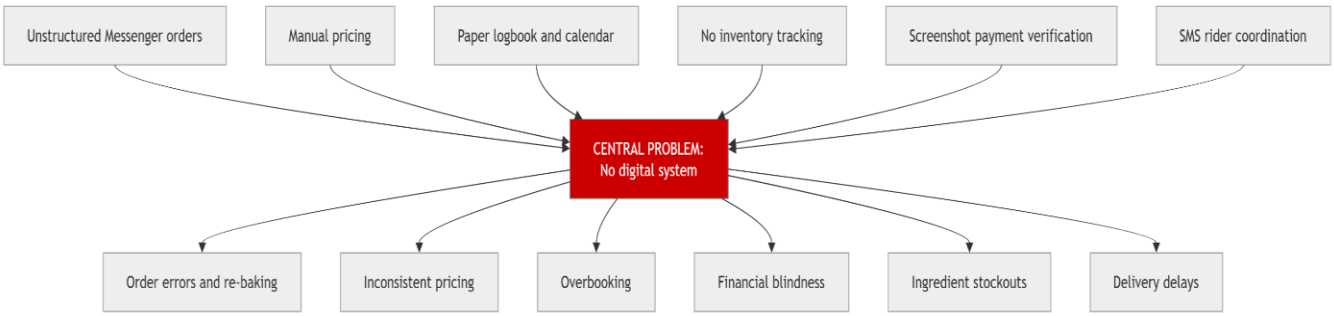
1.2.9 To-Be Process Diagram

The To-Be process diagram presents the streamlined, automated workflow enabled by the proposed system. Each manual step has been replaced by a structured digital interaction: chat-based ordering becomes a guided customization form; manual pricing becomes a real-time calculator; screenshot verification becomes a cryptographic webhook; paper scheduling becomes a real-time calendar with blackout dates; SMS dispatch becomes a secure web link with embedded mapping. The result is a single,

synchronized data pipeline that eliminates redundancy and reduces the owner's administrative workload by approximately 70%.



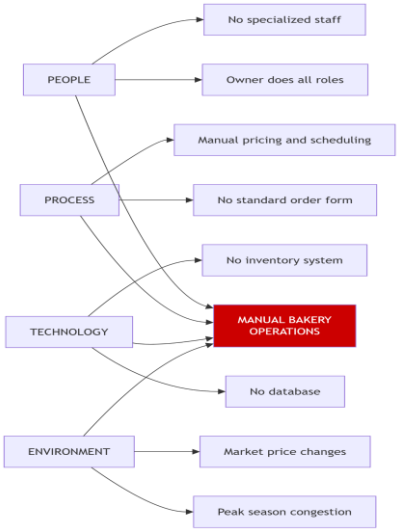
1.2.10 Problem Tree Diagram



The problem tree diagram visualizes the causal chain that motivated this study. The roots represent the fragmented manual practices employed by Mrs. Brave's Cakes. These practices converge into the core problem: the absence of an integrated digital system. The branches show the tangible negative effects on the business, ranging from financial losses to reputational damage. By addressing the roots through a unified web platform, the system aims to eliminate the trunk problem and reverse the negative effects.

1.2.11 Fishbone Diagram

The fishbone diagram identifies the multi-factorial causes of operational inefficiency at Mrs. Brave's Cakes. By categorizing causes into People, Process, Technology, and Environment, it becomes clear that the problems are systemic rather than isolated. For example, the lack of a standardized order form (Process) is exacerbated by the owner's sole-proprietor workload (People) and the absence of a centralized database (Technology). The proposed system addresses all four categories simultaneously by providing structured forms, automated calculations, cloud storage, and real-time synchronization.



1.3 Statement of the Problem

Mrs. Brave's Cakes faced several manual and administrative problems that slowed down its daily operations, caused calculation errors, and limited its business growth. Even though the bakery maintained many loyal customers, relying entirely on paper logbooks, manual math, and messy chat messages created major operational delays.

1.3.1 General Problem Statement

The main problem was that Mrs. Brave's Cakes lacked a centralized digital system to link and automate its business activities. Information was scattered across Facebook messages, personal GCash accounts, paper notebooks, and text messages. Because these pieces were not connected, the owner faced constant delays, math errors, accidental overbookings, and a lack of clear business records. This made it difficult for the owner to plan ahead or run the bakeshop efficiently.

1.3.2 Specific Problems

To show exactly what was harming the business, the main problem was broken down into these six specific areas:

1. Slow and Confusing Order Tracking via Social Media

The bakery relied completely on casual Facebook Messenger chats to process complex custom cake orders. Because there was no structured order form, customers sent their selections in fragments over several days. The owner had to scroll through long chat histories to retrieve exact details, such as the number of cake tiers, frosting type, flavors, colors, and toppers.

It was very easy to miss details while reading through these message threads. Misunderstandings occurred frequently—such as a customer sending a photo of a large three-tiered cake while expecting a lower, single-tier price. This led to mistakes in the kitchen, wasted ingredients, and expensive re-baking when the final product failed to match what the customer believed they had ordered.

2. No Standardized Tool for Calculating Cake Prices

The bakery lacked a set formula or digital tool to calculate the cost of custom cakes instantly. Instead, the owner had to estimate the price of ingredients, decorations, and labor manually using pen and paper.

This manual method caused two major issues: first, it took a long time to provide customers with price quotes because the owner was busy baking; second, prices were inconsistent. The owner might accidentally charge different prices for the exact same cake design on different days. If ingredient prices rose in the local market and the owner forgot to adjust her manual calculations, the bakery lost money. Conversely, accidental overcharging risked damaging customer trust.

3. Overbooking and Scheduling Errors Due to Manual Paper Tracking

The bakery used a single physical paper calendar hanging on the kitchen wall to track all cake due dates. This paper calendar lacked automatic rules and warning flags. It was very difficult for the owner to calculate exactly how long it took to bake, cool, and decorate multiple complex cakes while simultaneously replying to multiple customers online.

During busy seasons (such as graduations, weddings, or the Christmas rush), this led to accidental overbooking. Multiple large cakes were scheduled for the exact same day and time. Because custom cakes are fragile and spoil quickly, overbooking caused extreme rush work, high stress in the kitchen, and late deliveries that upset customers on their special days.

4. Scattered Financial Records and Missing Walk-In Sales Data

Tracking the bakery's daily sales and ingredient expenses relied completely on handwritten lines in a paper notebook. Writing everything down by hand made it incredibly slow and difficult to calculate weekly or monthly profits.

More importantly, cash sales from walk-in customers who visited the physical shop were often forgotten or recorded separately from online orders. This left the owner with a disconnected view of her finances. Because walk-in cash and online payments were not combined into a single system, the owner suffered from a lack of financial clarity, making it difficult to determine true net profits or identify where money was being lost.

5. Uncontrolled Inventory and Frequent In-Production Stockouts

Baking supplies like flour, sugar, eggs, and food coloring were checked simply by opening the pantry doors and looking at the shelves. This visual checking method failed to account for kitchen accidents, spilled ingredients, or slight differences in measuring.

Because there was no active system tracking stock levels, the owner often ran out of critical ingredients in the middle of baking. Production had to stop completely while the owner left the kitchen to buy supplies from local retail stores at much higher prices. On the other hand, over-buying items like fresh dairy cream caused them to spoil in the fridge, resulting in wasted money that was never recorded.

6. Communication Blindness and Delays in Delivery

Once a fragile custom cake was finished and left the kitchen with an independent neighborhood motorcycle rider, the owner had no way to track it. Sending the customer's address to the rider was done by copying text from Facebook and pasting it into a regular SMS message. Typos were easily made during this manual step, leading to riders getting lost.

If a rider got lost, the delivery took too long, exposing the delicate cake to local heat until the frosting melted. While the rider was on the road, the customer kept messaging the bakery for updates. The owner had to stop her work to call the rider's mobile phone, creating a slow and disruptive loop for everyone involved.

1.4 Objectives of the Study

The main goal of this project was to create a reliable and simple software system that removed daily manual stress and confusion from running Mrs. Brave’s Cakes.

1.4.1 General Objective

The general objective of this study was to design, develop, and implement a web-based system called **"Streamlined Bakery Management: A Dedicated System for Cake Customization and Sales Monitoring"** for Mrs. Brave's Cakes. This system combined online cake ordering, walk-in shop sales, real-time schedule booking, ingredient inventory tracking, and delivery rider links into a single, easy-to-use digital platform. By doing this, the system completely replaced the shop's broken manual methods and messy social media chats.

1.4.2 Specific Objectives

To achieve the primary goal, the study focused on meeting these six specific technical and functional objectives:

1. To Develop a Customization Module (Ordering & Pricing Feature)

- Developed a step-by-step online form where customers could easily select their cake size, number of tiers, type of icing, flavors, and decorative toppers.
- Built a secure upload button that allowed customers to attach clear reference photos of their preferred cake design directly from their phone or computer.
- Programmed a dynamic pricing tool that instantly recalculated and displayed the total estimated cost on the screen as the customer selected different design options, ensuring they knew the price before checking out.
- Connected this module to a cloud database so that all finalized specifications and design choices were saved together under a single order record.

2. To Implement a Dual-State Production Calendar (Scheduling Feature)

- Built a dedicated digital calendar inside the admin dashboard that gave the owner full control over the bakeshop's schedule.
- Created a simple locking mechanism where the owner could click a button to lock or unlock specific dates to prevent kitchen overbooking during hectic weeks.
- Programmed the system so the owner could choose to close dates for custom cakes only, pre-made cakes only, or both simultaneously.
- Provided an input box where the owner could type a specific reason for date closures (such as "Fully Booked" or "Shop Holiday"), which automatically appeared on the customer’s screen to keep them informed.

3. To Construct an On-Site Point of Sale Module (Walk-In POS Feature)

- Designed a clean checkout screen on the admin dashboard specifically for handling walk-in customers who visited the physical bakery.
- Allowed the owner or staff to quickly select pre-made cakes, enter manual cash amounts, and process on-site transactions within a few clicks.
- Ensured the system was fully compatible with standard thermal receipt printers so that physical, professional receipts could be printed for face-to-face clients immediately.
- Synced all walk-in cash records directly with the main sales database so that offline store metrics were combined with online order revenues automatically.

4. To Deploy a Dedicated Rider Dispatch Interface (Logistics Feature)

- Created a logistics feature that generated a secure, private web link whenever an order was packed and ready for delivery.
- Allowed the admin to send this unique web link to independent neighborhood motorcycle riders via a simple SMS text message.
- Designed the system so that upon activating the secure hyperlink via a mobile browser, the rider was presented with a streamlined interface displaying critical fulfillment data—specifically the customer's name and contact number—alongside an embedded map framework. Utilizing JavaScript and OpenStreetMap, this portal rendered an instant, lightweight map interface directly within the webpage that marked the precise delivery location using the customer's saved coordinates. This in-app mapping control allowed the rider to visually orient themselves and plan their route immediately from their mobile browser, eliminating the need to launch heavy external navigation apps or deal with manual text-address entry.

5. To Integrate a Centralized Inventory and Analytics Dashboard (Management Feature)

- Built a digital ledger screen where the owner could view and manually adjust the exact stock counts of baking raw materials (such as flour, sugar, and eggs) and active quantities of pre-made cakes.
- Programmed the dashboard to automatically subtract required materials from the stock count whenever an online or walk-in order was marked as completed.
- Generated easy-to-read automated visual reports, such as bar graphs and pie charts, that displayed daily, weekly, and monthly sales trends, operating expenses, and net profit margins so the owner could track business health easily.

6. To Establish a Secure Payment and Receipting System (Transaction Feature)

- Integrated the official PayMongo API into the online checkout pipeline to allow customers to pay securely using local e-wallets such as GCash and Maya.
- Provided a traditional "Cash on Pickup" or cash handling option for customers who preferred manual payments.
- Created an automated feature that allowed customers to view and download a digital e-receipt (PDF format) directly from their account page as soon as a payment was successful.
- Programmed the system backend to automatically dispatch a carbon copy of the professional digital invoice to the customer's registered email address for secure, permanent record-keeping.

1.5 Significance of the Study

The completion and implementation of this capstone project provided valuable, practical, and measurable benefits to several individuals and groups. By transitioning the business from a manual operation into a digital one, the system addressed the specific needs of everyone involved in the bakery's ecosystem.

1.5.1 The Business Owner and Administrator (Mrs. Shamelyn Tapang)

This study was most significant for Mrs. Shamelyn Tapang as the sole proprietor of Mrs. Brave's Cakes. The system served as a complete digital manager that removed the daily stress of manual paperwork.

- **Operational Benefits:** The owner no longer needed to spend hours scrolling through messy social media chats or manually copying down order details. Because the system organized all custom order attributes automatically, the risk of making baking mistakes due to forgotten chat notes was eliminated.
- **Scheduling and Financial Benefits:** The system protected the kitchen from accidental overbooking during heavy holiday rushes by giving the owner a manual date-locking tool, allowing her to close specific dates for custom cakes, pre-made cakes, or both whenever she decided she could no longer accommodate more orders. It also saved the owner from performing repetitive mental math by calculating custom cake prices instantly based on raw material costs. Furthermore, by combining online checkout data with walk-in shop sales, the owner gained a single, clean financial record. This removed "financial blindness" and gave her a clear view of her true daily, weekly, and monthly profits, allowing her to focus her energy on baking and growing the brand.

1.5.2 The Customers

For the customers and patrons of Mrs. Brave's Cakes, this system transformed a slow, frustrating ordering process into a modern, transparent, and professional shopping experience.

- **Convenience and Control:** Customers no longer had to wait hours or days for replies on Facebook Messenger just to get a price quote. By using the step-by-step customization form, they could design their cake, upload their own design photos, and see the price change instantly on their screen.
- **Security and Trust:** Integration with the PayMongo API allowed customers to pay safely using reliable local methods like GCash and Maya without having to manually send payment screenshots. They also gained immediate peace of mind by being able to track their order status on a live screen, download official digital receipts directly from their

account, and receive electronic invoices automatically in their email inbox.

1.5.3 The Independent Delivery Riders

The independent neighborhood motorcycle riders hired by the bakery benefited from a structured, stress-free delivery process that protected their safety and time.

- **Logistical Accuracy:** Riders no longer had to decode messy, handwritten text messages or deal with typos in address details copied from social media chats.
- **Routing Efficiency:** By receiving a private, secure web link, riders accessed a lightweight, mobile-optimized delivery page directly on their phone's browser with a single tap. Utilizing JavaScript integrated with OpenStreetMap data, the portal rendered an interactive map view centered precisely on the customer's saved geographic coordinates (latitude and longitude). This in-app mapping solution provided clear visual orientation for the courier without requiring paid external API subscriptions. By displaying exact location pins directly within the system, this setup minimized manual routing errors, prevented riders from getting lost in unfamiliar locations, sped up fulfillment windows, and protected fragile custom cakes from structural damage or melting due to prolonged heat exposure.

1.5.4 The Student Developers

For the authors of this capstone project, the study served as a direct bridge between classroom theories and real-world software engineering.

- **Technical Skill Application:** This project provided the developers with hands-on experience in full-stack web development, Python backend coding, and NoSQL database management using Google Firestore. It challenged them to build a real database structure that handled complex, customizable items.
- **Professional Problem Solving:** It gave the team invaluable experience in analyzing broken, manual business workflows for a local Philippine micro-enterprise (MSME) and designing a real-world software solution that solved actual financial, operational, and logistical bottlenecks.

1.5.5 Future IT Researchers and Developers

This manuscript and its documented system design served as a valuable reference blueprint for future Information Technology students and software developers.

- **Academic Referencing:** Future developers tasked with building similar systems could use this study to understand how to handle complex, multi-tiered product configurators, dynamic pricing structures, and localized payment gateway webhooks.
- **System Design Guidance:** The detailed workflow diagrams, database structures, and methodology steps written in this paper provided a solid technical guide for anyone developing integrated e-commerce and point-of-sale (POS) systems for specialized, inventory-dependent food businesses or small retail enterprises in the Philippines.

1.5.6 The Local Community and Economy

The digital transformation of Mrs. Brave's Cakes contributes to the broader economic development of Iloilo City and the Philippine MSME sector. By demonstrating that a small, home-based bakery can successfully adopt cloud-based tools, this study provides a replicable model for other local food enterprises. A more efficient bakery means reduced waste, better utilization of local ingredients, and the potential to hire additional staff during peak seasons. Furthermore, by keeping delivery coordination in-house rather than outsourcing to third-party apps, the system helps retain revenue within the local community.

1.5.7 The Philippine Food and Bakery Industry

The Philippine bakery industry is dominated by micro and small enterprises that often lack access to affordable, tailored software. This study contributes to the industry by documenting the specific requirements of a custom cake bakery—variable pricing, perishable inventory, fragile delivery—and demonstrating a cost-effective solution. The system's use of PayMongo for local e-wallets and OpenStreetMap for free mapping sets a precedent for other MSMEs that cannot afford expensive enterprise resource planning (ERP) systems or proprietary delivery APIs.

1.5.8 Environmental Impact

By enabling precise inventory tracking and automated deductions, the system helps reduce food waste—a significant problem in perishable goods businesses. When ingredients are tracked accurately, over-purchasing is minimized, and spoilage is reduced. Additionally, digital receipts and paperless order tracking reduce the bakery's reliance on paper logbooks and printed receipts, contributing to a smaller environmental footprint.

1.6 Scope and Delimitation

To clearly define the boundaries of the capstone project, this section outlined the exact features built into the system (Scope) and the specific technical constraints left out of the project (Delimitation).

1.6.1 Scope of the System

The project was a unified, web-based management platform specifically designed for Mrs. Brave's Cakes. It was engineered using a responsive web frontend (accessible on computers and smartphones), a Python backend framework to run the main software logic, and Google Firestore as a NoSQL cloud database for live data saving. The system was structurally divided into three primary front-facing interfaces:

1. The Customer Interface

This frontend portal allowed the general public and patrons of the bakery to view the shop digitally. Its sub-features included:

- **Digital Product Catalog:** A clean display screen showing pre-made cakes, prices, and descriptions, equipped with a client-side filtering system for rapid product discovery.
- **Interactive 3D Cake Customization and Consultation Module:** A step-by-step customization portal where users could select cake attributes (tiers, sizes, shapes, flavors, frosting, candles, and toppers) paired with a responsive, draggable **3D Canvas Live Preview powered by Three.js (WebGL)**. Running client-side directly within the web browser, this interactive Three.js engine dynamically rendered 3D cake meshes (.gltf/.glb formats) in real-time, allowing users to orbit, rotate, zoom, and visually inspect their custom design from all 360-degree angles. The engine dynamically applied material texture mappings for frosting variations and procedural color parameters based on user selections. This module fed directly into a dedicated cake consultation portal that consolidated all customer-selected configurations, special allergy/handling instructions, and attached design reference images, accurately preserving the complete structural profile upon order conversion.
- **Dynamic Pricing and Distance-Based Fee Calculator:** A script that instantly computed and displayed the updated price as cake options were selected, while automatically computing delivery fees based on real-time kilometer-distance calculations.
- **Promotions, Vouchers, and Automated Free Gifts Engine:** A dynamic perk system that evaluated checkout conditions (such as the first 5 orders or dynamic 10-order targets) to award free gifts and vouchers. This system updated the customer dashboard and invoices instantly without automatically inflating or interfering with accumulated loyalty stamps.
- **Automated Loyalty Card Management:** A digital loyalty stamp card that tracked customer purchases. It featured an automated roll-over algorithm: upon reaching the 10th order milestone, the card reset to 0 upon claim reward

validation, while carrying over any excess stamps (e.g., maintaining a remaining balance of 2 stamps if a user claimed their reward at a 12/10 stamp state).

- **PayMongo Checkout and Downpayment Integration:** A secure online payment pipeline that processes transactions via GCash and Maya e-wallets, protected by an integrated CAPTCHA framework to block automated bot submissions and spam. The interface enforced downpayment rules, requiring secure payment confirmation codes (Reference IDs) to log downpayment states.

- **Interactive Order Cancellation Module:** A standard feature allowing customers to submit an order cancellation request directly from their dashboard, mandating that a valid text justification reason be submitted before processing.

- **Compliance and Information Module:** Dedicated frontend views accessible to users, including an "About Us" section that rendered the business history of Mrs. Brave's Cakes to increase brand credibility, alongside standard Terms of Service and Privacy Policy blocks embedded within the checkout layout to ensure data privacy transparency.

- **Gemini AI-Powered Chatbot Assistant:** An intelligent, conversational chat interface powered by the Gemini AI model via API, allowing customers to receive immediate, automated text responses regarding store policies, operating hours, and basic ordering FAQs, with session initiation capabilities available to the shop owner.

- **Digital Invoice System:** Generated downloadable PDF e-receipts on-screen containing exact transaction details, verified proof of payments, and remaining balances, while automatically dispatching delivery copies to the user's Gmail.

- **Biometric Authentication Interface (WebAuthn API):** A secure, passwordless authentication module that allowed registered users to log into their client profiles using their smartphone or computer's native fingerprint scanner, providing cryptographic security that reduced credential-stuffing risks.

- **Real-Time Order & Delivery Status Tracker:** A live order tracking interface where customers can monitor their cake order through five fulfillment stages: *"Pending," "Baking," "Decorating," "Ready for Pickup,"* and *"Out for Delivery."* The interface renders an interactive map using **Leaflet.js** integrated with **Geoapify API services** and **OpenStreetMap** tile data. When an order is dispatched, the map displays the designated delivery destination, route trajectory, estimated delivery fee, and real-time status updates synced via Firebase Firestore.

2. The Administrator Dashboard

This secure back-office interface was restricted exclusively to the bakery owner, Mrs. Shamelyn Tapang. Its sub-features included:

- **On-Site Point of Sale (POS) Module:** A quick checkout interface for entering offline cash sales when walk-in customers bought products at the physical store counter.

- **Thermal Printer Compatibility:** Built-in code settings that allowed the admin to print physical transaction receipts immediately using standard connected thermal receipt rollers.

- **Dual-State Production Calendar:** A visual calendar where the admin could click to lock or unlock specific dates, completely blocking incoming orders for custom cakes, pre-made items, or both, while displaying a custom note (e.g., "Fully Booked") on the storefront.

- **Manual Inventory Control Ledger:** A database screen where the owner could view and manually adjust ingredient balances (flour, sugar, eggs) and manage quantities of available pre-made cakes.

- **Automated Material Subtraction:** A backend script that automatically calculated and subtracted the required baking materials from the inventory stock whenever an order was marked as paid.

- **Visual Business Analytics Dashboard:** Automatically processed raw sales and expense data into easy-to-read bar graphs and pie charts showing total gross revenue, operational costs, and true net profit margins.

- **Historical Consumption Analytics Feature:** A Python-based analytics module that aggregates historical order data and completed transactions into weekly, monthly, and all-time sales/expense summaries, peak order-day

identification, and top-customer breakdowns — giving the administrator a clear, data-driven view of past consumption trends to inform manual inventory planning.

3. The Rider Dispatch Portal

This lightweight, simple interface was built for the independent motorcycle couriers hired by the bakeshop. Its sub-features included:

- **Decentralized Access Web-Link:** A unique, secure web URL link generated by the system that the admin could text to a rider. The link opened instantly in the rider's phone browser without requiring them to create an account or log in.
- **Static Deliveries Manifest Screen:** Displayed a basic, clean mobile layout containing the customer's name, phone number, delivery time, and cake details.
- **Delivery & Location Component:** A web-based mapping module powered by **Geoapify API** and **Leaflet.js** that processes customer delivery addresses into map coordinates and calculates delivery routes. The component securely syncs delivery status updates and location markers to the Firebase backend, updating the customer's interactive tracking map frame dynamically without requiring external native mobile applications or continuous device tracking.
- **Fulfillment Status Toggle:** A simple tracking button that the rider clicked to change an order status from "On the Way" to "Delivered," which updated the admin dashboard instantly.

1.6.2 System Functional Boundaries and Limits

To ensure that the software remained realistic to build within the school term and stayed effectively aligned with the needs of a local micro-business, the system was strictly bounded by a System Boundary Matrix. Table 1.1 lays out the exact operational features included inside the scope versus the technical boundaries that were explicitly delimited.

Table 1.1: Functional Scope and Delimitation Matrix Table

Core System Module	Features INCLUDED in the Scope	Features DELIMITED (Excluded) from the Scope
Ordering & Design	Step-by-step selection of cake attributes (tiers, shapes, sizes, frosting types, add-ons). Real-time, interactive, and draggable 3D cake design preview for instant spatial visualization. Uploading design reference images. Integrated Gemini AI chatbot assistant to handle customer FAQs, store policies, and order queries. Dedicated cake consultation module that preserved design details and attached images upon layout conversion.	The system did not generate automated baking blueprints, structural stability load-bearing tests, or 3D printable asset exports for the kitchen staff.

Pricing Module	Live calculation of custom quotes based on chosen attributes, rush-order fees, and dynamic kilometer-distance-based delivery fees. Dedicated application of statutory Senior Citizen and PWD discounts within the checkout workflows.	No automatic dynamic inflation pricing based on external market data feeds was implemented. Base ingredient costs had to be updated manually by the admin.
Scheduling Module	Shared calendar with date locking, capacity tracking limits, and custom closure reason banners (e.g., "Fully Booked").	No multi-branch schedule routing was included. The system only tracked the capacity of the single main Mrs. Brave's Cakes kitchen.
Walk-In Shop POS	Counter checkout screen for cash sales, tracking pre-made stock, and sending thermal print commands. Passwordless biometric fingerprint login powered by the WebAuthn API for secure, instantaneous user authentication.	No hardware barcode scanning was supported. All shop entries were mouse/touch-driven.
Inventory System	Digital manual stock tracking, automated ingredient-level subtraction upon paid orders, and raw supply edits.	No Internet of Things (IoT) hardware scales, smart pantry shelf sensors, or automated warehouse re-ordering tools were integrated.

Business Analytics	Automated visualization of sales, expenses, and profits. Rule-based demand estimation (using Python aggregation logic) that calculates projected ingredient needs for the next cycle by analyzing historical usage patterns and current stock levels.	No automated tax filing systems or integration with local government accounting platforms (such as the BIR) were included. Reports were generated for internal business use only.
Payment Gateway	PayMongo API checkout integration supporting digital GCash and Maya wallets, plus cash alternatives. Secure confirmation code (Reference ID) capture for verifying downpayments, protected by CAPTCHA security against automated bot spam. Included dynamic Free Gift and Voucher rules alongside an automated Loyalty Stamp card with roll-over stamp balances.	No credit card data storage was implemented. The platform did not store, save, or handle raw credit card numbers to maintain strict data security standards.
Rider Delivery Link & Tracking	The Web Link Generation and Dispatch Notification module provides an in-app mapping framework powered by Leaflet.js, OpenStreetMap, and Geoapify API services to convert customer delivery addresses into precise map coordinates through geocoding and to calculate optimized delivery routes. When an order is dispatched, the system automatically generates a secure tracking link and sends an SMS notification directly to the customer. This link opens the Customer Order Tracker page, which displays a live interactive map featuring calculated route details, marked delivery drop-off points, and real-time order status updates synchronized through Firebase Firestore.	No direct API links to external global shipping or third-party courier aggregates (e.g., Lalamove or GrabExpress) were built. The portal ran exclusively for independent, internal drivers.

1.6.3 Technical Delimitations

To prevent any confusion regarding the software boundaries during the project defense, the four primary exclusions listed in the matrix were explained below in detail:

1. Client-Side 3D Hardware & WebGL Requirements (No 3D Asset Export or Blueprints)

The Customer Interface included an interactive, draggable **Three.js** 3D Canvas Live Preview that rendered real-time visual configurations of custom cake shapes, tiers, frosting, and attributes for spatial visualization. Because Three.js relies on WebGL for GPU-accelerated canvas rendering inside the browser, the interactive viewer required a modern, WebGL-compliant web browser (such as Google Chrome, Safari, or Microsoft Edge) with basic graphics acceleration enabled on the user's host device. However, the system architecture did not generate automated baking blueprints, structural load-bearing mass calculations, or structural integrity assessments for the kitchen staff. Furthermore, the platform did not compile, render, or export the interactive Three.js scenes into standardized, manufacturing-ready 3D asset files (such as .STL or .OBJ formats) for external 3D printing software.

2. No Automated Hardware, IoT, or Native Device Driver Integrations

The inventory module functioned strictly as a data ledger inside the software environment, tracking material stocks via manual dashboard entries and automated backend subtraction scripts. The system architecture did not natively establish low-level communication channels with physical Internet of Things (IoT) hardware scales, automated pantry shelf sensors, or smart refrigeration nodes. Furthermore, while the Administrator Dashboard provided localized thermal receipt printing compatibility to utilize the owner's existing physical thermal POS printer, this integration was executed entirely through standard web-browser print layouts; the application did not include native machine drivers, custom firmware wrappers, or proprietary hardware communication layers to directly manipulate local printing hardware peripherals.

3. Exclusively Web-Based Application Architecture (No Native Apps)

The entire management system was designed and compiled strictly as a responsive web application. The application ran smoothly inside standard internet browsers (such as Google Chrome, Safari, and Microsoft Edge) across desktop computers, laptops, tablets, and smartphones. The system was not compiled, packaged, or written as a native mobile application (such as an Android APK or an iOS IPA file), and it was not uploaded or distributed through public mobile market platforms like the Google Play Store or Apple App Store.

4. No Third-Party Delivery Fleet API Connections

The system provided an internal dispatch module to coordinate neighborhood delivery riders manually managed by Mrs. Brave's Cakes. While the platform integrated the **Geoapify API** to process address geocoding, calculate delivery route distances, and render interactive mapping layers for order tracking, it operated completely independently of commercial logistics networks. The software did not integrate with the automated corporate backend APIs of third-party on-demand delivery services (such as GrabExpress, Lalamove, or Borzo). The system did not automatically dispatch commercial third-party riders or synchronize corporate delivery account balances; all delivery assignments had to be initiated manually by the administrator.

1.7 Definition of Terms

To ensure complete clarity and a uniform understanding of the concepts used throughout this project, the following terms were defined operationally based on how they functioned within this study:

- Add-ons: Operational: Extra customizable cake decorations—such as edible prints, fondant characters, or custom toppers—selected by the customer on the website that instantly increased the calculated total price of the cake.
- Administrator: Operational: Referred directly to the bakery owner, Mrs. Shamelyn Tapang, who possessed full secure access to the back-office dashboard to handle ordering, change pricing, view financial charts, update stocks, print receipts, and create delivery links.
- Admin Dashboard: Operational: The secure back-office interface where the bakery owner manages orders, inventory, bookings, and analytics.

  Agile Development: Operational: An iterative software development methodology used in this project, characterized by short sprints, continuous feedback, and adaptive planning.

- API (Application Programming Interface): Operational: In this study, it referred specifically to the PayMongo API bridge, which was built into the checkout page to securely connect the bakery system with local digital wallets like GCash and Maya.
- As-Is Process: Operational: The existing manual workflow of the bakery before the system was implemented, documented through interviews and direct observation.
- Blackout Dates: Operational: Specific calendar dates that were manually locked or disabled by the owner through her dashboard. Once locked, it prevented customers from placing online orders for those days, keeping the kitchen from getting overbooked.
- Cake Customization Module: Operational: The step-by-step online choice form on the customer's website that allowed users to click through and choose their cake tiers, size, icing, flavors, and design features comfortably.
- Cake Quote Calculator: Operational: The frontend script that dynamically computes the total price of a custom cake based on selected attributes, eliminating manual arithmetic.
- Cash on Delivery (COD): Operational: A payment method where the customer pays in cash upon receiving the order, used as a fallback when online payment is not chosen.
- Cloud Database: Operational: A digital storage space hosted on the internet rather than on a physical office computer. It instantly saved and synced all customer entries, store transactions, and stock logs so they could be viewed securely from any device.
- Cloudinary: Operational: A cloud-based media management service used to store, optimize, and deliver cake reference images uploaded by customers and administrators.
- Data Informatics Dashboard: Operational: The main visual screen on the admin dashboard that automatically processed raw business sales and expenses into easy-to-read graphs and charts to help the owner see her business trends.
- Data Latency: Operational: The slow delay or lag that occurred when information took too long to be updated or sent between different channels, which this system solved through real-time cloud tracking.
- Decentralized Portal: Operational: A very simple, standalone web interface built specifically for a single user type (like the delivery riders) that worked instantly through a private link without requiring an account login.
- Dual-State Calendar: Operational: A calendar management feature that allowed the owner to switch a date's availability between two simple choices: Locked (Closed) or Unlocked (Open) for incoming shop orders.
- Dynamic Pricing: Operational: A background software script that instantly recalculated and updated the total price on the screen as a customer clicked different tiers or premium cake design features.
- E-Receipt: Operational: The electronic, professional PDF invoice automatically generated by the system after a successful digital transaction, which the customer could download on the site or receive inside their private Gmail inbox.
- FCM (Firebase Cloud Messaging): Operational: Google's push notification service used to send real-time alerts to the admin dashboard when a rider marks an order as delivered.
- Financial Blindness: Operational: The dangerous business state of not knowing true shop profits, revenue leaks, or ingredient costs because records were completely manual, written down on separate pieces of paper, or left completely unrecorded.
- Firebase Authentication: Operational: Google's identity management service used for secure user login and session control, supporting Email/Password, Google OAuth, and WebAuthn biometric authentication.
- Firestore: Operational: Google's NoSQL document database used as the centralized data store for all system collections, enabling real-time synchronization via WebSockets.
- Gemini AI Chatbot: Operational: An intelligent conversational assistant powered by the Gemini 2.5 Flash-Lite model,

integrated into the customer interface to automatically answer frequently asked questions about store policies, operating hours, and order tracking.

- ISO/IEC 25010: Operational: An international software quality standard used to evaluate the system across functional suitability, usability, security, and performance efficiency.
- Inventory Monitoring: Operational: The digital ledger screen within the admin dashboard used by the owner to track, subtract, and adjust stock counts of raw materials (flour, sugar, eggs) and pre-made store cakes, completely replacing paper logbooks.
- Leaflet.js: Operational: An open-source JavaScript library used to render interactive maps in the rider portal and customer order tracking page.
- Loyalty Stamp Card: Operational: A digital reward system that tracks customer purchases and awards free gifts after a set number of orders, with an automated roll-over algorithm that carries excess stamps after reward redemption.
- NoSQL Architecture: Operational: A modern database framework that organized data as flexible text sheets or files instead of rigid table rows and columns, making it easy to store varying custom cake details that changed from order to order.
- OpenStreetMap (OSM): Operational: A free, editable map of the world used as the base layer for the rider portal's WebGIS interface and customer delivery tracking.
- PayMongo: Operational: A Philippine payment gateway that processes GCash, Maya, and card transactions, integrated into the system's checkout pipeline to automate payment verification via webhooks.
- Point of Sale (POS): Operational: The dedicated checkout screen within the admin dashboard used to quickly record face-to-face cash sales when walk-in clients bought products at the physical shop counter, designed to print receipts instantly using thermal rollers.
- Python Backend Framework: Operational: The hidden main programming engine running on the web server that processed all business logic, handled calculations, communicated with the database, and connected the system with external email and payment servers.
- RBAC (Role-Based Access Control): Operational: A security model that restricts system access based on user roles (Admin vs. Customer), ensuring that only authorized users can access administrative functions.
- Resend API: Operational: A transactional email service used to dispatch automated e-receipts to customers after successful payment verification.
- Responsive Web Application: Operational: A website engineered to automatically change its size, shape, and layout to fit seamlessly on any screen, whether a user opened it on a desktop computer or a mobile smartphone.
- Rider Dispatch Interface (Rider Portal): Operational: The lightweight web environment opened via a text message link by external motorcycle couriers to view an assigned customer's name, phone number, static map delivery coordinates, and update the completion status.
- Static Coordinates: Operational: The fixed geographical map numbers (latitude and longitude) generated from the address text pinned by a customer during checkout, which were passed to mobile map apps to show a rider exactly where to drive.
- System Testing: Operational: The phase of testing that evaluated the fully integrated system against end-to-end business workflows, ensuring all modules communicated correctly.
- Thermal Printer Compatibility: Operational: The software code instructions that allowed the admin dashboard to send instant print commands to small, inkless thermal checkout machines to print physical receipts for walk-in store clients.
- Three.js: Operational: An open-source JavaScript library used to render the interactive 3D cake preview within the customer customization module, powered by WebGL.
- To-Be Process: Operational: The proposed automated workflow enabled by the system, designed to replace the manual

As-Is process.

- UAT (User Acceptance Testing): Operational: The final testing phase where real users (the business owner and selected customers) evaluated the system's usability and functionality using an ISO/IEC 25010-based questionnaire.
- Unit Testing: Operational: The testing of individual software components in isolation to verify that each function produced the expected output.
- Unstructured Data: Operational: Scattered, unorganized pieces of information that did not follow a set structure—such as casual chats, random images, or long text threads inside Facebook Messenger logs—making it difficult to track orders systematically.
- WebAuthn: Operational: A web standard for passwordless, biometric authentication (fingerprint or Face ID) that allowed registered users to log in securely using their device's native hardware.
- WebGIS: Operational: A geographic information system delivered over the web, used in the rider portal to display interactive maps and route calculations without requiring a native mobile app.
- Webhook Listener: Operational: A background server script that waited for an automatic confirmation signal from the PayMongo network, instantly switching an order's status to "Paid" in the database the moment a customer finished their GCash or Maya transfer.

CHAPTER 2

REVIEW OF RELATED LITERATURE AND STUDIES

This chapter provides a comprehensive review of various literatures, studies, and existing systems—both local and foreign—that are relevant to the development of the "**Streamlined Bakery Management: A Dedicated System for Cake Customization and Sales Monitoring**" for Mrs. Brave's Cakes. The developers gathered information from academic journals, books, and existing software to provide a theoretical and technical foundation for the study.

2.1 Foreign Literature and Studies

2.1.1 Digital Transformation in Micro-Enterprises

According to Verhoef et al. [1], the modern retail sector underwent a profound digital transformation driven by evolving consumer expectations and technological innovation. As consumer behaviors shifted toward digital touchpoints, traditional and informal retail micro-enterprises were increasingly compelled to integrate structured online platforms into their existing, manual operations to achieve long-term viability.

Verhoef et al. [1] emphasized that moving toward a dedicated digital infrastructure enabled businesses to transition from fragmented, informal workflows into an agile, strategic operation that directly aligned technological capabilities with organizational needs. This shift allowed small businesses to run smoothly without depending on unorganized, disconnected communication channels that isolated critical transactional data. By adopting centralized web platforms, micro-enterprises scaled up their operations, optimized resource allocation, and ensured that transaction histories were systematically captured and securely preserved over extended operational periods.

- **Relevance to the Current Study:** For Mrs. Brave's Cakes, migrating from informal Facebook messaging chats to a structured, centralized web platform directly reflected the digital transformation framework outlined by Verhoef et al. [1], ensuring a formalized, error-reduced, and sustainable long-term operation.

2.1.2 Human Oversight in Perishable Inventory Management

According to Wallace and Stark [2], automated systems for managing perishable goods frequently suffer from predictive inaccuracies due to uncontrolled environmental variables such as temperature fluctuations and handling inconsistencies. Their critique demonstrated that rigid algorithmic models often failed to account for real-world deviations, leading to either premature disposal of viable stock or the sale of degraded products. Consequently, they argued that effective inventory management for perishables requires a hybrid approach where digital tracking is supplemented by active human verification to ensure quality assurance and reduce waste.

For small artisanal food enterprises like bakeries, this principle is critical because ingredients like cream, eggs, and fondant are highly sensitive to storage conditions. A system that relies solely on automated deduction may overlook spoilage events or substitution needs. By retaining administrative control over inventory validation, businesses can bridge the gap between theoretical stock levels and actual usable assets.

- **Relevance to the Current Study:** This supported the implementation of an Admin-Led Inventory Model for Mrs. Brave's Cakes. Instead of adopting a fully hands-off, automated model, implementing an admin-guided dashboard aligned directly with the conclusions of Wallace and Stark [2]—enabling Mrs. Tapang to retain administrative oversight over unstandardized kitchen variables while simultaneously benefiting from structured digital data organization.

2.1.3 User Interface (UI) Design for Product Customization

Lowry et al. [3] emphasized that effective user interface design played a major role in improving customer trust, reducing confusion, and enhancing online transaction completion rates in e-commerce environments. The developers explained that cluttered and visually overwhelming interfaces increased users' cognitive workload, making decision-making more difficult during digital purchasing activities. Structured layouts and guided workflows helped users complete complex product customization processes more efficiently.

- **Relevance to the Current Study:** This study supported the interface structure of the cake customization platform for Mrs. Brave's Cakes. By organizing customization options into a clear and guided workflow, the system minimized customer confusion when selecting cake sizes, flavors, fillings, and decorative themes.

2.1.4 Real-Time Inventory Control and Operational Accuracy

DeHoratius and Raman [4] established that inventory record inaccuracy posed significant financial and operational risks to retail enterprises. Their empirical analysis demonstrated that discrepancies between physical inventory and system records frequently stemmed from manual logging errors and delayed transaction synchronization. By transitioning to centralized inventory tracking frameworks, organizations mitigated stock discrepancies and improved demand fulfillment accuracy.

Maintaining real-time synchronization between order entries and stock logs prevented physical stock depletion and eliminated manual recording friction. When inventory changes were updated instantly across an enterprise dashboard, business owners

gained complete operational visibility, preventing overcommitment of limited stock items.

- **Relevance to the Current Study:** The inventory discrepancies previously experienced by Mrs. Brave's Cakes were directly addressed by the real-time stock synchronization principles validated by DeHoratius and Raman [4]. Instant cloud synchronization ensured that whenever a cake order was confirmed or ingredients were deducted, the admin panel reflected the exact stock levels, preventing ingredient shortages during production.

2.1.5 Mitigation of Scheduling Conflicts and Operational Latency

Bard and Morton [5] investigated capacity management and operational scheduling under stochastic operational constraints. Their study demonstrated that relying on uncoordinated manual scheduling systems introduced severe operational latency, causing capacity overbooking and resource misallocation. By deploying structured mathematical constraints and centralized scheduling software, organizations systematically eliminated booking overlaps and optimized operational throughput.

Eliminating data latency was identified as a crucial requirement for service-oriented operations where processing capacity was constrained by time and labor resources. Automated scheduling verification prevented double-booking risks and provided managers with immediate visibility over daily operational capacity.

- **Relevance to the Current Study:** For Mrs. Brave's Cakes, the scheduling conflicts and double-bookings identified in manual logbooks were a direct result of operational data latency. Implementing the automated booking constraint framework discussed by Bard and Morton [5] ensured that Mrs. Tapang's digital order calendar automatically validated capacity limits, preventing double-booked reservation dates.

2.1.6 Cognitive Load and Decision-Making in Complex Customization Interfaces

Lowry et al. [3] further explained that users experienced cognitive overload when digital platforms presented excessive information simultaneously or utilized disorganized navigation structures. The study highlighted that segmenting user tasks into smaller, guided steps improved user confidence, reduced frustration, and increased task completion rates within e-commerce systems. Simplified workflows helped customers make decisions more comfortably during online transactions.

- **Relevance to the Current Study:** This principle supported the implementation of the Cake Customization Module within the proposed system. Instead of displaying all customization options at once, the platform organized the ordering process into multiple guided stages to reduce customer confusion and improve order accuracy.

2.1.7 Security and Trust in E-Commerce Transaction Interfaces

According to Lowry et al. [3], establishing robust security features and clear data protection mechanisms was critical for building consumer trust and minimizing transactional risks in online environments. For smaller web platforms, implementing verified, standardized transactional interfaces reduced the threat of data vulnerabilities and deceptive actions.

The study highlighted that relying on well-established, transparent e-commerce frameworks helped businesses secure electronic transactions, safeguard sensitive information, and prevent the operational vulnerabilities associated with informal or manual verification procedures. Unifying payment verification into a secure gateway removed the heavy risk of human error during cash checks and eliminated manual transaction fraud.

- **Relevance to the Current Study:** This research directly validated the integration of secure payment mechanisms for Mrs. Brave's Cakes. By automating the payment verification process through structured transaction gateways rather than relying on manual GCash screenshot confirmations, the system aligned with the trust and security principles of Lowry et al. [3], shielding the business owner from fraudulent payment slips.

2.1.8 Cloud-Integrated Point of Sale (POS) Frameworks for Retail Operations

Kim and Park [6] evaluated consumer and business acceptance of cloud-based point-of-sale systems using the Technology Acceptance Model (TAM). Their empirical findings revealed that system compatibility, real-time data synchronization, and user interface simplicity were the primary drivers of user acceptance in retail environments. Connecting frontend sales counters with cloud databases eliminated data fragmentation and ensured unified accounting across sales channels.

An integrated retail platform that connected frontend consumer interactions with localized point-of-sale systems allowed a business to eliminate isolated data silos. This ensured that over-the-counter walk-in transactions and online e-commerce orders operated under a single, synchronized database ledger.

- **Relevance to the Current Study:** This concept directly supported the inclusion of the walk-in POS module within the admin dashboard for Mrs. Brave's Cakes. By linking physical counter sales with online cake records into a single cloud database as advocated by Kim and Park [6], Mrs. Tapang avoided data fragmentation and gained total visibility over her combined business revenue.

2.1.9 Lightweight Dispatch Interfaces for Last-Mile Logistics

Chen et al. [7] explained that inefficient manual dispatch coordination contributed to routing delays and communication gaps in on-demand food delivery systems. Their study proposed intelligent dispatch optimization techniques capable of improving route efficiency and enhancing delivery coordination. Similarly, Lu, Liu, and Bi [8] discussed how structured routing systems and precise geographic data improved delivery performance and reduced navigation-related inefficiencies during last-mile logistics operations.

- **Relevance to the Current Study:** These studies supported the development of the Rider Dispatch Portal for Mrs. Brave's Cakes. By automatically generating structured location coordinates and integrating lightweight, rider-accessible delivery pages, the system minimized manual address handling and improved delivery efficiency for independent couriers.

2.1.10 NoSQL Document-Oriented Database Architecture for Flexible Product Configurations

According to a landmark study by Han et al. [9], traditional relational databases utilizing rigid tables, rows, and columns struggled significantly when handling highly customizable product structures. The developers established that for systems featuring items with unpredictable and shifting variables—such as custom sizes, varying flavors, multiple tiers, and decorative accessories—a NoSQL document-oriented database framework was vastly superior.

Han et al. [9] demonstrated that NoSQL databases allowed data to be stored as flexible, independent document objects that did not force empty database fields or break system stability when an item's design attributes varied from order to order. This structural agility drastically lowered server query delays, prevented data crashes, and ensured that the system could handle complex customer data modifications in real time without requiring massive database restructuring.

- **Relevance to the Current Study:** This database engineering principle directly validated the selection of Google Firestore as the NoSQL database for Mrs. Brave's Cakes. Because custom cake orders contained unpredictable design attributes that changed completely from one client to another, utilizing the NoSQL document structure supported by Han et al. [9] ensured that the backend saved varying data cleanly without database lag or rigid layout errors.

2.1.11 Predictive Data Modeling for Stock Demand and Material Forecasting

In the field of supply chain automation, Carbonneau, Laframboise, and Vahidov [10] evaluated the application of advanced predictive modeling—including machine learning—to forecast future stock demands for small business networks. Their research highlighted that traditional, human-led inventory practices were often purely reactive, failing to account for hidden purchasing trends, seasonal surges, or bulk acquisition opportunities, which frequently resulted in costly stockouts or material waste.

By implementing predictive data models, business systems automatically evaluated historical monthly sales data to spot repeating shopping trends, holiday rushes, and seasonal event surges. Carbonneau et al. [10] proved that predictive algorithms provided accurate material demand forecasts, allowing small-scale operators to acquire raw ingredients in advance at bulk wholesale prices rather than reacting to sudden stock shortages.

- **Relevance to the Current Study:** While Carbonneau et al. [10] utilized complex machine learning architectures, this study adapted their core principle—proactive rather than reactive inventory management—through a lightweight, rule-based approach suitable for a micro-enterprise scale. The developed system implements a statistical forecasting module in Python that aggregates historical order logs to estimate future ingredient requirements (such as flour, sugar, and eggs). By deriving demand projections from deterministic data processing rather than autonomous model training, the system effectively reduces the risk of mid-production shortages while maintaining operational simplicity and cost-efficiency for Mrs. Brave's Cakes.

2.1.12 Point of Sale (POS) Operational Usability and Data Integrity in Retail Counters

Sharafat and Khan [11] examined the operational usability and data integrity of Point of Sale (POS) systems across retail counters. The developers found that physical checkout counters relying on manual calculations or offline cash drawers suffered from high financial risk, transaction leakage, and unrecorded sales metrics.

Sharafat and Khan [11] highlighted that a successful modern retail platform had to provide an integrated storefront POS screen that shared a singular database with the online e-commerce site. When over-the-counter sales were processed through a structured digital checkout ledger, human calculation errors significantly decreased, receipt documentation was managed cleanly, and management gained complete financial visibility over combined business earnings.

- **Relevance to the Current Study:** This concept directly supported the construction of the Walk-In POS Module within the admin dashboard for Mrs. Brave's Cakes. Following Sharafat and Khan's [11] findings, integrating the physical shop counter with the online database ensured that face-to-face cash sales were safely logged beside online orders, protecting the bakery from revenue gaps.

2.1.13 Automated Electronic Invoicing and Transaction Verification in E-Commerce Channels

Establishing immediate transaction transparency was identified as a core requirement for building long-term digital customer loyalty. Gwebu and Wang [12] analyzed how automated digital document generation affected consumer trust during online transactions. The study demonstrated that when a system instantly generated a downloadable e-receipt and dispatched an automated email invoice copy immediately after a transaction, customer anxiety regarding payment safety was minimized.

Gwebu and Wang [12] pointed out that automated digital receipt systems served a dual purpose: they provided customers with instant legal proof of purchase and simultaneously protected the business by creating an unalterable, automated data record that could not be edited or faked by malicious users.

- **Relevance to the Current Study:** This research directly validated the automated Secure Payment and Receipting System built for Mrs. Brave's Cakes. By automatically generating PDF e-receipts and sending a permanent duplicate directly to the customer's email via the Python backend, the system reduced the risks of manual receipt handling and screenshot fraud as emphasized by Gwebu and Wang [12].

2.1.14 Comparative Performance Optimizations of NoSQL Storage in E-Commerce Environments

According to Faraz and Chausson [13], the explosive growth of web-based retail exposed structural bottlenecks in traditional relational databases. When an e-commerce platform allowed users to freely construct heavily customized, non-standard products, relational database engines suffered from massive table join complexities, causing high server latency and transaction timeout errors.

Faraz and Chausson [13] demonstrated that transitioning to a dynamic NoSQL schema avoided database normalization traps by allowing structured multi-attribute payloads to sit cleanly within single-document objects. This layout ensured fast, direct lookups, allowing the platform to dynamically render variable pricing and custom product configurations without causing visual stutter or server lag during heavy seasonal customer traffic spikes.

- **Relevance to the Current Study:** This performance analysis directly supported the NoSQL database setup chosen for Mrs. Brave's Cakes. Because customers could build unique cake profiles with changing variables (tiers, cake flavors, custom message scripts, and accessories), Faraz and Chausson's [13] framework guaranteed that the backend handled varying document attributes instantly without dropping operational speeds.

2.1.15 Web-Based Point of Sale Architecture and Inventory Synchronization

Soegoto and Palalungan [14] explored the engineering and implementation of web-based inventory information systems for commercial management. Their study demonstrated that manual register logging isolated crucial store data from executive decision-making, resulting in severe data blind spots and inventory leaks when over-the-counter sales operated separately from digital systems.

Soegoto and Palalungan [14] proved that building a lightweight, web-based Point of Sale console that shared a single master data layer with backend inventory tables provided high system usability. This unified ledger approach automatically reduced manual transaction entry workloads, standardized daily counter tallies, and provided micro-retailers with real-time visibility over total stock movements and revenue.

- **Relevance to the Current Study:** This study directly validated the technical design of the Walk-In Counter POS Module inside the admin console for Mrs. Brave's Cakes. Following the web-based architecture evaluated by Soegoto and Palalungan [14], the system connected over-the-counter walk-in transactions to the exact same cloud inventory values used by online cake customizers, protecting Mrs. Tapang from unrecorded sales gaps.

2.1.16 Multi-Objective Inventory Optimization for Perishable Food Systems

According to a production study by Qiu et al. [15], scheduling production and managing inventory for perishable materials introduced intense operational challenges. Relational tracking methods failed to account for shelf-life degradation, forcing operators into reactive, high-waste cycles. The authors proved that deploying coordinated replenishment policies reduced material waste by establishing rigid algorithmic boundaries before raw stock degraded.

- **Relevance to the Current Study:** This framework supported the predictive inventory sub-module of the system. By mapping ingredient decay properties within the admin panel, Mrs. Tapang could systematically track her raw materials (such as milk, cream, and eggs), minimizing kitchen waste and cutting bulk ingredient acquisition expenses.

2.1.17 Data-Driven Inventory and Booking Prioritization Architectures

Qiu et al. [15] emphasized the importance of integrating inventory monitoring, delivery coordination, and replenishment planning within food-related operations handling perishable products. Their study demonstrated that data-driven inventory systems improved forecasting accuracy, reduced material shortages, and optimized operational resource allocation. Automated inventory planning mechanisms helped businesses respond more efficiently to fluctuating product demand and production requirements.

- **Relevance to the Current Study:** This study supported the integration of customer booking records and ingredient consumption tracking within the proposed system for Mrs. Brave's Cakes. By utilizing historical order data to estimate material usage and monitor stock availability, the platform helped reduce ingredient shortages and improved production planning during high-demand periods.

2.1.18 Algorithmic Dynamic Pricing Engineering in Digital Commerce

According to an e-commerce study by Enache [16], traditional fixed-retail pricing models failed to preserve profit margins when raw production inputs fluctuated. Enache [16] established that software platforms should incorporate dynamic price modeling to assess real-time production variables, labor times, and variable component cost tiers to guarantee stable financial conversions.

- **Relevance to the Current Study:** This pricing framework served as the structural foundation for the Cake Quote Calculator. Rather than forcing Mrs. Tapang to calculate prices manually over chat, the system computed the pricing of complex multi-tiered cakes using live ingredient costs, protecting her profit margins automatically.

2.1.19 Session-State Security Constraints in Public Web Platforms

In a cybersecurity analysis presented at the IEEE Symposium on Security and Privacy, Rautenstrauch et al. [17] evaluated session security vulnerabilities across commercial web platforms. The study proved that relying on loose, unsecured browser communication links exposed sensitive endpoint paths to data injection or fraudulent transaction falsification. Standardizing session tokens across user actions protected data endpoints against malicious execution.

- **Relevance to the Current Study:** This security study directly dictated the access controls programmed into the Rider Dispatch Portal. By using encrypted, short-lived secure URLs rather than open database endpoints, the system allowed neighborhood delivery couriers to view exact drop-off coordinates without needing a permanent login account, maintaining absolute client data privacy.

2.1.20 Predictive Forecasting Models for Bakery and Confectionery Markets

According to a market analysis by Mickiewicz and Britchenko [18], consumer demand patterns within the bread and bakery products sector were constantly changing based on seasonal event rushes and shifting preferences. The developers discovered that bakeries relying entirely on human intuition to forecast production requirements consistently suffered from unrecorded revenue losses or excess ingredient spoilage. Implementing historical time-series analytics allowed operations to optimize sustainable production and safety.

- **Relevance to the Current Study:** This market principle guided the baseline logic of the system's dashboard. By feeding past cake order data into the Python backend, the platform visualized precise purchasing trends, allowing Mrs. Brave's Cakes to preemptively order specialty decorative assets and ingredients prior to peak calendar seasons.

2.1.21 Digital Transformations in Global SMEs

According to a global report by the Organisation for Economic Co-operation and Development (OECD) [27], small and medium-sized enterprises (SMEs) across 38 countries were increasingly adopting digital tools to remain competitive in modern markets. The report found that SMEs that integrated cloud-based systems experienced a 15–25% increase in productivity and a 20% reduction in operational costs. However, the study also identified persistent barriers to adoption, including limited digital infrastructure, cybersecurity concerns, and a lack of awareness of low-cost cloud solutions available to small businesses. The OECD emphasized that for SMEs in developing economies, the most successful digital transformations were those that used affordable, easy-to-learn platforms that did not require specialized IT staff.

- **Relevance to the Current Study:** This global perspective directly validated the technical and economic design decisions made for Mrs. Brave's Cakes. By choosing free-tier cloud services (Google Firestore, Render, Cloudinary) and designing an interface that requires minimal technical skill, the proposed system directly addresses the adoption barriers identified by the OECD. The system's use of PayMongo's PCI-compliant payment gateway also responds to the cybersecurity concerns raised in the report, ensuring that the bakery can benefit from digital transformation without exposing itself to unnecessary financial risk.

2.1.22 Global E-Wallet Adoption and Financial Inclusion

According to a global market analysis by Statista [28], the number of digital wallet users worldwide was projected to reach 5.2 billion by 2026, with the Asia-Pacific region accounting for the majority of this growth. The report noted that digital wallets had become the leading payment method for global e-commerce transactions, surpassing traditional credit cards and bank transfers. The study highlighted that e-wallets were particularly effective in emerging economies where traditional banking infrastructure remained limited, allowing small businesses to accept digital payments without the need for expensive point-of-sale terminals or merchant bank accounts.

- **Relevance to the Current Study:** This global trend provided the macro-economic justification for integrating PayMongo into the proposed system. By supporting GCash and Maya—the two most widely adopted e-wallets in the Philippines—the platform ensures that Mrs. Brave's Cakes remains aligned with global payment trends while serving the actual payment preferences of its local customers. The system's ability to accept digital payments without requiring the bakery to maintain a traditional merchant account directly addresses the financial inclusion gap identified in this global study.

2.1.23 Cloud Computing Adoption for SMEs

According to a regional analysis by Deloitte [29], cloud computing adoption in the Asia-Pacific region had grown substantially, with businesses increasingly migrating from on-premise infrastructure to cloud-based solutions. The report found that small enterprises that adopted cloud systems experienced a 20–30% reduction in IT costs and a 15–25% increase in operational efficiency. The pay-as-you-go model of cloud services was identified as particularly beneficial for small businesses because it eliminated the need for upfront hardware investments, ongoing maintenance costs, and dedicated IT personnel. The study also noted that cloud adoption enabled real-time data synchronization and automatic backups, which were critical for businesses operating across multiple sales channels.

- **Relevance to the Current Study:** This regional analysis directly supported the serverless architecture chosen for the proposed system. By building the platform on Google Firebase and Render—both of which operate on pay-as-you-go or free-tier models—Mrs. Brave's Cakes avoids the capital expenditure and maintenance overhead that would otherwise make a digital system unaffordable for a micro-enterprise. The real-time synchronization capability of Google Firestore, in particular, enables the unified inventory ledger that keeps online orders and walk-in POS sales in perfect sync, a benefit that Deloitte identifies as a key advantage of cloud adoption.

2.1.24 User Experience Design for Non-Technical Users

According to research by Nielsen Norman Group [30], effective user interface design for non-technical users required minimizing cognitive load through clear labeling, consistent navigation, and immediate feedback. The study found that complex forms and dense information displays caused user frustration and increased task abandonment rates. Breaking complex tasks into smaller, sequential steps—a principle known as "progressive disclosure"—significantly improved user confidence and task completion rates. The research emphasized that systems designed for users with limited technical backgrounds should prioritize clarity and predictability over feature density.

- **Relevance to the Current Study:** This UX principle directly guided the design of both the customer-facing and admin-facing interfaces of the proposed system. The multi-step cake customization module applies progressive disclosure by guiding customers through size, flavor, icing, shape, layers, toppers, and add-ons one decision at a time, rather than presenting a single overwhelming form. Similarly, the admin dashboard uses familiar UI patterns—status

badges, single-click actions, and straightforward menus—so that Mrs. Tapang, a non-technical sole proprietor, can manage the system without requiring advanced technical training.

2.1.25 The Importance of Data Security in E-Commerce

According to a cybersecurity report by Kaspersky [31], small and medium-sized businesses were increasingly targeted by cyberattacks, with 39% of SMBs reporting that they struggled to fund adequate cybersecurity measures. The report found that businesses handling online payments were at the highest risk, particularly those that stored sensitive financial data such as credit card numbers or e-wallet credentials. Kaspersky emphasized that the most effective security strategy for small businesses was to avoid storing sensitive financial data altogether and instead rely on PCI-DSS compliant third-party payment gateways to handle all transaction processing.

- **Relevance to the Current Study:** This cybersecurity principle directly shaped the payment architecture of the proposed system. By offloading all payment processing to PayMongo—a PCI-DSS compliant gateway—the platform ensures that Mrs. Brave's Cakes never stores, handles, or exposes sensitive customer financial data. This significantly reduces the bakery's security risk while providing customers with a secure, trusted payment experience. The system's reliance on cryptographic webhook verification for payment confirmation also aligns with Kaspersky's recommendation to use automated, server-side verification rather than manual, client-side methods.

2.2 Local Literature and Studies

2.2.1 Over-the-Counter Cash Operations and Retail POS Systems in Local Bakeries

According to a local study by Natividad et al. [19], micro-food businesses in the Philippines heavily relied on effective point-of-sale (POS) environments to maintain operational efficiency and stability. Despite shifts toward digital marketplaces, localized neighborhood retail operations required structured systems that could systematically log inventory and handle over-the-counter transactions smoothly. Natividad et al. [19] emphasized that the quality of a POS system directly affected its acceptance among small-scale food operators. When a system provided reliable transaction features and real-time inventory adjustments, it removed the risks of manual calculation errors, protected physical cash management, and satisfied the practical operational needs of Filipino micro-retailers.

- **Relevance to the Current Study:** This local research directly justified building a walk-in POS module within the admin dashboard for Mrs. Brave’s Cakes. By incorporating an easy-to-use checkout interface into the system, Mrs. Tapang could process physical walk-in customers seamlessly beside her online orders. This prevented financial blindness from unrecorded cash register sales and ensured her inventory balances updated automatically, satisfying the precise system quality requirements outlined by Natividad et al. [19].

2.2.2 Cost-Effective Last-Mile Delivery Dispatching for Philippine MSMEs

In the Philippine e-commerce ecosystem, local food business operators faced severe fulfillment and logistics hurdles. De Guzman et al. [20] studied the real-world execution of localized e-commerce food delivery systems and highlighted that small enterprises frequently suffered from high logistical barriers, delivery delays, and platform communication friction. The developers noted that manual dispatch processes often resulted in coordination breakdowns between food preparation and the dispatching of delivery riders. For Philippine micro-enterprises to preserve profit margins and ensure food quality upon arrival, they needed to adopt practical, streamlined order management and delivery workflows that minimized communication gaps during the last-mile fulfillment phase.

Relevance to the Current Study: This directly validated the design of the Rider Dispatch Portal for Mrs. Brave’s Cakes. Instead of making Mrs. Tapang deal with complex corporate delivery apps or slow, manual texting patterns that introduced communication bottlenecks (as identified by De Guzman et al., [20]), the system allowed her to coordinate her deliveries independently. She could securely send clean destination data to her neighborhood riders using a quick web link, establishing a practical logistics workflow well within her operational budget.

2.2.3 Socioeconomic Transitions and E-Commerce Trajectories for Local Small-Scale Enterprises

In a business management thesis published under the Ramon V. Del Rosario College of Business at De La Salle University, Manila, Alberto et al. [21] investigated the operational shifts forced upon native micro and small-scale operations during periods

of extreme market disruption. The developers established that while transitioning to digital channels introduced clear opportunities for market survival, small-scale owners faced severe data tracking, logistics, and resource management hurdles due to a lack of structured, tailored IT infrastructure. Standalone, unintegrated digital channels frequently led to fragmented customer records and tracking anomalies.

Relevance to the Current Study: This local assessment underscored the necessity of moving Mrs. Brave's Cakes away from chaotic, non-integrated social media message logs (such as unorganized Facebook Messenger threads). Implementing a structured online web ordering dashboard directly answered the operational problems outlined by Alberto et al. [21] by centralizing separate transaction paths into a single cloud ledger.

2.2.4 Data Driven POS and Inventory Systems for Small Philippine Food Businesses

Divina et al. [22] developed a web-based Point of Sale and Inventory System for Pastil sa Tabi, a small Philippine food enterprise that previously relied on manual sales and stock recording—a process the developers found directly caused inaccurate transaction records, stock depletion without warning, and delayed managerial decisions due to inaccessible data. Their PHP/MySQL system integrated transaction processing with inventory management so that every sale automatically updated stock in real time. When evaluated by 50 respondents against ISO/IEC 25010 software quality standards, the system rated highly for functionality and usability, with the developers specifically noting improved accuracy in transaction recording and more reliable inventory updates as the clearest operational gains.

Relevance to the Current Study: This local research directly justified building a Walk-In Point-of-Sale (POS) module within the admin dashboard for Mrs. Brave's Cakes. By incorporating an easy-to-use checkout interface into the system, Mrs. Tapang could process physical walk-in customers seamlessly beside her online orders. This prevented financial blindness from unrecorded cash register sales and ensured her inventory balances updated automatically, satisfying the precise system quality requirements outlined by Divina et al. [22]. Unlike the advanced demand forecasting layers explored in broader supply chain literature, this study focuses on establishing a unified, real-time transaction ledger first, providing the necessary historical data foundation for future analytical enhancements.

2.2.5 Empirical Inventory Control and Its Effects on Customer Retention and Repeat Purchases

In a retail optimization study published in the World Wide Journal of Multidisciplinary Research and Development, Rosario [23] examined the relationship between structured stock management policies and long-term customer satisfaction within the Philippine market. Utilizing a descriptive correlational research design, Rosario [23] proved that structured inventory control significantly influenced consumer retention rates and repeat purchases. When a local business failed to systematically track its available material resources, it suffered from unexpected stockouts, which directly led to broken delivery promises, unfulfilled orders, and ruined consumer trust.

Relevance to the Current Study: This empirical insight directly justified the creation of the system's automated Inventory Warning Threshold Alert Engine. Following the inventory principles highlighted by Rosario [23], the platform monitored bulk baking materials (e.g., flour, butter, sugar) and fired real-time visual warnings to the admin panel when materials dipped too low, protecting Mrs. Brave's Cakes from losing custom cake sales to unexpected ingredient stockouts.

2.2.6 Comparative Usability of Local Mobile Wallets and the Case for Automated Verification

Gumasing et al. [24], surveying 165 Philippine e-wallet users using the System Usability Scale, found that GCash scored highest for usability and user satisfaction (SUS = 80.55) among the three most common local mobile wallets, with Maya and GrabPay both scoring lower and GrabPay ranking as the least accessible overall. Notably, the study also found that even GCash — the top performer — was affected by recurring app crashes and transaction-processing issues, pointing to real stability gaps in the apps micro-entrepreneurs already depend on for daily transactions.

Relevance to the Current Study: This finding supports two decisions in the system design for Mrs. Brave's Cakes. First, prioritizing GCash and Maya as the platform's supported payment methods follows Gumasing et al.'s [24] usability rankings — they are the two most usable and most locally adopted options for customers. Second, because even a leading e-wallet is prone to transaction glitches and app crashes, relying on a customer's screenshot as proof of payment is inherently unreliable — a glitch mid-transaction can leave a screenshot showing a payment that never actually completed. This is why the system verifies payment through PayMongo's automated webhook rather than a manually reviewed screenshot: it checks the transaction's actual server-side status rather than trusting a client-side image that could be incomplete, outdated, or altered.

2.2.7 Web-Based Reservation System and Real Time Availability for Small Businesses

In a systems development study, Ramento et al. [25] documented how Nature Spring Resort's manual reservation process — handled through direct booking requests with no centralized tracking — resulted in delayed processing, hassled reservations, and a real risk of human error in tracking what was actually available. The developers developed a web-based Reservation and Management System using agile methodology, and their key finding was that giving both the business and the customer real-time visibility into availability was what actually reduced the possibility of overbooking, rather than any single feature in isolation. When evaluated by IT experts against ISO 25010:2015 software quality standards, the system scored well specifically on functionality and reliability — the two qualities most tied to preventing double-booked slots.

Relevance to the Current Study: This finding shaped the Blackout Date Controller on the reservation screen of Mrs. Brave's Cakes. Rather than relying on the owner to manually track how many orders a given day could still take — the same manual bottleneck that caused Nature Spring Resort's overbooking problem — the system checks current orders against daily kitchen capacity in real time before a customer is even allowed to select a delivery date, applying the same real-time-availability principle Ramento et al. [25] validated in a different, but structurally similar, small-business booking context.

2.2.8 Familiarity and Readiness as Adoption Drivers for Non-technical Operators

In a multiple-case study of nine Philippine MSMEs navigating digital adoption during COVID-19, Mia et al. [26] found that a lack of technical skill was not actually the barrier to adoption it was assumed to be. Business owners without any IT background — such as the founder of Basic Movement — still adopted digital tools successfully, either by sticking to platforms they were already personally familiar with or by outsourcing the technical implementation to someone else entirely. The real barrier the developers identified was hesitation and readiness: some owners resisted new digital tools not because the tools were hard to use, but because they weren't yet convinced the change fit how they already ran the business.

Relevance to the Current Study: This finding shaped the interface philosophy behind the admin control panel for Mrs. Brave's Cakes. Rather than assuming Mrs. Tapang needed to learn new technical concepts, the platform was built around interactions she is already familiar with from everyday apps — straightforward menus, clear status badges, and single-click actions — so that adoption depends on comfort and trust in the new workflow rather than on acquiring new technical skills, consistent with what Mia et al. [26] observed among similarly non-technical Philippine MSME owners.

2.2.9 Integrated Data Storage for Omnichannel Micro-Retail Systems

Alberto et al. [21] further noted that micro-enterprises operating across multiple informal channels suffered from severe accounting fragmentation when physical counter logs operated separately from digital order records. The authors demonstrated that consolidating sales touchpoints into a unified master ledger removed accounting blind spots, secured operational transparency, and saved small business owners hours of manual cross-checking.

Relevance to the Current Study: This local study provided a strong academic basis for linking the online customer ordering interface and the internal Walk-In Counter POS Module into one database for Mrs. Brave’s Cakes. By sharing a single cloud data layer, the system ensured that walk-in sales and online orders updated the same ledger instantly, preventing fragmented cash logs.

2.2.10 Operational Workflow Simplification in Small-Scale Philippine Bakeries

Natividad et al. [19] established that system acceptance among micro-food enterprises depended heavily on matching software complexity to the existing skill levels of operators. Complex interfaces with redundant data entry steps caused operational delays and user frustration during busy operational hours. Simplifying data entry forms and providing automated feedback mechanisms significantly improved software utility and operational throughput.

Relevance to the Current Study: This finding directly guided the development of the Cake Customization and Order Management modules. By replacing long text-based order negotiations with structured drop-down options and automated quotation summaries, the platform reduced administrative workload and minimized order processing time for Mrs. Tapang.

2.2.11 Digital Adoption Among Filipino MSMEs

According to a study by the Asian Development Bank [32], the COVID-19 pandemic accelerated digital adoption among Philippine micro, small, and medium enterprises (MSMEs), but also exposed significant gaps in digital readiness. The study found that while many MSMEs quickly adopted e-commerce platforms and digital payment methods during lockdowns, the initial impact on business performance was mixed. Businesses that lacked proper training and support systems often struggled to realize the full benefits of digitalization. However, the study also found that over time, MSMEs that persisted with digital

tools experienced positive long-term effects on sales and operational efficiency. The report recommended that digital solutions for Philippine MSMEs should be "low-touch" and "high-impact"—easy to learn, requiring minimal training, and delivering immediate operational benefits.

- **Relevance to the Current Study:** This ADB study provided critical validation for the phased, iterative development approach taken for the Mrs. Brave's Cakes system. The finding that digital transformation yields positive results over time supports the study's conclusion that the system's value will continue to grow as the bakery and its customers become more familiar with the platform. The recommendation for "low-touch, high-impact" solutions directly influenced the system's design philosophy, which prioritizes intuitive interfaces, automated calculations, and immediate operational benefits over complex features that would require extensive training.

2.2.12 The Role of E-Wallets in Philippine Financial Inclusion

According to the Bangko Sentral ng Pilipinas [33], the adoption of digital financial services in the Philippines grew dramatically, with account ownership among Filipino adults rising to 56% in 2021. The survey found that e-wallet usage was the primary driver of this growth, with GCash and Maya emerging as the most widely used digital payment platforms. Notably, the report featured a photograph of a customer at a local bakeshop making a QR Ph payment on its cover, directly illustrating how digital payments had penetrated even the smallest neighborhood food businesses. The BSP emphasized that expanding digital payment acceptance among micro-retailers was a key pillar of its national financial inclusion strategy.

- **Relevance to the Current Study:** This BSP survey provided authoritative local evidence for the strategic decision to integrate GCash and Maya via PayMongo. The fact that the BSP's own report chose a bakeshop as the representative example of digital payment adoption confirms that the proposed system is aligned with both the national financial inclusion agenda and the actual payment habits of Filipino consumers. By enabling Mrs. Brave's Cakes to accept digital payments, the system contributes directly to the BSP's goal of expanding digital payment acceptance among micro-retailers.

2.2.13 Barriers to Digital Adoption Among Philippine MSMEs

According to a study by the Philippine Institute for Development Studies [34], women-led micro, small, and medium enterprises (WoMSMEs) in Metro Manila faced significant barriers to fully benefiting from e-commerce adoption. The study identified the primary barriers as the digital divide (limited access to reliable internet and devices), a lack of awareness of government support programs, and insufficient technical training. The researchers found that while many WoMSMEs had adopted basic digital tools like social media pages and messaging apps, few had transitioned to more structured platforms such as e-commerce websites or integrated inventory systems. The study recommended that private-sector solutions—particularly those designed around the actual workflows of small businesses—could play a critical role in bridging this gap.

- **Relevance to the Current Study:** This PIDS study directly related to the proposed system's focus on Mrs. Brave's Cakes, a woman-led micro-enterprise. The barriers identified—lack of training, limited awareness of available tools, and hesitation to adopt structured platforms—were precisely the challenges that the proposed system was designed to overcome. By providing a tangible, easy-to-use digital tool that demonstrates the benefits of e-commerce and structured inventory management, the system serves as a practical, private-sector solution to a nationally recognized problem. The study's recommendation for workflow-aligned solutions validates the design principle of building the system around Mrs. Tapang's existing daily routines rather than requiring her to adapt to a generic template.

2.3 Synthesis of the State of the Art

The landscape of bakery management technology was starkly divided between two operational extremes, leaving a significant technological gap for localized artisanal bakeshops.

2.3.1 The Entry Level: Unstructured Social Media Platforms

For Mrs. Brave's Cakes, Facebook Messenger served as the primary storefront tool since its inception. While highly accessible and effective for customer reach, social media platforms functioned fundamentally as communication channels rather than structured enterprise management systems. They lacked structured data persistence, automated dynamic pricing logic, centralized capacity scheduling, and synchronized inventory tracking. Relying heavily on informal messaging led directly to communication gaps, manual calculation delays, and logistical blindness. When a customer requested a custom cake, the owner had to manually retrieve design specifications from lengthy, fragmented chat threads, resulting in operational latency and a high probability of human error.

2.3.2 The High Level: Corporate Enterprise Resource Planning (ERP) Systems

On the opposite end of the spectrum were complex Enterprise Resource Planning (ERP) systems used by large-scale commercial bakery chains. While these corporate platforms offered advanced automated features, they were heavily over-engineered and financially unviable for a sole proprietor. They required substantial recurring subscription costs, specialized IT personnel for database administration, and rigid, standardized product catalogs. Because these corporate platforms were built for uniform, mass-production environments, they lacked the operational flexibility required to accommodate creative customization, variable design attributes, and client-submitted reference photos that Mrs. Tapang's artisanal bakeshop relied on.

2.3.3 The "Middle Ground" Solution

The proposed system was designed to fill this gap by offering a simple, custom-built middle ground. It provided a small bakeshop with the organization, speed, and safety of a professional management tool without the high cost or complexity of corporate software. By unifying cake customization, stock tracking, order scheduling, rider dispatching, and walk-in sales into a single web application, the system replaced messy chat orders with a smooth, tailored platform built specifically for Mrs. Brave's Cakes.

2.3.4 Operational Innovations Matrix

The system synthesizes the state of the art through four distinct operational innovations:

Technological Dimension	Entry Level (Social Media Channels)	High Level (Corporate ERP Systems)	The Proposed System Solution
Product & Order Management	**Unstructured Input:** Manual parsing of raw design parameters from long, unstructured chat threads was required.	**Rigid Uniformity:** Standardized, uniform mass production catalogs offered zero room for item alterations.	**Flexible Customization:** A Cake Customization Module with image uploads preserved custom design flexibility.
Sales Channel Integration	**Isolated Logs:** Online messages were completely separated from face-to-face transactions.	**Siloed Legacy Systems:** Multi-terminal architectures required isolated server cross-checking.	**Unified Master Ledger:** An interconnected web platform linked online queues and Walk-In POS modules under one database.
Last-Mile Logistic Operations	**Manual Coordination:** Staff relied on copying and pasting geographic text descriptions into text strings for riders.	**Enterprise Fleet Apps:** Fleet operations required expensive tracking networks that mandated continuous driver account logins.	**Rider Dispatch Portal:** Provided secure, token-validated hyperlinks that instantly transmitted precise checkout coordinates to an embedded .js and OpenStreetMap frame directly within the rider's mobile browser, completely removing the need to manually open external native mapping applications.

Inventory & Billing Framework	**Paper Logbooks:** Manual processes were prone to calculation errors, data losses, and screenshot payment fraud.	**Complex Supply Chains:** Automated replenishment matrices proved unsuited for local market price shifts.	**Local Sync & Gateway:** Custom Python tracking sheets were paired with automated bills and PayMongo API verification.

2.3.5 State of the Art Comparison Table

Entry Level Extreme: Social Media	The Custom Middle Ground: Proposed System	High Level Extreme: Corporate ERPs
[Social Media Platforms]	**[PROPOSED SYSTEM]**	**[Corporate ERP Systems]**
• Zero Data Structure	**• Workflow-Aligned UX**	**• Rigid Mass Production**
• High Transaction Fraud Risk	**• Automated Secure Payments**	**• Prohibitive Subscription Costs**
• Operational Coordination Blindness	**• Flexible Order Customizer**	**• Requires Dedicated IT Staff**

2.3.6 Technical Justification of System Synthesis

This synthesis addressed the core problems identified in Chapter 1 by implementing targeted technical solutions derived from established international and local computing paradigms:

- **Tailored Customization vs. Mass Production:** Unlike rigid corporate catalogs, this platform prioritized a multi-step customizer and reference photo upload system. This approach preserved the artistic, artisanal nature of the pastry designs while enforcing the strict database structure of a professional order form.
- **Unified Omnichannel Operations (Web Storefront + Walk-In POS):** Standard e-commerce templates only handled online clients. This system introduced an on-site Point of Sale (POS) module directly into the admin dashboard, compatible with local thermal printers. This enabled the owner to process physical walk-in sales simultaneously with online queues, eliminating financial data fragmentation.
- **Cost-Effective Independent Logistics (The Rider Portal):** Rather than forcing the business to lose revenue to high third-party delivery application commissions, the system generated lightweight, secure web links for independent neighborhood riders. These links transmitted static coordinates to native navigation applications, resolving last-mile
- **Localized Financial and Inventory Synchronization:** Designed specifically for the Philippine retail climate, the system bypassed complex international credit card processors in favor of the PayMongo API for seamless GCash and Maya transactions. It coupled this with automated downloadable e-receipts and direct Gmail notifications, paired with a flexible manual digital inventory ledger that allowed the owner to track baking supplies at her own pace.

2.3.7 Synthesis of Literature and Its Application to the Proposed System

The body of literature reviewed in this chapter converges on several key themes that directly informed the design of the Streamlined Bakery Management System. First, the transition from informal, chat-based commerce to structured digital platforms is essential for MSME scalability [1]. Second, inventory management for perishable goods requires a hybrid approach that combines digital tracking with human oversight [2]. Third, user interface design must minimize cognitive load to accommodate non-technical users [3]. Fourth, real-time data synchronization is critical for preventing stockouts and overbooking [4], [5]. Fifth, secure payment gateways and automated invoicing build customer trust and reduce fraud [12]. Sixth, NoSQL databases offer superior flexibility for highly customizable products [9], [13]. Seventh, localized payment methods and cost-effective logistics solutions are vital for Philippine MSMEs [24], [20].

The proposed system synthesizes these principles into a single, cohesive platform. The cake customization module applies the staged-interface principle to reduce cognitive load. The inventory module combines automated deduction with manual administrative oversight. The booking calendar implements real-time capacity validation. The payment module leverages PayMongo's webhook system for automated verification. The rider dispatch portal provides a lightweight, zero-install solution for last-mile logistics. The entire system is built on a NoSQL Firestore database to accommodate the variable nature of custom cake orders. In doing so, the study bridges the gap between the "entry level" of unstructured social media and the "high level" of corporate ERP systems, delivering a "middle ground" solution that is both practical and scalable for a micro-enterprise.

Conclusion

In conclusion, this system represented a specialized technological innovation for local food-based MSMEs. It acknowledged that a sole proprietor did not need a million-peso enterprise software suite, yet could no longer survive on a manual notebook, pen, and a fragmented social media chat thread. By consolidating online e-commerce customization, an on-site physical POS, and an external rider dispatch interface into a single NoSQL cloud ecosystem, it provided a professional, scalable, and highly secure digital home for Mrs. Brave's Cakes—effectively bridging the gap between traditional baking and modern digital commerce.

2.4 Conceptual Framework of the Study

The conceptual framework of this study followed the traditional Input-Process-Output (IPO) Model, which established a structured approach to transforming identified business problems and technical requirements into a fully functional software solution.

Fig. 2.1: Conceptual Framework of the System



2.4.1 Input (The Foundations)

The Input phase consisted of the baseline data, domain knowledge, and technical requirements needed to initiate the development of the system:

- **Business Demands:** The manual operational workflows, inventory inconsistencies, scheduling bottlenecks, and delivery tracking blind spots identified at Mrs. Brave's Cakes.
- **Technical Software and Hardware Requirements:** The tech stack utilized a responsive web frontend, a Python backend framework, and Google Firestore for real-time NoSQL data storage. To handle core system operations, the platform integrated Cloudinary for image hosting, Resend for transactional email notifications, webauth for security, and Firebase Authentication for user credentials. Additionally, the PayMongo API was integrated for local payment

gateways, alongside hardware configuration parameters to interface with the on-site thermal receipt printer for the walk-in Point of Sale (POS) module.

- **Knowledge Base:** The reviewed local and foreign literature, software engineering principles, system analysis methods, and database management theories gathered by the developers to establish a sound developmental blueprint.

2.4.2 Process (The Development)

The Process phase encompassed the specific Software Development Life Cycle (SDLC) methodologies executed by the developers to engineer the platform:

- **Requirements Analysis:** Defined the exact data structures for custom orders, user authentication routes, dynamic pricing rules, and rider tracking coordinates.
- **System Design:** Conducted architectural modeling, mapped out user interface (UI/UX) layouts, and defined the Firestore collection schemas.
- **Coding and Integration:** Executed full-stack development for the Customer Customizer, the Admin Dashboard with visual analytics, the walk-in Point of Sale (POS) module, and the Rider Dispatch Portal web links.
- **Testing and Evaluation:** Conducted system debugging, PayMongo API integration testing, thermal receipt printer compatibility validation, and User Acceptance Testing (UAT) with the shop owner.

2.4.3 Output (The Solution)

The Output phase represented the final, tangible product delivered by the study:

- **The Implemented Software System:** The final realization of the **"Streamlined Bakery Management: A Dedicated System for Cake Customization and Sales Monitoring"** for Mrs. Brave's Cakes. This materialized as a fully synchronized, omnichannel cloud ecosystem that professionalized online ordering, secured walk-in transactions, automated downloadable email receipts, and streamlined last-mile delivery logistics.

2.5 Theoretical Framework

This study is anchored on several established theories and models that guided the design, development, and evaluation of the system.

2.5.1 Technology Acceptance Model (TAM)

The Technology Acceptance Model (TAM) [35] posits that an individual's acceptance of a technology is determined by two primary factors: perceived usefulness and perceived ease of use. Perceived usefulness is the degree to which a person believes that using a particular system will enhance their job performance. Perceived ease of use is the degree to which a person believes that using the system will be free of effort.

In the context of Mrs. Brave's Cakes, TAM explains why the system was designed with a strong emphasis on simplicity and direct operational value. Mrs. Tapang, a non-technical sole proprietor, needed to see immediate benefits (usefulness) and experience minimal friction (ease of use). The user interface was intentionally built with familiar patterns—menus, status badges, single-click actions—so that adoption depended on comfort rather than technical skill acquisition. The UAT results, which scored Usability at 4.78 (Excellent), validate that the system successfully addressed both TAM constructs.

2.5.2 Unified Theory of Acceptance and Use of Technology (UTAUT)

The Unified Theory of Acceptance and Use of Technology (UTAUT) [36] extends TAM by incorporating four key constructs: performance expectancy, effort expectancy, social influence, and facilitating conditions. Performance expectancy is the degree to which using the system will help the user attain gains in job performance. Effort expectancy is the degree of ease associated with the use of the system. Social influence is the extent to which the user perceives that important others believe they should use the new system. Facilitating conditions are the degree to which the user believes that an organizational and technical infrastructure exists to support use.

For this study, UTAUT provides a comprehensive lens for understanding the multiple factors that influence whether Mrs. Tapang and her customers will adopt the system. Performance expectancy is addressed by the system's automated pricing, inventory deduction, and payment verification. Effort expectancy is addressed by the responsive, mobile-friendly interface. Social influence is addressed by the growing norm of digital transactions in the Philippines, particularly GCash and Maya. Facilitating conditions are addressed by the use of free or low-cost cloud services (Firestore, Render, Cloudinary) that do not require the bakery to maintain its own servers. The system's design explicitly targets all four UTAUT constructs.

2.5.3 ISO/IEC 25010 Software Quality Model

The ISO/IEC 25010 standard defines a quality model for software products, comprising eight characteristics: functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability. This study adopted a subset of these characteristics for its evaluation criteria: functional suitability, usability, security, and performance efficiency.

The choice of ISO/IEC 25010 was deliberate: it provides an internationally recognized, objective framework for evaluating software quality, rather than relying on subjective developer opinion. The UAT questionnaire was adapted from this standard, with statements mapped to each characteristic. The results—an overall mean of 4.79 (Excellent)—demonstrate that the system meets high standards across all four evaluated dimensions. This theoretical grounding ensures that the evaluation is both rigorous and comparable to other software quality assessments.

2.5.4 Agile Development Model

The Agile Development Model is an iterative and incremental approach to software development that emphasizes flexibility, customer collaboration, and rapid response to change. Unlike traditional waterfall models, Agile breaks the project into small, manageable increments called sprints, each delivering a potentially shippable product increment.

This study utilized Agile because the requirements of a small bakery are not static. During development, Mrs. Tapang provided feedback that led to revisions in the cake customization interface, the pricing logic, and the booking calendar. The Agile approach allowed the developers to incorporate these changes without disrupting the entire project. The five stages—Requirements Analysis, System Design, System Development, Testing, and Deployment—were executed iteratively, with each sprint producing a working prototype for review. This methodology is documented in Chapter 3, Section 3.2.

2.5.5 Diffusion of Innovation Theory

The Diffusion of Innovation theory [37] explains how new ideas and technologies spread through a social system over time. The theory identifies five adopter categories: innovators, early adopters, early majority, late majority, and laggards. It also identifies five factors that influence adoption: relative advantage, compatibility, complexity, trialability, and observability.

For this study, the theory helps explain the potential for the system to be adopted not only by Mrs. Brave's Cakes but also by other local bakeries and food MSMEs. Relative advantage is demonstrated by the system's ability to reduce manual errors and save time. Compatibility is ensured by designing the system around existing workflows. Complexity is minimized through a simple, intuitive interface. Trialability is provided by the free tier of cloud services, allowing other businesses to test the system without financial commitment. Observability is achieved through the tangible benefits—faster order processing, secure payments, and real-time analytics—that can be seen and measured. This theoretical lens supports the study's broader impact on the MSME sector.

2.6 Review of Related Systems

To further contextualize the proposed system, the developers examined existing platforms that offer similar functionalities. The following comparison highlights the gaps that the proposed system fills.

System	Description	Strengths	Limitations for Mrs. Brave's Cakes

Facebook Marketplace / Messenger	Social media platform used by the bakery prior to the system.	Free, widely used, familiar to customers.	No structured order forms, no automated pricing, no inventory tracking, no payment verification, no scheduling.
GrabFood / Lalamove	Third-party delivery and food ordering platforms.	Large customer base, integrated payment, rider network.	High commission fees, no customization module for custom cakes, no inventory synchronization, no control over branding.
Shopee	E-commerce platform for online stores.	Robust product catalog, payment integration, order management.	Subscription cost, not tailored for custom cake attributes, no walk-in POS, no rider dispatch portal.
Bakery POS Systems (Square, Lightspeed)	Specialized POS systems for retail bakeries.	Inventory management, receipt printing, sales analytics.	Designed for walk-in retail, not custom online orders; no integration with local e-wallets like GCash.
Generic ERP Systems (SAP, Oracle NetSuite)	Enterprise resource planning systems for large businesses.	Comprehensive modules for finance, inventory, HR, sales.	Prohibitively expensive, overly complex, requires dedicated IT staff, not suitable for a sole proprietor.
Proposed System	Custom-built web application for Mrs. Brave's Cakes.	Combines online customization, walk-in POS, inventory, scheduling, rider dispatch, and automated payments in one platform. Uses free/low-cost cloud.	Limited to single-branch operations; no native mobile app; no automated 3D printing or IoT integration.

The review of related systems reveals a clear gap in the market. On one end, social media platforms like Facebook Messenger offer free communication but lack structured data management. On the other end, enterprise ERP systems offer comprehensive features but are financially and technically out of reach for a micro-enterprise. Third-party delivery apps charge high commissions and do not accommodate custom cake orders. The proposed system occupies a unique "middle ground": it is purpose-built for a custom cake bakery, integrates with local payment methods, and operates on a low-cost, serverless cloud infrastructure. This synthesis justifies the development of a tailored solution rather than adopting an off-the-shelf product.

CHAPTER 3

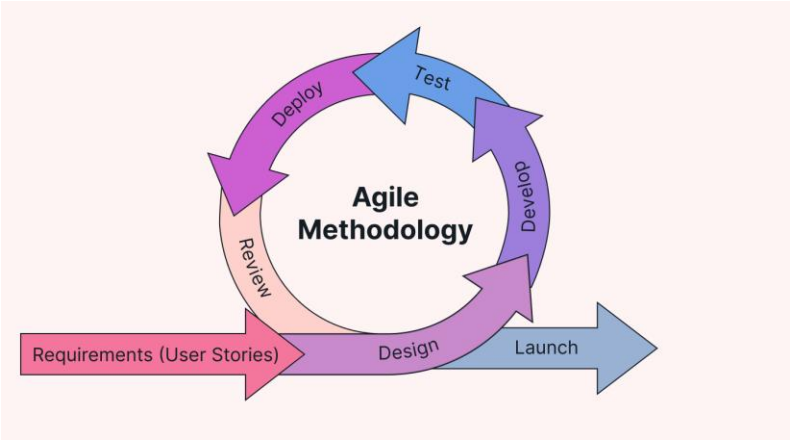
METHODOLOGY

3.1 Research Design

The study utilized a developmental research design, focusing directly on the systematic design, development, and operational evaluation of the proposed Streamlined Bakery Management: A Dedicated System for Cake Customization and Sales Monitoring for Mrs. Brave's Cakes. This design model was uniquely suited for software engineering and capstone research, as it shifted the focus from purely theoretical analysis to the active creation and empirical evaluation of an actionable instructional or technological product. By applying this framework, the study systematically analyzed the establishment's fragmented, manual workflows—which previously relied on physical paper logs, handwritten notes, and disjointed social media messaging threads—and transformed them into integrated, automated digital modules built upon a single NoSQL cloud ecosystem. The execution of this developmental architecture was divided into two distinct, interconnected tracks: product development and product evaluation. The development track followed a structured software engineering pipeline engineered to build three core operational components: the customer-facing online cake customization system, the on-site physical point of sale (POS) retail tracking module, and the decentralized, coordinate-based external rider dispatch portal. Each component addressed specific data silos identified during initial baseline observations, mapping real-time data flows across inventory tiers, active custom cake design profiles, and sales ledgers. The evaluation track concluded the pipeline with an empirical testing phase. This phase subjected the fully implemented platform to objective quality metrics and multi-stakeholder user acceptance testing (UAT). By bridging targeted technical engineering with operational testing, this research design validated that the completed web ecosystem resolved real-world logistical bottlenecks. Ultimately, it ensured that the system provided a professional, highly secure, and structurally scalable digital home for Mrs. Brave's Cakes, establishing a repeatable development blueprint for local food-based MSMEs transitioning into modern digital commerce.

3.2 System Development Methodology

The project utilized the Agile Development Model as its core system development life cycle (SDLC) framework. This iterative approach allowed for step-by-step programming and continuous adjustments based on actual feedback, changing operational demands, and technical requirements of Mrs. Brave's Cakes. Rather than enforcing a rigid, linear progression, the Agile methodology embraced flexibility, breaking down the entire cloud-integrated platform into manageable functional increments. This ensured that features like the Cake Customizer or the Point of Sale (POS) module could be developed, tested, and refined rapidly. By organizing development around tight feedback loops, the developers adapted to unexpected technical constraints or shifting business needs without disrupting the foundation of the platform's architecture.



The development process was broken down into five main stages:

3.2.1 Requirements Analysis

In this initial stage, the actual operational needs and structural parameters of the business were systematically collected directly from the owner, Mrs. Tapang. The primary goal of this phase was to dissect, catalog, and translate the manual, fragmented day-to-day routines of Mrs. Brave's Cakes into a rigid set of software requirements and data metrics. By executing a thorough workflow audit, the developers mapped out every operational touchpoint—from the exact moment a customer inquired about a cake design to the final payment realization and last-mile delivery hand-off. This investigative analysis was critical for identifying all the necessary data variables that had to be processed by the database engine, establishing a clear technical baseline for development. To replace the store's reliance on physical paper logbooks, sticky notes, and unstructured order details coming in from chaotic Facebook Messenger chat threads, the requirements gathering focused heavily on standardizing order information. For the Customer Customizer module, the developers isolated the exact structural data variables needed to run the Cake Customization Module. This included cataloging all custom cake variations such as the number of available tiers, precise dimensions and size charts, core batter flavors, specialized frosting options, decorative trim variables, and maximum file constraints for user-uploaded reference photos. Simultaneously, the requirements analysis defined the exact metrics needed for the real-time sales tracking dashboard and the walk-in Point of Sale (POS) component. This involved mapping out data fields for raw ingredient costing, markup margins, order status states (e.g., Pending, Preparing, Out for Delivery, Completed), cash-drawer inputs, and local digital payment transaction parameters. Finally, the logistics pipeline was audited to determine the precise order variables—such as recipient contact details, pinned delivery coordinates, delivery time windows, and active rider tracking statuses—required by independent couriers via decentralized web links. Gathering these comprehensive requirements eliminated operational blind spots, ensured the Google Firestore NoSQL collections were designed with the correct schemas, and provided a clear, structured blueprint for the upcoming system development sprints.

3.2.2 System Design

The system design phase focused on transforming the raw requirements collected during the analysis stage into detailed visual wireframes, user interface (UI) layouts, and a robust backend database architecture. Rather than relying on rigid, traditional relational databases that could suffer from latency during rapid inventory updates, the technical layout of this platform focused heavily on component modularity and real-time data flow synchronization across all channels. By establishing clear design blueprints prior to writing the source code, the developers ensured that the customer experience was logically structured and the administrative controls remained highly intuitive for the bakeshop's operators. For the front-end user experience, an interactive customization interface form was designed for the cake customizer module to make the complex ordering and checkout process straightforward for clients. Instead of overwhelming users with a dense, single-page questionnaire that could lead to confusion, the interface broke the customization pipeline down into logical, sequential nodes. The client first selected the base parameters, moved to flavor tiering and sizing configurations, uploaded their visual design references, and finished with delivery logistics and payment processing. This configuration reduced user cognitive load, prevented transaction abandonment, and ensured that all localized order details—such as design references and dedication text—were cleanly parsed before hitting the server. Concurrently, the user experience layouts for the on-site walk-in Point of Sale (POS) dashboard and the decentralized Rider Portal were mapped using responsive design frameworks, ensuring seamless readability across mobile screens and desktop terminals. For the back-end database architecture, Google Firestore was utilized to structure the system records. The NoSQL document-oriented setup was organized into distinct, highly decoupled collections optimized for lightning-fast read/write queries, low latency, and real-time updates. This schema ensured that an update in the ingredient inventory ledger instantly reflected across the admin dashboard without causing database locks, creating a synchronized, omnichannel data environment for Mrs. Brave's Cakes.

The architecture was structurally organized into five core collections:

- **The Orders Collection** served as the main repository for all customer-initiated requests. It stored unique document IDs mapped to custom cake parameters, including tier counts, exact dimensions, chosen frosting and batter flavors, embedded URLs for client-uploaded design references hosted on Cloudinary, custom message inscriptions, active workflow statuses (Pending, Baking, Ready for Pickup, Out for Delivery), and payment status indicators.

- **The Products Collection** handled the static and semi-static catalog of the bakeshop. It documented all standard ready-made products, pre-baked items available for instant purchase, standard baseline pricing matrices, available flavor variants, and real-time daily stock availability limits to prevent over-purchasing during peak retail hours.

- **The Transactions Collection** served as the central financial ledger for the entire establishment. It unified data streams by compiling on-site cash registers from walk-in POS sales alongside automated online e-commerce checkout logs managed via the PayMongo API hook. Each document contained strict timestamps, gross sales totals, applied tax rates, ingredient cost deductions, and payment methods.

- **The Bookings Collection** managed time-sensitive schedules and calendar slot allocations. It logged pickup dates, targeted delivery windows, special venue instructions, and pinned geographic coordinates for delivery routes. This

collection was engineered to prevent double-booking on limited-capacity dates, ensuring the baking staff was never logistically overloaded.

- **The Inventory Collection** operated as the foundational raw material tracking engine. It logged the current quantities of baseline ingredients (such as flour, sugar, eggs, and specialized fondants), units of measurement, safety reorder thresholds, and expiration tracking parameters. It linked directly to the production engine to allow for automatic stock deductions as orders moved into production.

3.2.3 System Development

The implementation phase transformed the architectural blueprints and data models into a production-ready, full-stack web platform. The system was built using a modern full-stack web development architecture, utilizing a highly responsive frontend for user interface components, a robust Python backend framework to execute advanced business logic and calculations, and Google Firestore for real-time cloud data storage and multi-terminal database synchronization. Rather than engineering the codebase as a single monolithic entity, the programming workflow was strategically split into practical, decoupled development sprints. This modular approach ensured that each core feature was developed, integrated, and unit-tested independently before being deployed to the live staging environment.

The software development track was systematically divided into four critical system features:

- **The Customer Storefront** The development began with a public-facing web portal engineered for prospective clients. This interface included an interactive, graphical multi-step cake customization engine where clients selected structural modifications in real-time. The frontend code linked directly to an asynchronous backend logic processor that dynamically calculated instant pricing based on raw ingredient dimensions, flavor choices, and complexity markups. Upon checkout, the module handled payment tokenization and transmitted the structured payload to the database, automatically updating the centralized order pipeline.

- **The Admin Dashboard** This private, credential-secured workspace served as the operational headquarters for the business owner, Mrs. Tapang. The frontend architecture featured dynamic data visualization libraries that read directly from Firestore transactions to render real-time sales graphs, monthly profit metrics, and categorical revenue distribution charts. Furthermore, the backend logic integrated system commands to format and output clean financial logs, manage user permissions, and handle manual inventory entry configurations for recording raw material restocks.

- **The Point of Sale (POS) Module** Embedded natively within the internal Admin Dashboard, this module provided an optimized, low-latency interface designed explicitly for walk-in, face-to-face retail operations. When an on-site transaction occurred, the operator encoded order details on a cash-basis layout. The backend architecture automatically processed these inputs, triggered immediate inventory deductions within the ingredient database collection, committed the record to the central sales ledger, and sent structured print commands to an on-site local thermal receipt printer via low-level interface protocols.

- **The Rider Portal Link** is a lightweight web-based logistics interface designed to operate without requiring delivery drivers to download or install a dedicated mobile app. Upon order dispatch, the backend system dynamically generates a secure, tokenized, and time-limited URL that is transmitted directly to the assigned rider via SMS or messaging webhooks. Accessing this link opens a responsive WebGIS navigation dashboard powered by Leaflet.js and OpenStreetMap data.

  The portal provides real-time route optimization, step-by-step turn directions, delivery verification tools (including digital customer signature capture and photo delivery proofs), and live status toggles (e.g., *In Transit*, *Arrived*, *Delivered*). All interactions trigger low-bandwidth WebSocket events back to the primary backend database to update the customer-facing order tracker in real time.

3.2.4 Testing

After the development sprints, the system underwent rigorous component and system integration testing to clear out software bugs, prevent logic errors, and ensure optimal system reliability under active retail conditions. Rather than executing a single, superficial check at the very end of the production line, the testing phase was structured to validate both the isolated business logic inside individual code functions and the end-to-end data pipelines running across third-party microservices. This

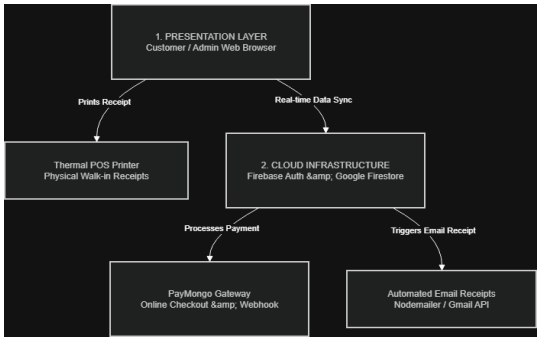
comprehensive diagnostic review ensured that data transitions across the customer frontend, the Python backend, and the Google Firestore database were effectively synchronized. A primary focus of the functional testing track was validating the reservation calendar logic and the automated payment webhooks. The component testing framework stress-tested the booking engine to confirm that the reservation calendar correctly blocked dates and dynamically adjusted booking capacities in real time, preventing double-bookings or kitchen operational overloads during high-demand seasonal weeks. Concurrently, system integration testing evaluated the asynchronous communication between the bakeshop platform and external financial gateways. The developers systematically tested the PayMongo API integration to verify that the system changed order statuses to 'Paid' instantly upon successful GCash, Maya, or credit card transactions, triggering immediate updates within the administrative workflow dashboard and logging the entry into the transactions collection. Beyond cloud-based data processing, the testing pipeline included a dedicated local hardware integration phase. Because Mrs. Brave's Cakes relied on physical transactions for walk-in retail operations, local hardware testing was conducted with a physical thermal receipt printer. This process tested the raw ESC/POS command strings generated by the point of sale (POS) module, verifying that the system could cleanly format, align, and print physical paper receipts right after a walk-in sale was processed. By combining digital API testing with real-world hardware verification, this phase helped ensure that the completed platform was stable, secure, and ready for deployment without risking data loss or transaction drops during peak operational hours.

3.2.5 Deployment and Prototype Review

The working web prototype was hosted on a secure cloud-server environment, establishing a live, fully accessible staging platform for the establishment. This production-ready deployment build served as the primary environment for actual User Acceptance Testing (UAT), shifting the software from a local sandbox into a real-world testing theater. By utilizing scalable cloud hosting, the developers ensured that database connectivity, automated API responses, and multi-user synchronization loops could be evaluated under realistic conditions that mimicked the daily traffic spikes experienced by a busy commercial bakeshop. During this operational evaluation phase, the business owner, Mrs. Tapang, along with a selected group of target end-users—comprising bakeshop staff, active retail customers, and independent delivery couriers—interacted directly with the live web application. Testers systematically executed end-to-end workflows to measure the platform's ease of use, interface response times, data input accuracy, and general performance when processing daily bakery orders, calculating real-time cake customization dimensions, logging ingredient expenses, and dispatching decentralized rider coordination links. The empirical data, usability feedback, and performance metrics gathered during this live deployment tracking phase were analyzed against institutional quality benchmarks. This ensured that any final user interface adjustments or backend latency optimizations were completed before the platform was officially handed over as the permanent digital management engine for Mrs. Brave's Cakes.

3.3 System Architecture

The system utilized a serverless client-cloud architecture to handle real-time data updates and simplify system maintenance. Instead of relying on a traditional, high-maintenance local server setup, the web-based front-end application connected directly to secure cloud services, managed databases, and external third-party payment processing gateways. This architectural pattern reduced the overhead of managing hardware infrastructure locally at the bakeshop, ensuring high system availability, automatic scaling during peak ordering seasons, and secure data isolation across all active terminal endpoints.



The architecture structured into three main components:

3.3.1 Presentation Layer (The Client)

This component served as the user-facing side of the web application, operating as the primary gateway for all human-computer interactions within the establishment's ecosystem. Built utilizing modern web technologies, it ran entirely within the user's native web browser, eliminating the need for client-side software installations and ensuring cross-platform compatibility. The presentation architecture leveraged responsive web design methodologies, allowing the user interface layouts to adapt smoothly and dynamically to different screen dimensions and device orientations.

This responsive architecture ensured that customers could access the online storefront and intuitively customize a cake on a smartphone, independent delivery riders could cleanly check pinned location coordinates and update order milestones on a mobile device while in transit, and the owner could comfortably monitor the comprehensive bakery dashboard and financial charts on a desktop or laptop screen.

This layer managed the absolute visual interface of the platform, taking charge of client-side routing, state management, and user input validation. It displayed the interactive multi-step cake customizer form to the user, breaking a historically complex ordering workflow down into step-by-step sequential phases. To optimize the user experience and reduce server overhead during design sessions, the presentation layer computed instant price estimates locally on the screen. It did this by executing real-time logic calculations whenever a user selected specific cake sizes, tier counts, or base materials, providing immediate cost feedback before any data was sent to the backend database.

Furthermore, this layer dynamically formats the graphical layout of the walk-in Point of Sale (POS) module, structuring the sales interface specifically to communicate with local hardware. It handles the structural translation of over-the-counter transactional summaries into precise ESC/POS data alignments, enabling the POS interface to directly output a physical thermal paper receipt the moment a walk-in sale is executed—eliminating the need for digital email dispatch during in-person transactions.

3.3.2 Cloud Infrastructure and Database Layer

Data processing, event handling, and persistence tasks were managed securely in the cloud through the Google Firebase ecosystem. This serverless layer reduced the need for a local database server, physical on-site storage arrays, and complex database maintenance protocols, significantly reducing localized technical overhead for the business. By executing processes within a highly scalable cloud infrastructure, the platform helped ensure continuous uptime and instantaneous synchronization across all distributed user terminals. This cloud architecture was structurally split into two primary functions:

- **Firebase Authentication** This service handled identity management and secure session protocols for the system. It enforced strict access controls by verifying user credentials and issuing secure JSON Web Tokens (JWT) to manage session states. This service ensured that sensitive back-end routes—such as the administrative dashboard, raw inventory controls, supplier cost sheets, and financial data records—remained securely isolated and accessible only to the authorized owner, Mrs. Tapang, and trusted bakeshop personnel. It prevented unauthorized endpoint access, mitigating the risks of data breaches or malicious modifications to transaction histories.

- **Google Firestore (NoSQL)** This component served as the centralized, document-oriented cloud database for the entire enterprise. Departing from rigid, traditional relational databases that rely on heavy table joins, Google Firestore structured data into flexible documents and collections, optimized for fast querying and concurrent reads/writes. Because it utilized native WebSockets to run real-time data listeners instead of relying on traditional static HTTP polling requests, the database established an instantaneous data feedback loop.

  Consequently, the moment a transaction was processed on the walk-in Point of Sale (POS) module or a new custom cake design was submitted by an online customer, the state change triggered a live update. The data propagated instantly across the cloud, updating the administrator's dashboard, scheduling calendar, and ingredient inventory ledger in real time without requiring a manual page refresh. This structural immediacy provided the bakeshop with automated inventory deductions and accurate data synchronization.

3.3.3 External Integration Layer

To handle specialized business operations and offload complex computing tasks from the core application, the web architecture connected directly to external microservices through industry-standard Application Programming Interfaces (APIs). By decoupling these specific financial and communications processes from the central application stack, the platform achieved greater security, higher transaction throughput, and a professional user experience. This layer coordinated all communication with external platforms and was split into two specialized service integrations:

- **PayMongo API Gateway** This integration served as the centralized financial processing node for all digital customer checkout operations. When a customer finalized an order using the online cake customization interface, the platform securely forwarded the payment information to the PayMongo API gateway to process GCash, Maya, or card payments. Because the application never directly handled or stored sensitive credit card credentials or digital wallet pins, this integration kept the platform compliant with security standards.

  The system relied on asynchronous communication through API Webhooks. The moment a payment was successfully processed within the digital wallet network, the gateway executed a secure, encrypted POST request (a real-time confirmation callback) to the system's backend framework. Upon receiving and validating this webhook signature, the backend instantly updated the target transaction record, changing the order status to Paid inside the Google Firestore database and moving the order directly into the kitchen's production queue.

- **Notification and Automated Receipting System:** This module operates as an automated customer relationship and transactional logging engine dedicated to online orders. Utilizing the **Resend API**, this system establishes an automated event link with the database. The moment an online order status document updates in the Firestore repository—transitioning to Paid upon PayMongo webhook confirmation—the state change triggers a serverless event handler. The backend processes this trigger by compiling an automated transaction summary and dynamically generating a structured digital e-receipt dispatched directly to the customer's registered email inbox.

  The backend framework processed this trigger by pulling the customer's profile information, compiling an automated transaction summary, and dynamically generating a downloadable, structured digital receipt. This compiled summary was then instantly dispatched directly to the customer's registered email inbox. This process ensured immediate, paperless verification for the consumer while maintaining an organized, searchable digital trail for Mrs. Brave's Cakes.

3.4 Tools and Technologies Used

To build and deploy the system, a modern, serverless web development stack was selected to ensure real-time data synchronization, robust cloud security, and a smooth user experience. Rather than building a monolithic application that required complex infrastructure management, this modular technology stack decoupled user interface execution from backend processing. This structural approach minimized system latency, optimized data transmission over local mobile networks, and ensured high availability for Mrs. Brave's Cakes.

The specific tools that utilized are broken down below:

3.4.1 Frontend Technologies

The client-side interface relied entirely on modern web standards and responsive frameworks to ensure cross-platform execution without requiring native app installations. This layer processed presentation logic, dynamic state rendering, and real-time client-side inputs directly within the user's web browser.

- **HTML5 & CSS3:** HTML provided the structural framework for the application's pages, including forms, input controls, image uploads, and interactive interface components. CSS was responsible for the visual presentation, implementing responsive layouts, typography, color schemes, and user interface styling.

- **JavaScript (Vanilla):** JavaScript served as the core programming language for engineering the user interface logic and managing client-side application behavior. It acted as the functional engine driving the browser, enabling asynchronous data fetching (via Fetch API), dynamic component rendering, and real-time price calculations in the multi-step custom cake builder without requiring constant server refreshes.

- **Bootstrap Framework:** This component-driven frontend framework was utilized for crafting the application's comprehensive visual styling and responsive layout grids. It ensured that the customer-facing storefront, the dense data views on the admin POS dashboards, and the lightweight rider verification links were fully responsive across mobile phones, tablets, and desktop laptops.

3.4.2 Cloud Infrastructure & Backend Services (Serverless/Hybrid)

The backend layer of the application utilized a hybrid approach, combining a robust Python-based web framework for core business logic with managed cloud services for data persistence and authentication.

- **Python (Flask Framework):** Flask served as the primary backend web framework. It handled core business logic, dynamic pricing calculations, routing, and the orchestration of external API requests (such as PayMongo webhooks and Resend email dispatch). Its lightweight nature made it ideal for the microservices architecture required by the bakeshop.

- **Google Firebase (Authentication & Firestore):** Google Firebase was deployed as the Backend-as-a-Service (BaaS) provider. Firebase Authentication handled identity management and secure session protocols (including Google OAuth and WebAuthn biometric logins). Google Firestore served as the centralized, cloud-native NoSQL document database. Departing from rigid relational schemas, Firestore stored data as flexible documents, utilizing native WebSockets to push real-time, low-latency updates directly to the administrator's dashboard and POS terminal without requiring manual page reloads.

3.4.3 Third-Party APIs and Hardware Integrations

To ensure platform security and a frictionless user experience, the system incorporated three distinct external services. First, it integrated webauth at critical interaction points—such as registration, login, and checkout—to distinguish human users from automated bots, effectively mitigating brute-force attacks and spam submissions. Second, for customer service automation, the system utilized the Gemini 2.5 Flash-Lite model via API, powering an intelligent conversational chatbot capable of answering frequent customer inquiries about product availability, store hours, and order tracking without administrative intervention. Lastly, the application connected to the PayMongo API Gateway to manage digital payment processing; when a checkout was initiated, the system securely routed the customer to the PayMongo interface to authenticate and complete electronic wallet payments (such as GCash), ensuring that real-time transaction states were safely synchronized back to the administrative sales logs.

- **PayMongo API Gateway** The PayMongo API was integrated directly into the web application's checkout pipeline to handle real-time digital transaction processing. This payment gateway allowed the platform to securely accept localized electronic payment methods—specifically GCash, Maya, and local credit cards—without directly processing, storing, or exposing sensitive financial details, bank credentials, or digital wallet pins inside the bakeshop's system database. By shifting payment tokenization to PayMongo's secure servers, the platform maintained a highly secure transaction environment that seamlessly updated order workflows the moment funds were cleared.

- **Cloudinary Media Management API** Cloudinary served as the dedicated, cloud-native content delivery network (CDN) and media asset storage engine for the entire platform. To ensure that the customer-facing storefront and interactive cake customizer remained incredibly lightweight and fast-loading, heavy media file handling was completely offloaded from the core Google Firestore database. Cloudinary automatically captured, optimized, compressed, and resized custom cake sample photos and reference images uploaded by both the administrator and customers. It then served these media files through high-speed edge servers, ensuring that high-resolution design images loaded quickly across local mobile networks without introducing latency to the backend.

- **Resend API (Transactional Email Engine)** The Resend API served as the platform's automated customer communication engine. This specialized email delivery service bridged the database state machine with the customer's inbox. The moment an administrator changed an order status within the dashboard backend—such as transitioning an active workflow entry from Pending to Approved, Baking, or Out for Delivery—the backend automatically compiled the transaction metadata, routed it through the Resend API, and dispatched a cleanly formatted, responsive email notification directly to the customer. This provided immediate operational updates and digital receipts without requiring manual communication from the bakery staff.

- **Thermal Printer Driver and Native Print Configuration** For on-site walk-in transactions, a localized hardware integration protocol was established inside the Point of Sale (POS) layout module. Because the system ran inside a standard web browser environment, the frontend utilized customized CSS3 print media queries and structured text formatting rules aligned with the standard ESC/POS layout requirements. The system processed completed transaction metrics and stripped away standard browser margins, headers, and navigation UI components. It then passed a raw, cleanly formatted receipt layout directly to the local physical thermal receipt printer using native web browser print pipelines, enabling the bakeshop to generate instant, physical paper receipts right at the counter.

3.4.4 Development Environments

To maintain source code integrity, track system modifications, and streamline the software engineering pipeline across all five

development stages, a combination of industry-standard coding environments and version control systems was established. These tools provided the foundational workspace required to write, test, debug, and safely back up the platform's multi-layered architecture.

- **Visual Studio Code (VS Code)** Visual Studio Code served as the primary Integrated Development Environment (IDE) for writing, debugging, and managing the project's entire unified codebase throughout the development lifecycle. This highly extensible code editor was configured with specialized technical modules, including syntax highlighters, real-time code linters, and embedded terminal instances. These tools allowed the developers to write clean frontend layout scripts, format complex backend business logic, and test cloud integration rules within a single workspace. VS Code's integrated local debugging tools enabled the developers to isolate runtime script errors and intercept API call errors before code was pushed to production, accelerating the overall development sprint timeline.

- **Git and GitHub.** These tools were used for source code version control, tracking software modifications throughout the development process, and maintaining secure cloud-based backups of the project repository. Git ran locally on the development machines to record incremental code changes and enable the developers to create separate development branches for testing new features, such as enhancements to the cake customization interface, without affecting the stable version of the application.

  GitHub served as the remote repository for storing and synchronizing the project's source code. It provided centralized version management, preserved the complete development history, supported collaboration among the developers, and maintained an audit trail of software revisions throughout the capstone project.

3.5 Data Gathering Techniques

To gather the necessary operational data and system requirements for designing the platform, the developers used primary qualitative data gathering methods tailored specifically to the business environment of Mrs. Brave's Cakes. Relying on qualitative inquiry ensured that the software development life cycle was informed by the actual nuances, pain points, and administrative workflows experienced by the establishment. Rather than drawing conclusions from generalized bakery models, this targeted approach allowed the developers to extract precise functional specifications directly from the daily operations of the store.

The data collection process relied on two main techniques:

3.5.1 Semi-Structured Interviews

The developers conducted formal, semi-structured interviews with the business owner, Mrs. Shamelyn Tapang. These qualitative sessions targeted understanding the exact business logic, operational rules, financial rules, and administrative workflows of the bakery. Because this methodology utilized a semi-structured format, the developers used a prepared interview guide as a structural baseline while maintaining the flexibility to ask follow-up questions as new operational details emerged. This open-ended approach was critical for uncovering underlying business processes that may not have been formally documented in the bakeshop's current logs.

The primary objective of these interviews was to extract critical operational data points and business constraints required to build the platform's backend rules, specifically targeting three core business areas:

- **Pricing Matrices** The interviews focused on defining how baseline prices scale according to specific cake tiers, precise structural dimensions, and specialized cake flavor combinations. The developers mapped out the exact mathematical rules the bakery uses to calculate raw material usage alongside labor costs for multi-layered items. This ensured that the system's dynamic pricing engine accurately mimicked the owner's manual pricing logic when calculating totals for a client.

- **Add-on Cost Calculations** The sessions focused on mapping out the pricing tiers for premium customization variables that alter production time and material consumption. This included establishing cost rules for intricate custom fondant work, specialized icing textures, edible printing components, and decorative cake toppers. Pinpointing these variables allowed the developers to construct precise data inputs within the interactive customizer form, ensuring that any add-on selected by an online user dynamically calculated and added the correct surcharge to the invoice before checkout.

- **Logistical Pain Points** The inquiries targeted documenting the specific administrative challenges, tracking errors, and

operational bottlenecks that occurred within the current manual business framework. The developers investigated how the bakery tracked partial down payments, verified digital payment screenshots sent over chat applications, managed delivery coordination errors with independent couriers, and handled order volume overbooking during holiday peak seasons.

The empirical data gathered during these interviews directly established the calculation rules, backend algorithms, database validations, and workflow constraints used in the system's interactive cake builder and administrative order tracking modules. This ensured that the finished digital platform structurally reinforced and automated the actual business policies of Mrs. Brave's Cakes.

3.5.2 Direct Observation

The developers spent dedicated time on-site directly observing the daily, live operations of Mrs. Brave's Cakes to understand exactly how transactions, design inquiries, and logistical coordination moved from the initial customer contact point to final fulfillment. This immersive observation phase allowed the developers to look past anecdotal summaries and capture the actual behaviors, mechanical steps, and operational friction points within the bakeshop. By operating as passive observers within the business premises, the developers analyzed the layout of the current workspace and documented human-to-computer or human-to-paper interactions in real time.

This observation phase will focus on two main operational areas:

- **The Customer Inquiry Pipeline** The developers reviewed how incoming customer order requests, reference inspiration photos, and intricate structural modifications were manually handled, negotiated, and scheduled through Facebook Messenger chats. Observation focused on tracking the timeline of a typical inquiry: how long it took to manually calculate a price quote, how design modifications were discussed without visual standardization tools, and how final agreements were pinned down. This allowed the developers to witness firsthand the vulnerabilities of chat-based commerce, such as overlooked client messages, untracked design changes, and the lack of a structured verification method for down-payment screenshots.

- **The Manual Record-Keeping Process** This track involved closely observing how day-to-day business data was captured inside the physical shop. The developers watched how raw ingredient tracking, material inventory deductions, walk-in cash sales, production schedules, and freelance rider dispatch notes were recorded inside physical notebooks and paper logbooks. Special attention was paid to the mechanical transition points—specifically, how a baker verified if enough fondant or flour was in stock before accepting a flash order, and how physical receipts were hand-written for walk-in retail clients.

These direct observations provided the empirical baseline needed to design the unified Admin Dashboard, the walk-in Point of Sale (POS) module, and the static link dispatch portal. By documenting where manual calculations slowed down, where transcription errors occurred, and where data silos existed, the developers ensured that the new serverless cloud software directly addressed and resolved real-world operational bottlenecks for Mrs. Brave's Cakes.

3.6 Testing Procedures

Before final deployment, the software underwent a structured, multi-tiered testing phase to identify hidden runtime bugs, secure end-to-end data pipelines, and ensure all system features worked exactly as intended in a live environment. This verification phase bridged the raw programming sprints with live production, verifying that each digital module satisfied both its individual technical performance metrics and its macro-level business operational objectives.

The testing process broke down into three distinct levels:

3.6.1 Unit Testing

The developers isolated and tested individual software components to make sure they functioned correctly on their own before connecting them to the rest of the application ecosystem. By focusing on isolated code segments, function blocks, and mathematical calculations entirely separate from external database states or server environments, unit testing ensured that foundational logic blocks were structurally sound and error-free. This approach prevented minor programming syntax bugs from cascading into major system integration failures later in the development cycle.

This verification process specifically focused on three critical functional areas within the platform's codebase:

- **Dynamic Price Calculation Logic.** Unit testing was conducted on the system's dynamic price calculation module to verify the accuracy of its pricing algorithm. The developers used a series of predefined test cases containing different combinations of cake sizes, flavors, layers, toppers, candles, and selected add-ons. The computed total generated by the system was compared with manually calculated expected values to ensure that the application produced accurate, real-time price estimates before order information was stored in the database.

- **Cart Total Accuracy of the Walk-in POS Module** The point-of-sale module's transactional calculation scripts underwent rigorous verification. This involved writing assertions to check how the local state management array accumulated individual retail product items, applied specific localized discounts, calculated correct tax deductions, and computed the gross final bill. These test vectors ensured that the on-site cash counter interface performed calculations with zero structural rounding errors or total mismatches during rapid walk-in checkout operations.

- **Input Form Validation Control** The developers tested the individual input validation gates across the customer registration, customization submission, and reservation scheduling panels. These unit tests purposefully executed negative testing scenarios—submitting empty strings, invalid email syntax structures, improperly formatted phone numbers, or negative numeric bounds into the text fields. This ensured that the client-side forms instantly intercepted, flagged, and blocked malicious, blank, or broken submissions right inside the user's browser, preventing corrupt data nodes from hitting the cloud database.

3.6.2 System Testing

After the individual parts and sub-modules were verified, the application was tested as a complete, integrated whole. Rather than evaluating features in isolation, system testing subjected the fully compiled, unified application to comprehensive operational stress tests. The team simulated the entire end-to-end operational workflow under realistic network conditions to ensure that data passed seamlessly, securely, and with minimal latency between the client-side frontend interfaces, the cloud database, and external microservice networks.

This testing phase specifically focused on validating two macro-level system pipelines:

- **The End-to-End E-Commerce and Cloud Synchronization Workflow** The team traced a complete transaction lifecycle to ensure all decoupled cloud layers communicated in harmony. Testing began by initiating a mock transaction within the customer-facing cake builder portal. The system had to successfully transition from capturing custom design parameters to processing a mock payment via the PayMongo API.

  The team then verified that the external webhook securely triggered the system's backend, confirmed the clearance of funds, and instantly synchronized the transaction state inside the Google Firestore database. Finally, the test validated that this database mutation simultaneously updated the live administrative dashboard calendar for the kitchen staff and triggered the automated transactional email receipt delivery to the customer's inbox without any manual intervention.

- **The Local Hardware Point of Sale (POS) Workflow** System testing also evaluated the physical boundaries of the platform by conducting live hardware simulations. The developers processed walk-in transactions through the specialized POS terminal layout to confirm that the software could translate digital checkout balances into physical real-world outputs.

  This test confirmed that the application successfully stripped away web browser interfaces, applied the necessary layout styles, and transmitted raw formatting instructions directly to the local thermal receipt printer. The test was successful only when the physical hardware cleanly outputted a correctly aligned, legible paper receipt immediately upon walk-in order finalization.

3.6.3 User Acceptance Testing (UAT)

The final testing phase involved handing the system over to real users to validate its practical effectiveness, intuitive flow, and operational value within a live retail environment. Shifting the software out of the development sandbox and into the hands of the actual stakeholders allowed the developers to evaluate real-world usability. This phase confirmed that the application design

matched human behavior and that the administrative modules aligned with the daily pacing of a commercial baking environment.

The UAT track brought together two specific user groups to interact with the live cloud prototype:

- **The Primary Administrator (Business Owner)** Mrs. Shamelyn Tapang directly operated the administrative console, the walk-in Point of Sale (POS) interface, and the inventory ledger. Her testing track simulated a busy day of operations, focusing on processing incoming custom orders, evaluating the clarity of the scheduling calendar, updating material stock metrics, and checking the performance of the local thermal receipt printer. Her feedback was critical for verifying that the backend features successfully automated and simplified her executive business workflows.

- **The Target End-Users (Regular Customers, Staff, and Delivery Riders).** A selected sample group of regular customers, alongside retail staff and independent delivery riders, evaluated the developed system through their respective portals. Customers used the online cake customization interface to personalize cake orders and complete simulated online checkouts. Staff and riders assessed the usability of managing customer orders, viewing delivery locations through integrated maps, updating order statuses, and processing transactions efficiently under real-world operating conditions.

These participants evaluated the platform based on its ease of use, interface clarity, system feedback speeds, and how well it actually solved the shop's day-to-day manual tracking and overbooking problems. The qualitative feedback, friction points, and feature ratings gathered during this phase were meticulously documented by the developers. This evaluation data was used to make final user interface layout refinements, database query optimizations, and performance adjustments prior to the official system turnover, ensuring the finished software was effectively tailored to the operations of Mrs. Brave’s Cakes.

3.7 Evaluation Criteria

The software was evaluated using adapted criteria based on the ISO/IEC 25010 System and Software Quality Requirements and Evaluation (SQuaRE) international standard. This approach ensured that the platform was objectively measured against recognized global software engineering metrics rather than relying on subjective feedback alone. By utilizing this structured quality framework, the developers assessed both the technical integrity of the serverless backend and the practical utility of the client interfaces.

The evaluation was conducted systematically by the business stakeholders, bakeshop staff, and selected end-users. The assessment focused on the following four key quality characteristics tailored to the operational needs of Mrs. Brave’s Cakes:

3.7.1 Functional Suitability

This criterion evaluated how completely, accurately, and appropriately the developed system executed its intended features under active business conditions. Rather than assessing the software as a generic web application, testing and evaluation under this parameter focused heavily on how well the platform satisfied the explicit functional requirements of the bakery. The goal was to confirm that the software contained all necessary tools to eliminate operational bottlenecks, enforce correct business logic, and support the staff's daily workflows without introducing errors.

The evaluation for Functional Suitability focused on four critical technical milestones:

- **Precision of the Dynamic Price Calculations** The evaluation verified that the software's backend calculation engine computed mathematically precise financial totals within the interactive customer-facing cake customizer. Evaluators checked if the pricing algorithm accurately aggregated base material fees, dimensional scaling modifiers, tier multipliers, and premium add-on surcharges (such as intricate fondant designs or decorative toppers). The final on-screen price matched the store's official pricing rules effectively, eliminating human calculation errors.

- **Reliability of the Walk-in Point of Sale (POS) Cart Module** This track assessed the functional completeness of the on-site cash counter interface. Testers evaluated the system's ability to maintain real-time cart state accuracy during fast-paced walk-in retail sales. The evaluation ensured that adding or removing pre-baked products from the cart, applying promotional discounts, computing local taxes, and processing cash collections synced instantly with the central ledger without dropping data packets.

- **Successful Generation of Automated Transaction Documents** This milestone measured the accuracy of the platform's automated notification and data processing tasks. The system was evaluated on its ability to capture

database triggers and instantly compile structured transaction summaries. Functional suitability was achieved when the platform successfully dispatched a downloadable, mathematically accurate PDF invoice to the customer's email via the notification API the moment an order was finalized or paid.

- **Physical Thermal Print Execution** Evaluators tested the hardware communication pipelines of the POS module. This involved confirming that the system could cleanly translate a digital transaction summary into raw, un-margined print commands. The application successfully transmitted formatting rules to the physical thermal receipt printer, outputting structured, properly aligned paper receipts immediately after an on-site sale was finalized.

By verifying these targeted operational areas, this evaluation ensured that the completed serverless platform successfully addressed, automated, and fixed the manual tracking problems, schedule overbookings, and financial logging inefficiencies it was engineered to resolve for Mrs. Brave's Cakes.

3.7.2 Usability

This criterion measured the user-friendliness, intuitiveness, and overall accessibility of the application's decentralized interfaces. Because the system bridged diverse user segments with varying levels of technical literacy—ranging from online retail shoppers and third-party delivery couriers to bakeshop personnel and the executive administrator—the user interface (UI) had to be self-explanatory and structurally consistent. The evaluation assessed how smoothly these distinct user groups could complete tasks without experiencing interface friction, cognitive fatigue, or input confusion.

The usability metrics were evaluated across two primary operational pipelines:

- **Customer Storefront Navigation and Customization User Experience (UX).** The evaluation assessed how easily sample customers could interact with the responsive web storefront and use the cake customization interface. Evaluators determined whether the interface provided clear visual feedback as users selected cake size, flavor, layers, decorations, add-ons, and uploaded design reference images.

  Usability under this sub-metric was measured by the extent to which first-time users could successfully customize a cake, view real-time price updates, review their order details, and complete the online checkout process without requiring assistance, troubleshooting guides, or administrative intervention.

- **Administrative Command Center Operability** Furthermore, this metric measured how quickly and comfortably the business owner, Mrs. Tapang, could learn to manage backend business operations through the unified Admin Dashboard. The assessment focused on the ergonomic layout of data fields, input forms, and navigation links. Evaluators measured her ability to adjust the ingredient inventory ledger, cross-reference calendar reservation slots, modify custom order statuses, and read the visual sales analytics charts without requiring advanced technical training.

By analyzing these targeted areas, the usability score verified that the layout designs, component arrangements, typographic scales, and color hierarchies provided by standard HTML5, CSS3, and JavaScript delivered a clean, efficient, and natural user experience across mobile devices, tablets, and desktop monitors.

3.7.3 Security

This criterion focused on the systematic protection of transactional data, user identity records, and administrative access privileges across the entire application architecture. Given that the platform processes digital payments and handles private consumer information, maintaining a rigid security posture was critical to preventing malicious data manipulation and unauthorized data exposure. The evaluation measured how effectively the application's cloud infrastructure established and enforced cryptographic guardrails, token validations, and endpoint restrictions under active business conditions.

The security evaluation specifically targeted two primary protection mechanisms within the system:

- **Identity Management and Role-Based Access Controls** The evaluation verified that the backend security configuration and Firebase Authentication rules correctly restricted unauthorized users from accessing the Admin Dashboard or hitting sensitive data endpoints. Evaluators tested the perimeter defenses of the system by attempting to access administrative URLs, financial logs, and inventory controls without a valid active session token.

Security success required that the system intercept unauthorized routing attempts, invalidate falsified session requests, and maintain strict cryptographic isolation between public customer storefront views and private backend administrative modules.

- **Financial Data Security and Tokenization** Additionally, the assessment confirmed that the PayMongo API integration securely processed digital wallet payments—specifically GCash and Maya—without introducing security compliance vulnerabilities. Evaluators audited the data transmission packets sent during checkouts to confirm that the platform never captured, exposed, or logged sensitive consumer financial data, PIN codes, or bank credentials within the local system or the central Firestore database.

  By confirming that payment operations relied entirely on secure, external API tokenization and encrypted asynchronous webhooks, this evaluation ensured that the platform provided a highly secure e-commerce environment for the customers of Mrs. Brave's Cakes.

3.7.4 Performance Efficiency

This criterion measured the overall responsiveness, execution speed, data throughput, and client-side resource usage of the system under operational conditions. In a serverless client-cloud architecture, performance efficiency was heavily tied to script optimization, asset management, and the architectural design of database queries. The evaluation analyzed how efficiently the system utilized cloud network pipelines and browser memory to minimize latency, ensuring that users did not experience structural delays, lagging inputs, or data sync drops during high-volume periods.

The performance efficiency evaluation specifically focused on two key technical execution fields:

- **Real-Time Data Synchronization and Query Latency** The evaluation analyzed how quickly the serverless architecture retrieved, mutated, and transmitted data packets across the cloud network. Evaluators measured the real-time synchronization speed of the Google Firestore database when updating the administrative booking calendar and inventory ledger.

  Performance success required that mutations—such as a successful custom order checkout or an instantaneous stock deduction—propagated via active WebSockets to the admin panel within milliseconds. This rapid synchronization loop was critical for ensuring accurate schedule availability, providing immediate operational visibility, and preventing double-booking conflicts during peak bakery seasons.

- **Asset Optimization and Mobile Responsive Render Times** Additionally, this metric evaluated the frontend rendering speeds and mobile responsiveness of the Tailwind CSS and Bootstrap storefront layouts. Because a majority of online consumers placed orders using cellular networks on mobile phones, the platform's initial page load times and core web vitals were closely monitored.

  The evaluation measured how efficiently the system displayed structural layouts and pulled optimized custom cake images from the Cloudinary CDN. The application had to achieve rapid document object model (DOM) rendering, ensuring that the interactive cake customizer remained smooth and lightweight on mobile web browsers without consuming excessive local device memory or battery power.

3.8 Supporting Diagrams

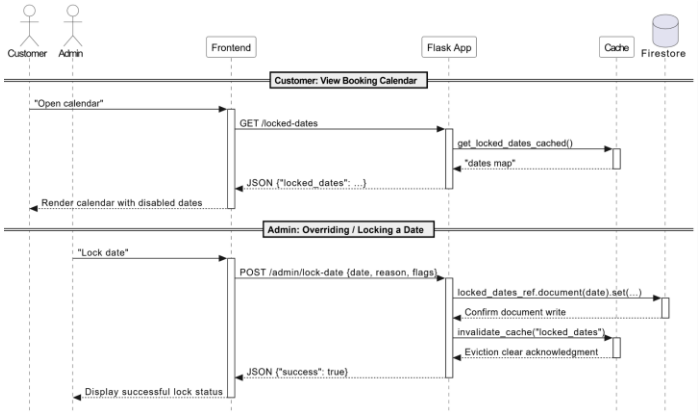
Because the platform utilized a NoSQL database architecture through Google Firestore, traditional relational Entity-Relationship Diagrams (ERDs) were not applicable to its non-relational, document-oriented structure. Relational structures rely on rigid foreign keys and schema enforcement, whereas Firestore operates on an independent architecture of nested collections and document nodes. Consequently, the system's operational processes, data transaction loops, and real-time state mutations were mapped out comprehensively using a series of specialized UML Sequence Diagrams.

These diagrams illustrated the exact message-driven, chronological interactions between the platform's primary actors and its decoupled service layers: the Customer, the responsive Frontend Web Interface, the Python Flask Application, the local Caching Layer, the Cloud Database (Firestore), external payment gateways (PayMongo), and external utility notification

services (FCM).

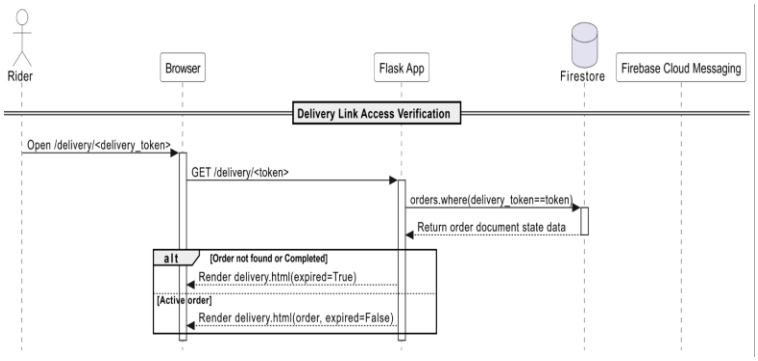
3.8.1 Calendar Optimization & Asynchronous Cache Invalidation Flow

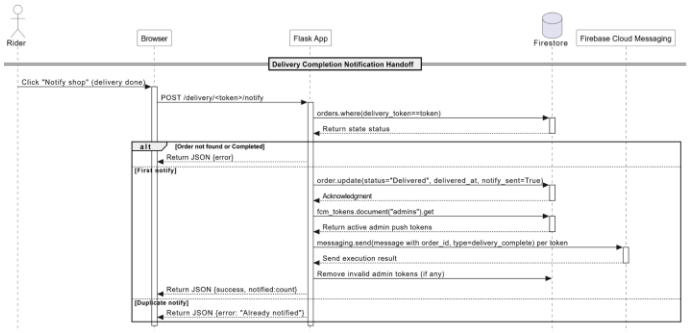
This diagram maps out the platform's scheduling optimization guards. It illustrates the real-time request-response routing deployed to handle calendar views and prevent holiday overbooking. It details how customer-facing date-checks were optimized through a localized memory cache to minimize Firestore read consumption, alongside how explicit administrative date-locks bypassed the cache to mutate core collections in Google Firestore and evict stale operational data instantly.



3.8.2 Delivery Fulfillment Link & Admin Notification Tracking

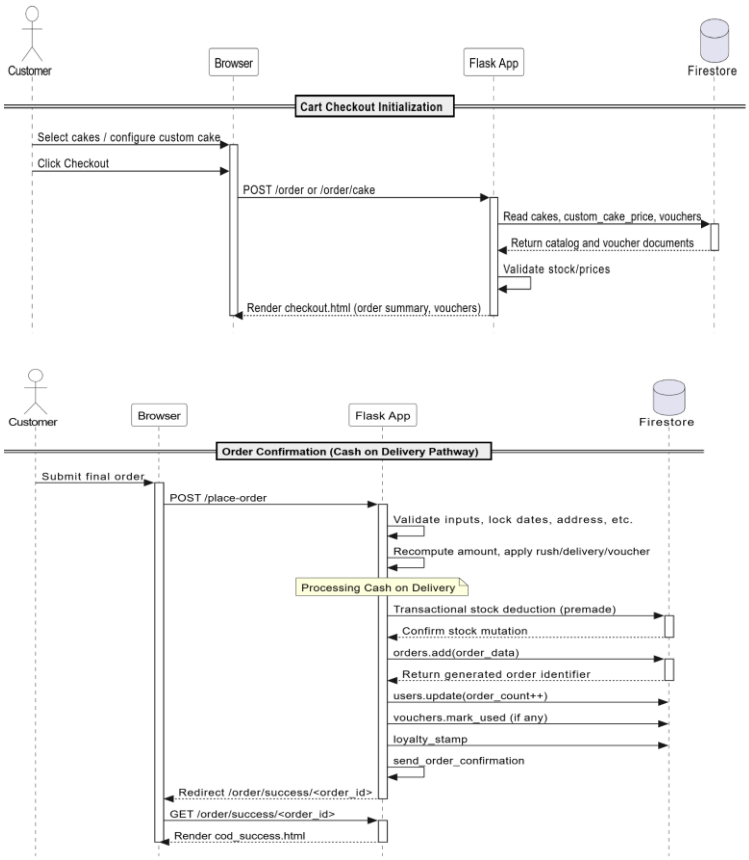
This diagram details the fulfillment orchestration pipeline. It shows the message exchange triggered when a delivery rider accessed a temporary transit routing token. It highlighted how the backend queried Firestore to prevent riders from accessing stale or expired order tokens, handled live fulfillment events, tracked single-token delivery verification updates, and leveraged Firebase Cloud Messaging (FCM) to push native alert states directly to the administrative monitoring layout.





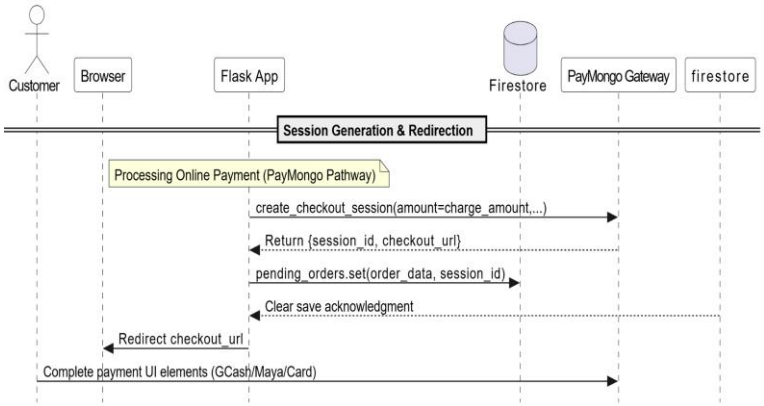
3.8.3 End-to-End Customer Checkout & Cash on Delivery Processing

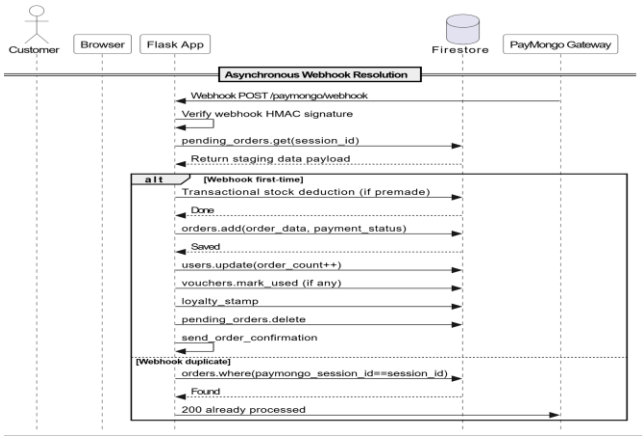
This diagram maps out the initial transactional logic branch within the platform checkout ecosystem. It presents the unified data sequence executing when a customer triggered their shopping basket handoff, validating stock variables and running price validation matrices internally before committing permanent database changes for Cash on Delivery (COD) transactions.

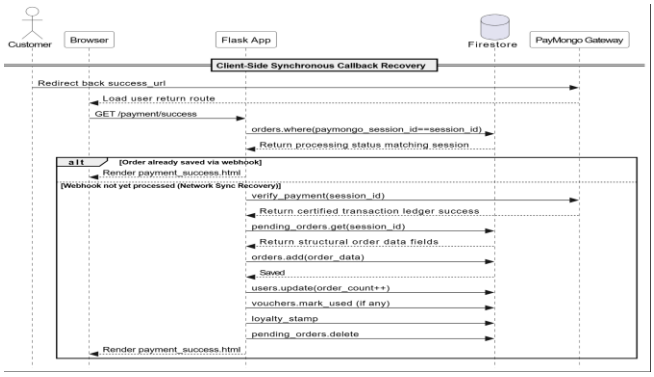


3.8.4 Online Payment Gateway Lifecycle & Webhook Processing

This diagram broke down the system synchronization logic executed for online settlement lanes using the external PayMongo API infrastructure. It detailed the complex asynchronous states running simultaneously between client browser interfaces, signature-verified payment confirmation webhooks, and programmatic recovery methods used to verify order consistency and preserve database integrity under unstable network conditions.







3.9 NoSQL Firestore Database Architecture and Data Dictionary

Because the system was built on a modern serverless infrastructure, it utilized Google Firestore, a cloud-hosted, document-oriented NoSQL database, rather than a traditional Relational Database Management System (RDBMS). Firestore abandons fixed tables, strict row boundaries, and foreign key constraints in favor of a hierarchical model composed of Collections and Documents. Documents contain specific key-value pairs, nested maps, arrays, and primitives. This schema-less flexibility allowed variable-heavy attributes—such as dynamic cake configurations, variable delivery link metadata, and tracking parameters—to be nested cleanly inside a single database reference node.

The following section provides the complete technical data dictionary for the core administrative security logging collection deployed within the cakeshop project instance.

3.9.1 Root Collection: admin_logs

A. Subsystem Context and Purpose

The admin_logs collection forms the foundational security auditing and administrative accountability subsystem of the platform. In a cloud-hosted enterprise architecture, tracking high-privilege operations is essential for ensuring operational transparency, maintaining internal data integrity, and establishing a clear historical record of system interactions. This collection serves as a dedicated, immutable activity log that automatically intercepts, structures, and archives every backend state mutation executed through the administrative control panel.

Whenever an authorized administrator performs a critical system action—such as updating an order status, locking production dates on the calendar, modifying product pricing models, logging store disbursements, or updating inventory records—the platform dispatches a background write operation to this path. By logging individual personnel details, client network identifiers, operational action descriptions, and targeted resource markers, the system ensures complete accountability. This comprehensive audit trail simplifies debugging during database state conflicts, assists in verifying administrative workflow steps, and guarantees compliance with digital security standards.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** Unique 20-character, cryptographically secure alphanumeric hash.
- **Generation Engine:** Automatically generated client-side by the Google Cloud Firestore SDK upon document insertion (e.g., 020N4vY8orjqc4mVavin).
- **Storage Pattern:** Flat, append-only transactional document log node. Documents within this collection are never modified or deleted by normal system routines to preserve audit integrity.
- **Indexing Strategy:** Single-field automatic indexes are applied to the timestamp and category fields to ensure high-speed sorting when rendering history timelines on the admin panel.

C. Field-Level Structural Rules

Table 3.4: admin_logs Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
action	String	No	"Changed order status to 'Accepted'"	A descriptive string detailing the precise database mutation or business logic operation executed by the administrator.
admin_name	String	No	"Justin Solamillo"	The explicit full display name extracted from the administrator's account profile to identify the individual responsible for the operation.
category	String	No	"order"	A rigid categorization string token used by the system to filter logs into specific operational modules. Accepted tokens are constrained to: "order", "inventory", "calendar", or "expense".
ip_address	String	No	"127.0.0.1"	The client internet protocol address captured at request execution time, utilized to track geographic access origins and detect suspicious remote sessions.
target	String	No	"Order #Y51X4gBTGXJjx1nrGrd6 — Justin Solamillo"	A specialized compound reference string that explicitly identifies the specific external collection document instance modified by this event.

timestamp	Timestamp	No	May 17, 2026 at 8:41:26 PM UTC+8	A server-side generated nanosecond clock marker recording the exact moment the write was committed to the Firestore cluster.

D. Operational Flow and Security Considerations

Data entries compiled within the admin_logs branch are strictly protected under server-side Firestore Security Rules to prevent unauthorized disclosure or tampering. Standard registered customer accounts, delivery rider profiles, and unauthenticated public visitors are completely restricted from reading, querying, creating, or modifying documents within this audit path.

Write access and entry retrieval are restricted exclusively to authenticated users possessing a verified administrator role token (request.auth.token.role == 'ADMIN'). Furthermore, the web application framework enforces structural validation on all incoming payload properties prior to commit, guaranteeing that audit identities, client network parameters, and action details cannot be manually altered, masked, or spoofed during operation.

3.9.2 Root Collection: cakes

A. Subsystem Context and Purpose

The cakes collection served as the central digital inventory and menu repository for the storefront web application. In a cloud-hosted e-commerce ecosystem, dynamic data extraction was required to keep customer views synchronized with the bakery's real-time physical production capabilities. This collection held structural documents for every active item available for public ordering—ranging from standardized everyday retail desserts to premium cakes requiring strict stock management tracking.

The Python Flask App repeatedly read from this collection to populate the customer menu grid, verify item prices during checkout payload computations, and evaluate whether an item had enough physical stock to satisfy incoming order arrays. By structuring these inventory records in a centralized collection node, the system ensured that changes made by administrators in the backend panel automatically updated customer catalog menus across all active browser windows.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character cryptographically secure alphanumeric hash key.
- **Generation Engine:** Instantiated automatically by the Google Cloud Firestore SDK upon catalog creation (e.g., OVJABOZNjwwYHp7Vbzhv).
- **Storage Pattern:** Independent document blocks containing flat primitives and static absolute resource strings.
- **Indexing Strategy:** Composite indexes are defined for category and status to let users quickly filter products on the frontend storefront while skipping unavailable or out-of-stock items.

C. Field-Level Structural Rules

Table 3.5: cakes Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
category	String	No	"Dessert"	The grouping tag used by frontend query loops to separate menu items into distinct user layout rows (e.g., "Dessert", "Custom", "Premium").
created_at	Timestamp	No	April 24, 2026 at 9:35:54 AM UTC+8	Records the exact chronological date and time the product was introduced into the digital catalog matrix.
description	String	No	"Pistachio Cheesecake"	A public-facing description text block used to details the specific flavor profile, ingredients, or allergens of the menu item.
image	String	No	"https://res.cloudinary.com/..."	An absolute URL pointing directly to an optimized web graphic asset stored within the project's external Cloudinary CDN bucket.
name	String	No	"Pistachio Cheesecake"	The master item string variable rendered as the prominent header text inside storefront cards and point-of-sale receipt slips.
price	Number (Integer)	No	250	The raw baseline unit cost of the item before shipping fees, rush service markups, or promotional coupon vouchers are applied.
quantity	Number (Integer)	No	8	The live physical inventory count. This value is decremented programmatically during automated checkouts to prevent over-purchasing.
status	Boolean	No	true	A boolean visibility control switch. Setting this to true exposes the item to the public web interface, while false hides it instantly without deleting its historical records.

D. Operational Flow and Security Considerations

Security rules configured for the cakes tree utilize an explicit read/write split based on authentication states. Public internet browsers are granted free read access (allow read: if true;) to fetch catalog names, pricing values, and image endpoints without needing an account session.

Conversely, creating, modifying, or deleting document properties inside this data tree is locked tight. It requires a validated Firebase Authentication signature identifying the user as an authorized employee (allow write: if request.auth.token.role == 'ADMIN';). This completely blocks unauthorized API requests from tampering with prices or inflating product stock counts.

3.9.3 Root Collection: custom_cake_price

A. Subsystem Context and Purpose

The custom_cake_price collection acts as the dynamic configuration utility for managing the cost variables of custom cakes. Instead of hardcoding pricing thresholds within the backend server logic, this data path provides editable rules that the multi-step frontend configuration calculator references directly. This approach ensures that updates to standard tier pricing or specialized service fees apply instantly to incoming user design selections.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** Alphanumeric descriptor matching specific pricing rule matrices.
- **Generation Engine:** Structured manually during database installation to provide reliable routing keys.
- **Storage Pattern:** Flat document entry maps containing numeric parameter lists.

C. Field-Level Structural Rules

Table 3.7: custom_cake_price Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable ?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
base_tier_1	Number (Integer)	No	1500	Minimum unit cost calculated for single-tier configuration projects before custom icing additions.
base_tier_2	Number (Integer)	No	2800	Baseline cost added automatically for multi-tiered structure layout choices.
fondant_modifier	Number (Integer)	No	500	Additional markup cost variable added for specialized icing designs.
rush_fee	Number (Integer)	No	350	Operational surcharge applied to calculations if selection dates sit within a 48-hour range.

update_marker	Timestamp	No	May 22, 2026 at 10:15:30 AM UTC+8	Standard calendar milestone tracking when adjustments were latest saved by system administrators.

D. Operational Flow and Security Considerations

Data assets mapped under the custom_cake_price collection enforce public evaluation permissions alongside secure administrative editing locks. Store clients are permitted standard fetch calls to parse active configuration elements into local pricing modules. Structural adjustments are completely restricted to administrative profile sessions (request.auth.token.role == 'ADMIN') to avoid external manipulation of financial rule matrices.

3.9.4 Root Collection: expenses

A. Subsystem Context and Purpose

The expenses collection functions as the primary financial ledger for tracking outbound operating costs, inventory replenishment outlays, and utility overhead expenses managed through the administration system. Whenever an administrator modifies ingredient volumes or purchases stock, the transactional system automatically saves a detailed record to this path.

By recording financial metrics—such as unit costs, precise timing logs, and item change logs—the application provides store management with a reliable audit trail for daily overhead tracking. This design isolates financial transactions from active inventory nodes, simplifying end-of-month accounting and budget calculations.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character secure alphanumeric unique identifier hash.
- **Generation Engine:** Generated automatically by the Cloud Firestore SDK at the moment of ledger document insertion (e.g., 02zMQALpTyyKH6sLa654).
- **Storage Pattern:** Flat root reference nodes acting as append-only ledger entries.

C. Field-Level Structural Rules

Table 3.8: expenses Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
cost	Number (Integer)	No	900	The total numeric monetary value of the financial expense transaction.
date	Timestamp	No	April 5, 2026 at 11:41:37 AM UTC+8	Explicit chronological tracking marker recording exactly when the expense was logged.

description	String	No	"Edited inventory: Flour(by kilo) - Qty: 10, Cost: ₱90.0"	Descriptive text detailing the breakdown, quantity, unit rates, and specific operational reason for the ledger entry.

D. Operational Flow and Security Considerations

Entries in the expenses collection are protected by backend validation constraints. Because these records contain sensitive financial data, public read access is entirely blocked.

Firestore Security Rules enforce that only authenticated accounts with a validated manager token (request.auth.token.role == 'ADMIN') can search, read, or append entries to this data path. To preserve audit trail integrity, modification and deletion operations are completely disabled once a document is committed.

3.9.5 Root Collection: fcm_tokens

A. Subsystem Context and Purpose

The fcm_tokens collection acts as the target address registry for the platform's real-time alert systems. To send asynchronous push notifications—such as order status updates, newly submitted custom cake requests, or instant message warnings—the Firebase Cloud Messaging (FCM) engine requires valid hardware registration token paths. This collection maps active device strings to user identity roles, allowing the backend framework to group and push out automated event broadcasts immediately without scanning the entire central user table.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** Fixed systemic alphanumeric strings designating specific account role classes.
- **Generation Engine:** Initialized intentionally on the console database configuration layer to act as predictable partition paths (e.g., admins).
- **Storage Pattern:** Single-document dictionaries containing multiple dynamic key-value pairs where keys map to randomized device handles and values store the cloud token string arrays.

C. Field-Level Structural Rules

Table 3.9: fcm_tokens Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
KPanLhrWCZ...	String	No	"cosGnr-X5iaJXpbK...KY4k5mV4VwqPSM63Poqg..."	A dynamic field key representing an active device instance identifier, mapping to its unique Firebase Cloud Messaging token string value.

RuNTPfmm9N...	String	No	"foR7xZWBwW56XoreOVGSdf:APA91b...jOuGw"	A dynamic field key storing the encrypted cloud message token value for an administrative device.
nlcXidMhGx...	String	No	"fxwhllytqLA1lldurbqqrj:APA91bFAJePc...RJ6AO1oYTyXZ..."	A dynamic field key mapping an additional unique connection instance to its live background push verification handle.

D. Operational Flow and Security Considerations

Because registration data inside fcm_tokens directly impacts device access vectors and client alert security, data mutation capabilities are strictly restricted. Client frontend applications can call programmatically bounded updates to register their own device vectors upon successful multi-factor authentication loops. Global retrieval paths or multi-node deletion routines are restricted entirely to verified administrator roles (request.auth.token.role == 'ADMIN') to prevent token hijacking or unauthorized alert blocking.

3.9.6 Root Collection: expenses

A. Subsystem Context and Purpose

The expenses collection serves as the dedicated financial ledger within the cloud database ecosystem, designed to record and structure all operational business disbursements. Rather than relying on physical logs or manual bookkeeping, the application programmatically tracks expenditures such as raw baking material procurement, packaging purchases, facility utility overheads, and hardware maintenance.

By centralizing these outflow records, background database pipelines aggregate total operating expenses against incoming store revenues retrieved from completed Point of Sale (POS) transactions and verified online custom cake checkouts. This real-time integration allows the administrative backend to dynamically compute current net profitability, monitor cost overhead trends, and generate accurate financial reporting summaries without requiring manual accounting calculations.

B. Document ID Structure & Storage Mechanics

- **Document ID Type**: 20-character secure alphanumeric unique identifier hash.
- **Generation Engine:** Provisioned automatically by the cloud database layer upon expenditure log entry initialization (e.g., 8k9LpQ2vR4tY1mW3zX7n).
- **Storage Pattern:** Flat root reference nodes tracking itemized business expenditure records.

C. Field-Level Structural Rules

Table 3.10: Expenses Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Purpose & Constraints
category	String	No	"Ingredients"	Classifies expense type (e.g., "Utilities", "Packaging").
amount	Number	No	1250.50	Financial cost in PHP (must be positive).
description	String	No	"Fondant procurement"	Brief details of the expense.
expense_date	Timestamp	No	May 12, 2026 at 2:30 PM	Date the expense occurred.
logged_by	String	No	"Tqt7OUsVnITjE4qx5ebJZQGoARd2"	Firebase Auth UID of the admin.
created_at	Timestamp	No	May 12, 2026 at 2:35 PM	Record creation time.

D. Operational Flow and Security Considerations

Data assets stored under the expenses collection are strictly isolated from client-facing storefront routes to safeguard sensitive corporate financial details. Unauthenticated guest visitors and standard registered customer accounts are fully restricted from reading, querying, or writing to this collection.

All read queries, record mutations, and administrative additions are strictly controlled by server-side Firestore Security Rules. Access is restricted exclusively to authenticated requests carrying a verified administrator role token (request.auth.token.role == 'ADMIN'), preventing unauthorized data access or unverified edits to financial histories.

3.9.7 Root Collection: locked_dates

A. Subsystem Context and Purpose

The locked_dates collection serves as the core scheduling validator and constraint engine for the storefront reservation flow. To maintain kitchen delivery guarantees and prevent order over-saturation, the platform reads this path before rendering the calendar date-picker tool to customers during checkout. If an administrator manually sets a calendar block or if automated operations detect that a specific production date has met its maximum volume capacities, the target date is indexed here to instantly grey out that slot on the frontend interface.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** Structured calendar date identifier strings formatted strictly as YYYY-MM-DD.
- **Generation Engine:** Provisioned explicitly based on the chosen blocked calendar date to allow high-speed direct document lookup queries (e.g., 2026-05-17).
- **Storage Pattern:** Flat systemic rule nodes storing operational block details.

C. Field-Level Structural Rules

Table 3.11: locked_dates Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
locked_at	Timestamp	No	May 17, 2026 at 11:32:27 PM UTC+8	Explicit chronological tracking marker recording exactly when the calendar restriction rule was committed.
locked_by	String	Yes	null	Stores the unique account profile handle or system automation ID that initialized the date block.
reason	String	No	"Fully Booked"	Descriptive status label displayed inside administrative calendars or tooltips to explain the scheduling block (e.g., "Fully Booked", "Holiday Closure").

D. Operational Flow and Security Considerations

Because scheduling data within locked_dates alters checkout rules and impacts incoming store revenues, the validation nodes rely on strict read/write permission divisions. Storefront buyers are granted open read parameters (allow read: if true;) to parse active block sets directly into the customer appointment calendar view. Conversely, creating new blocks or removing existing documents requires a verified administrator session verification claim (request.auth.token.role == 'ADMIN') to block unauthorized scheduling tampering.

3.9.8 Root Collection: notifications

A. Subsystem Context and Purpose

The notifications collection manages user-facing application alerts, acting as the historic log for systemic message events pushed to customer accounts. Whenever background payment hooks or administrative actions modify an order status, the server automatically appends a tracking block here. This allows client frontends to display reactive status notification lists, drop-down activity logs, and real-time alerts without constantly re-verifying active order detail matrices.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character secure alphanumeric hash identifier key.
- **Generation Engine:** Created automatically by the cloud framework routing rules upon notification document insertion (e.g., 0X69YpNEXxhavGcQrWM5).
- **Storage Pattern:** Flat, append-only history log nodes linked directly to user session targets.

C. Field-Level Structural Rules

Table 3.12: notifications Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
created_at	Timestamp	No	May 19, 2026 at 1:29:40 AM UTC+8	Explicit chronological tracking marker storing the exact time an event log was recorded.
is_read	Boolean	No	true	UI flag tracking whether the user has viewed the alert, filtering dashboard notification badge counts.
message	String	No	"Your order #JPyR9H6T has been accepted"	Clear body string containing the actual text message displayed directly to the end user.
order_id	String	No	"JPyR9H6T1RnGjrqx1org"	Cross-reference value pointing directly back to the matching source document in the orders path.
title	String	No	"Order Accepted"	The bold headline text string displayed at the top of customer layout cards or modal alert bubbles.

type	String	No	"status_update"	Categorization token used by client routing logic to load custom layout icon wrappers (e.g., "status_update", "chat").
user_id	String	No	"KPanLhrWCZfXGDATn4AfH6Ipd1E3"	Target consumer identification string mapping the alert record directly to its owner profile.

D. Operational Flow and Security Considerations

Data entries in the notifications tree employ selective database routing rules to ensure privacy. Clients are granted read and update access solely for documents where the profile string matches their active session token (allow read, update: if request.auth.uid == resource.data.user_id;). This allows users to mark their own notifications as read while blocking third-party interception. Full record additions remain isolated to internal server processes and administrative account actions (request.auth.token.role == 'ADMIN').

3.9.9 Root Collection: orders

A. Subsystem Context and Purpose

The orders collection acts as the primary transaction ledger for the e-commerce framework, preserving completed checkout details, payment processing metrics, and delivery parameters. When a customer completes a checkout loop via the web application, the system converts the temporary cart session into a permanent tracking node inside this collection.

The administration backend reads this path continuously to manage fulfillment pipelines, update order dispatch stages, look up delivery tokens, and verify financial accounting updates. Isolating individual purchases into self-contained document nodes protects transactional histories from changes to the core product inventory.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character secure alphanumeric hash key.
- **Generation Engine:** Generated automatically by the Cloud Firestore SDK upon verified checkout validation loops (e.g., 0JBipwJR1Uop5bHOqBcA).
- **Storage Pattern:** Document structures containing flat primitive fields, a nested metadata map (customer), and a dynamic item array (selected_items).

C. Field-Level Structural Rules

Table 3.13: orders Root-Level Field Schema

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose

amount	Number (Integer)	No	250	Total numeric transaction value calculated for the purchase.
created_at	Timestamp	No	May 14, 2026 at 11:47:40 AM UTC+8	System timestamp tracking exactly when the order record was created.
custom_components	Map / Null	Yes	null	Contains configuration attributes if the record handles a customized project layout.
delivery_date	Timestamp	No	May 15, 2026 at 10:00:00 PM UTC+8	Fulfillment milestone specifying the scheduled receipt date.
delivery_token	String	No	"FJZ8dEV94oxGy7nKhz2wCQ"	Secure alphanumeric verification token used to confirm delivery drop-offs.
delivery_type	String	No	"Pickup"	Mode of logistics fulfillment (e.g., "Pickup", "Delivery").
discount_amount	Number (Integer)	No	0	Deductible balance subtracted if a valid coupon code was applied.
inspo_image	String / Null	Yes	null	Absolute reference link to an external image upload if the user provided design references.
item	String	No	"Pistachio Cheesecake (₱250)"	Summary text string summarizing the primary line items for rapid admin previews.

notes	String	Yes	""	Optional text area string storing specific instructions provided by the buyer.
order_type	String	No	"premade"	Categorization tag separating ready-made menu stock from custom designs (e.g., "premade", "custom").
payment_id	String / Null	Yes	null	External transaction ID reference mapped from the integrated payment gateway.
payment_method	String	No	"Cash on Delivery"	Verification string logging the chosen payment modality (e.g., "Cash on Delivery", "Online Payment").
payment_status	String	No	"Pending"	Core transaction workflow state monitoring collection tracking (e.g., "Pending", "Paid").
reviewed	Boolean	No	true	System toggle tracking whether the client completed a feedback submission for the transaction.
rush	Boolean	No	false	Priority state modifier tracking whether expedited fulfillment processing was requested.
status	String	No	"Completed"	Master administrative status tracking order completion stages (e.g., "Pending", "Accepted", "Completed").
user_id	String	No	"KPanLhrWCZfxGDATn4AfH6Ipd1E3"	Unique cross-reference ID mapping the transaction directly to a user account profile.

voucher_discount	Number (Integer)	No	10	Total percentage or value reduction applied by a voucher match.
voucher_id	String / Null	Yes	"n3Zl2lpz5gFCQdYQsGdH"	Unique identifier string tracking the specific voucher used during checkout.

Table 3.14: orders Nested customer Map Schema

This sub-map organizes contact and delivery details for the buyer inside the single document envelope.

Nested Field Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Operational Purpose
address	String	No	"Pick Up at Shop"	Physical shipping location string or delivery collection directive.
age	String	Yes	""	Optional age attribute provided during checkout.
celebrant	String	Yes	""	Optional descriptive context noting the event celebrant's identity.
contact	String	No	"09123131232"	Verified telephone or mobile number used for logistical notification routing.
lat	Number / Null	Yes	null	Geographical latitude coordinate pin used for delivery address lookup.

lng	Number / Null	Yes	null	Geographical longitude coordinate pin used for delivery address lookup.
name	String	No	"Justin Solamillo"	Full identity name string of the customer receiving the order fulfillment.
occasion	String	Yes	""	Optional event theme description string used for baking customization context.

Table 3.15: orders Nested selected_items Array Schema

This array contains individual object dictionaries for each product included in the transaction.

Array Object Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Operational Purpose
cake_id	String	No	"OVJABOZNjwwYHp7Vbzhv"	Reference ID linking directly to the product source entry in the cakes collection.
cake_name	String	No	"Pistachio Cheesecake"	Product title rendered on order management summaries.
category	String	No	"Dessert"	Catalog grouping label used to help assemble production line tasks.

image_url	String	No	"https://res.cloudinary.com/..."	Absolute Cloudinary CDN resource URL used to display item previews within receipts.
max_qty	Number (Integer)	No	8	Snapshot tracking maximum inventory stock parameters when checkout completed.
price	Number (Integer)	No	250	Item unit cost calculated at the exact historical moment of transaction initialization.
quantity	Number (Integer)	No	1	Total item count purchased within the transaction line.
subtotal	Number (Integer)	No	250	Total subtotal cost matching product quantities against unit values.

D. Operational Flow and Security Considerations

Because information within the orders tree governs financial records and client delivery metrics, access endpoints are strictly isolated using security rules. Customers are granted read privileges exclusive to documents matching their active identity signature (allow read: if request.auth.uid == resource.data.user_id;). Creation tasks are programmatically checked against server timeline rules to make sure requested dates are not flagged as locked. Update permissions for tracking modifiers are blocked entirely unless authenticated with high-level administrative credentials (request.auth.token.role == 'ADMIN').

3.9.10 Root Collection: pending_orders

A. Subsystem Context and Purpose

The pending_orders collection manages active checkout sessions and unverified order configurations. It acts as a temporary holding area where order metadata is staged while awaiting external payment provider confirmation or formal checkout completion.

When a user initiates a checkout flow, the frontend app writes the pending selection details here. Once payment is confirmed by the system's integration hooks, backend processes migrate the document into the core orders collection and clear the pending slot. This design keeps incomplete checkout attempts from cluttering the main transaction logs.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character secure alphanumeric hash key.
- **Generation Engine:** Provisioned automatically by the cloud database layer upon checkout initialization (e.g., 0p8Lh8DIdfSbeA4U9wZq).
- **Storage Pattern:** Structurally identical to the orders collection schema, utilizing flat primitive attributes along with nested customer maps and selected_items object arrays.

C. Field-Level Structural Rules

Table 3.16: pending_orders Root-Level Field Schema

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
amount	Number (Integer)	No	500	The calculated balance value due for the unverified checkout transaction.
created_at	Timestamp	No	May 19, 2026 at 4:32:00 PM UTC+8	Log tracking the exact time the pending checkout instance was first saved.
custom_components	Map / Null	Yes	null	Contains active customizer attributes if tracking a personalized design project.
delivery_date	Timestamp	No	May 22, 2026 at 12:00:00 AM UTC+8	Staged fulfillment date picked by the customer during checkout.
delivery_token	String	No	"qg2sehks6KPanLhrWCZfXG"	Staged verification token string generated for subsequent drop-off authentication.
delivery_type	String	No	"Delivery"	Selected mode of logistics fulfillment (e.g., "Delivery", "Pickup").
discount_amount	Number (Integer)	No	0	Deductible balance reduction applied if an active coupon matches.

inspo_image	String / Null	Yes	null	Reference URL pointing to external layout image file assets provided by the buyer.
item	String	No	"Classic Chocolate Cake (₱500)"	High-level text summary summarizing line items for quick backend table rendering.
notes	String	Yes	"Deliver in the afternoon"	Text area string preserving specific delivery or layout directives from the user.
order_type	String	No	"premade"	Classification tag separating catalog items from custom designs (e.g., "premade", "custom").
payment_id	String / Null	Yes	"pay_xyz123456789"	Transaction path reference ID mapped from the external payment processor gateway.
payment_method	String	No	"Online Payment"	Logging string recording the selected payment pathway (e.g., "Online Payment", "Cash on Delivery").
payment_status	String	No	"Pending"	Workflow variable tracking state changes (e.g., "Pending", "Processing").
reviewed	Boolean	No	false	Default safety bit initialized to false, blocking review tasks before fulfillment.
rush	Boolean	No	false	Priority toggle tracking whether expedited processing surcharges apply.
status	String	No	"Pending"	Administrative status flag tracking staged checkout sessions.

user_id	String	No	"NnD912grZsOAqzVQlMuFEmFgJ1D3"	Core user profile hash mapping the transaction to a registered client.
voucher_discount	Number (Integer)	No	0	Numeric reduction value tracked when matching voucher entities.
voucher_id	String / Null	Yes	null	Unique identifier string tracking an applied voucher document.

Table 3.17: pending_orders Nested customer Map Schema

This sub-map organizes contact and fulfillment data fields for the buyer during checkout staging.

Nested Field Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Operational Purpose
address	String	No	"123 Baker Street, Cebu City"	Physical shipping location string or delivery collection directive.
age	String	Yes	""	Optional age attribute provided during checkout.
celebrant	String	Yes	""	Optional context naming the event celebrant.
contact	String	No	"09171234567"	Telephone or mobile number used for logistical notification routing.
lat	Number / Null	Yes	10.3157	Geographical latitude coordinate pin used for delivery lookup.

lng	Number / Null	Yes	123.8854	Geographical longitude coordinate pin used for delivery lookup.
name	String	No	"Althea Marie Roa"	Full name string of the customer receiving the order.
occasion	String	Yes	""	Optional event theme description string used for baking customization context.

Table 3.18: pending_orders Nested selected_items Array Schema

This array contains individual object dictionaries for each product included in the staged transaction session.

Array Object Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Operational Purpose
cake_id	String	No	"Z8Batr30nbAbX0OTrtpV"	Reference ID linking directly to the product source entry in the cakes collection.
cake_name	String	No	"Classic Chocolate Cake"	Product title rendered on order summaries.
category	String	No	"Dessert"	Catalog grouping label used to help assemble production line tasks.
image_url	String	No	"https://res.cloudinary.com/..."	Absolute Cloudinary CDN resource URL used to display item previews.
max_qty	Number (Integer)	No	5	Snapshot tracking available stock parameters during checkout.

price	Number (Integer)	No	500	Item unit cost calculated at the moment of checkout initiation.
quantity	Number (Integer)	No	1	Total item count purchased within the transaction line.
subtotal	Number (Integer)	No	500	Total subtotal cost matching product quantities against unit values.

D. Operational Flow and Security Considerations

Security controls for pending_orders restrict access vectors based on active user context constraints. Customers retain creation, read, and delete permissions for entries matching their identity signature (allow read, write: if request.auth.uid == resource.data.user_id;). This lets users clear stale cart data or cancel checkout attempts. Update actions on critical transaction values, payment fields, or status definitions are blocked unless authorized by administrative roles or verified server webhook routines.

3.9.11 Root Collection: reviews

A. Subsystem Context and Purpose

The reviews collection serves as the public feedback and rating repository for the storefront web application. It stores user-submitted testimonials, text commentaries, and satisfaction scores linked to verified transactions.

The client application reads from this data tree to render community feedback sections under individual item display cards and compute product scoring distributions. This provides direct social proofing to prospective buyers while allowing business supervisors to monitor overall customer satisfaction indicators from the control panel.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character secure alphanumeric hash key.
- **Generation Engine:** Generated automatically by the Cloud Firestore SDK when a user commits a new feedback entry (e.g., 5WnAnrG3I7T6XbVfPdQs).
- **Storage Pattern:** Flat document structure tracking independent customer feedback events.

C. Field-Level Structural Rules

Table 3.19: reviews Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose

cake_id	String	No	"OVJABOZNjwwYHp7Vbzhv"	Cross-reference identifier mapping the rating directly to a specific item inside the cakes catalog.
cake_name	String	No	"Pistachio Cheesecake"	Descriptive product label preserved within the entry envelope to optimize rendering lists.
comment	String	Yes	"Super sarap! Highly recommended"	Text area value preserving the written open-ended feedback commentary provided by the customer.
created_at	Timestamp	No	May 19, 2026 at 5:10:24 PM UTC+8	Explicit chronological tracking marker storing the exact time the rating was committed to the platform.
customer_name	String	No	"Justin Solamillo"	Text display name mapping the feedback to a public reviewer persona.
order_id	String	No	"JPyR9H6T1RnGjrqx1org"	Unique reference code matching the transaction record to confirm the submission is from a verified buyer.
rating	Number (Integer)	No	5	Numeric score constraint bounding user satisfaction indicators (typically evaluated on a strict scale of 1 to 5).
user_id	String	No	"KPanLhrWCZfxGDATn4AfH6Ipd1E3"	Target account token key associating the record entry back to its originating user profile.

D. Operational Flow and Security Considerations

Security rules within the reviews tree use a balanced permission matrix to protect data integrity. Public browsers are allowed read visibility (allow read: if true;) to seamlessly stream dynamic review text and calculate item ratings.

However, creation tokens are restricted to authenticated users whose active credential matches the document's author descriptor (allow create: if request.auth.uid == request.resource.data.user_id;). This submission process is programmatically verified against the target transaction node to ensure a matching order exists and has not yet been reviewed. Modifying or deleting existing entries is disabled for public clients, restricting those actions to system administrators (request.auth.token.role ==

'ADMIN') to prevent feedback tampering.

3.9.12 Root Collection: users

A. Subsystem Context and Purpose

The users collection serves as the central identity registry and profile repository for all registered accounts within the e-commerce application platform. When a customer creates an account via the frontend portal, the system instantiates a corresponding document within this data path.

The platform reads from this collection to customize client-side greeting interfaces, pre-populate shipping and contact forms during checkout operations, verify access authorization roles, and maintain stable communication channels via logged email headers. By maintaining distinct user metadata fields in an isolated document tree, the database architecture securely decouples sensitive identity properties from dynamic order histories and public review records.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** Fixed alphanumeric unique user identification hash string (UID).
- **Generation Engine:** Provided directly by the Firebase Authentication service upon successful registration to guarantee exact matching configurations across authentication modules (e.g., 0bYQ3E67fDPlmCrsTz6X).
- **Storage Pattern:** Flat root reference nodes archiving individual user metadata attributes.

C. Field-Level Structural Rules

Table 3.20: users Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
address	String	Yes	"123 Baker Street, Cebu City"	The default physical residence or shipping destination string stored to simplify checkout loops.
contact	String	Yes	"09171234567"	Primary phone or mobile number utilized for shipping coordination and dispatch tracking updates.
created_at	Timestamp	No	April 15, 2026 at 2:14:05 PM UTC+8	Chronological system log capturing the exact date and time the user account was first registered.
email	String	No	"user.test@gmail.com"	Authenticated electronic mail path used for system sign-ins, billing confirmations, and data lookups.

name	String	No	"Jane Doe"	Full legal or display identity string rendered across public profile spaces and order invoice documents.
role	String	No	"CUSTOMER"	System access control string variable assigning interface privileges across the platform (e.g., "CUSTOMER", "ADMIN").

D. Operational Flow and Security Considerations

Information stored inside the users tree governs account roles and sensitive customer data. Global data collection queries are blocked by default to prevent credential scraping. Under active Firestore Security Rules, logged-in clients can read or modify data properties only if their active session token matches the document ID (allow read, update: if request.auth.uid == userId;).

Crucially, critical privilege fields like role are completely locked against public client mutations. Escalating a user profile to administrative status requires a direct database update through the system console or an authorized backend script using a verified administrator token (request.auth.token.role == 'ADMIN').

3.9.13 Root Collection: walkin_orders

A. Subsystem Context and Purpose

The walkin_orders collection serves as the offline sales ledger, tracking over-the-counter purchases made physically at the bakery storefront. By isolating physical storefront retail transactions into a dedicated data tree, the system keeps regular digital cart checkouts separate from point-of-sale operations.

The administration panel reads from this path to aggregate overall operational revenue and update cross-channel material balances. This design ensures that walk-in stock deductions immediately sync with the centralized inventory levels, preventing online buyers from reserving raw materials that have already been consumed by physical storefront sales.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character secure alphanumeric hash key.
- **Generation Engine:** Instantiated automatically by the Cloud Firestore SDK upon point-of-sale receipt confirmation (e.g., 1E7bZfQDoXPlmCrsTz7W).
- **Storage Pattern:** Structured documents composed of flat primitive attributes and an embedded item array (selected_items).

C. Field-Level Structural Rules

Table 3.21: walkin_orders Root-Level Field Schema

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
amount	Number (Integer)	No	500	The total gross monetary value collected over the counter for the physical transaction.
created_at	Timestamp	No	May 25, 2026 at 3:12:18 PM UTC+8	Explicit chronological tracking marker storing the exact time the sale was logged at the register.
customer_name	String	No	"Walk-in Customer"	Standardized default identity placeholder used to denote unregistered physical shop buyers.
discount_amount	Number (Integer)	No	50	Deductible cash value subtracted directly at the point of sale for manual promotional overrides.
item	String	No	"Classic Chocolate Cake (₱500)"	Concise summary line used by administrative dashboard viewports for rapid invoice indexing.
payment_method	String	No	"Cash"	Verification indicator recording physical payment modalities (e.g., "Cash", "E-Wallet").
payment_status	String	No	"Paid"	Financial workflow state. For walk-in sales, this defaults immediately to "Paid".
status	String	No	"Completed"	Core tracking variable monitoring fulfillment execution (e.g., "Completed", "Cancelled").
voucher_discount	Number (Integer)	No	10	Total structural percentage or value reduction applied via physical token scanning.

voucher_id	String / Null	Yes	null	Reference identity string used if an active discount document was redeemed.

Table 3.22: walkin_orders Nested selected_items Array Schema

This array stores individual object maps for every item bundled within the physical point-of-sale transaction.

Array Object Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Operational Purpose
cake_id	String	No	"Z8Batr30nbAbX0OTrtpV"	Core reference identifier matching the product source node within the main cakes collection.
cake_name	String	No	"Classic Chocolate Cake"	Descriptive retail item label rendered across administrative sales reports.
category	String	No	"Dessert"	Product sorting classification used to balance daily bakery production summaries.
image_url	String	No	"https://res.cloudinary.com/..."	Resource asset link used to display item thumbnail graphics within the administrative history logs.
max_qty	Number (Integer)	No	12	Snapshot variable capturing the maximum available warehouse stock at the time of purchase.

price	Number (Integer)	No	500	Baseline unit retail cost of the product at the precise moment of physical checkout.
quantity	Number (Integer)	No	1	Total physical unit count bought by the customer during the checkout event.
subtotal	Number (Integer)	No	500	Line item total cost calculated by multiplying the product unit value by the purchase quantity.

D. Operational Flow and Security Considerations

Because entries in the walkin_orders path directly represent realized store revenue and alter core inventory balances, their access rights are tightly restricted. Public web clients are completely blocked from reading or querying this path.

Under the system's Firestore Security Rules, write mutations and historical search queries are granted exclusively to authenticated administrative accounts (request.auth.token.role == 'ADMIN'). This security layer prevents external users from tampering with storefront revenue ledgers or falsifying physical sales records.

3.9.14 Root Collections: conversation

A.Subsystem Context and Purpose

The conversations collection serves as the central communication ledger within the cloud database ecosystem, specifically designed to store, manage, and synchronize messaging threads between customers and bakeshop administrators. Rather than relying on unintegrated external social media applications, the custom cake portal handles all customer inquiries, design modifications, custom cake quote requests, and order status consultations directly through this dedicated real-time infrastructure.

By centralizing message thread metadata within this collection, background database pipelines feed real-time chat lists directly to the administrative communications portal. This enables bakeshop staff to instantly identify unread customer inquiries, flag complex orders that require direct administrative escalation, review automated cake consultation summary payloads, and sort active customer dialogues chronologically based on recent updates without incurring the high database read overhead of fetching entire message history logs.

B. Document ID Structure & Storage Mechanics

Document ID Type: Composite semantic key string containing a static prefix, epoch timestamp, and alphanumeric hash string (e.g., conv_1787723646378_5ufa86lgw).

Generation Engine: Automatically generated by client-side application controllers or backend cloud functions during thread initialization whenever a customer initiates a chat or completes a structured cake consultation workflow.

Storage Pattern: Stored as a flat root collection consisting of top-level conversation documents, where each document serves as a parent node containing high-level thread metadata and state flags, while hosting a nested messages subcollection for granular chronological message histories.

C. Field-Level Structural Rules

Table 3.23: conversations Field Schema

Field Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Operational Purpose
consultation_data	Map / Object	Yes	{"rush": "", "others_desc": ""}	Embedded map containing structured parameters collected during custom cake consultations.
conversation_id	String	No	"conv_1787723646378_5ufa86lgw"	System key string matching the document ID used to reference the conversation thread.
customer_name	String	No	"Justin Solamillo"	Full name of the customer participating in the chat thread.
email	String	No	"jual.solamillo.ui@phinmaed.com"	Contact email address associated with the customer account.
escalated	Boolean	No	true	Flag indicating whether the conversation requires direct admin intervention.

is_consultation	Boolean	No	true	Boolean flag determining if the thread originated from a cake consultation flow.
last_message	String	No	"[Consultation] Vanilla, 8 inches (P2300)..."	Text snippet capturing the most recent message sent in the thread.
last_updated	Timestamp	No	September 16, 2026 at 3:03:19 PM UTC+8	Server timestamp recording the latest activity time to sort chat threads.
unread	Boolean	No	true	Status flag indicating whether the latest message has not been viewed by staff.
user_id	String	No	"Tqt7OUsVnlTjE4qx5ebJZQGoArd2"	Unique account identifier string mapping the conversation to the customer.

D. Operational Flow and Security Considerations

Data assets stored under the conversations collection are governed by strict identity-based access control policies within server-side Firestore Security Rules to guarantee customer privacy. Unauthenticated public visitors cannot read or write to any conversation records.

Authenticated customers are granted read and update access exclusively to conversation documents where their unique Firebase Authentication user ID matches the user_id field within the document (request.auth.uid == resource.data.user_id). Meanwhile, full read, query, update, and management privileges across all active and historical conversation threads are reserved strictly for authenticated system accounts holding verified administrative security tokens (request.auth.token.role == 'ADMIN').

3.9.15 Root Collection: pending_consultations

A. Subsystem Context and Purpose

The pending_consultations collection functions as an intermediate transaction staging layer designed to manage temporary custom cake inquiry tokens and consultation links. When a customer completes a custom cake design consultation workflow, the system generates a time-bound consultation token stored within this collection. This mechanism allows customers to save their custom cake selections, inspiration parameters, and quote details temporarily before finalizing their order or initiating a direct administrative chat thread.

By enforcing expiration lifespans and usage state flags on these documents, the platform prevents stale or abandoned consultation sessions from cluttering live operational queues. Furthermore, storing these staging parameters centrally allows the application to verify token validity, check expiration status, and link active consultation sessions directly to user account profiles and active messaging threads.

B. Document ID Structure & Storage Mechanics

Document ID Type: 20-character, cryptographically secure alphanumeric unique identifier hash string.

Generation Engine: Automatically generated client-side or via cloud functions upon execution using the Google Cloud Firestore SDK (e.g., 4sW6oIMgyXeUt4sx2fjM250CQVbIlE3II86ORog6pjY).

Storage Pattern: Flat root-level collection holding temporary transaction documents. Once a consultation token is used or reaches its expiration timestamp, background cleanup jobs or security rules handle access restriction and record lifecycle management.

C. Field-Level Structural Rules

Table 3.24: pending_consultations Field Schema

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Purpose & Constraints
consultation_data	Map / Object	Yes	{"flowers": "", "inspo_ima..."}	Embedded map containing temporary cake design specs and inspiration parameters.
conv_id	String	No	"conv_1789197822472_5beda4d3"	Reference string linking the consultation to its associated chat thread.
created_at	Timestamp	No	September 12, 2026 at 3:24:22 PM UTC+8	Server clock marker recording the exact moment the consultation was created.

expires_at	Timestamp	No	September 19, 2026 at 3:24:22 PM UTC+8	Expiration timestamp defining the valid lifecycle window for the token.
used	Boolean	No	true	Flag indicating whether the consultation token has already been redeemed.
used_at	Timestamp	Yes	September 12, 2026 at 3:26:23 PM UTC+8	Timestamp recording when the consultation token was successfully processed.
user_id	String	No	"IJtzHVjKevWDYseH4dlrNNXCY9M2"	Firebase Authentication user ID mapping the token to the customer account.

D. Operational Flow and Security Considerations

Security policies applied to pending_consultations enforce strict identity mapping to protect user customization data. Unauthenticated public requests are entirely denied read and write permissions.

Authenticated customers are restricted to reading and modifying only those consultation staging tokens where the document user_id field matches their authenticated identity token (request.auth.uid == resource.data.user_id). System administrators possess full administrative rights (request.auth.token.role == 'ADMIN') to inspect, query, and invalidate pending consultation entries during order verification processes.

3.9.16 Root Collection: webauthn_credentials

A. Subsystem Context and Purpose

The webauthn_credentials collection serves as the secure cryptographic storage layer for WebAuthn passkey authentication within the system. Instead of relying solely on traditional password-based logins, this collection stores hardware-bound public keys, credential IDs, and signature counters that enable biometric and hardware key authentication (such as Touch ID, Face ID, or Windows Hello).

By storing public keys and signature metadata separately from core user account records, the platform supports multi-factor authentication and passwordless sign-in workflows. During authentication, the backend verifies cryptographically signed challenges against stored public keys while checking signature counters to prevent replay attacks.

B. Document ID Structure & Storage Mechanics

Document ID Type: Firebase Authentication User Unique Identifier string matching the authenticated user's account ID (e.g., Tqt7OUsVnlTjE4qx5ebJZQGoArd2).

Generation Engine: Explicitly assigned during WebAuthn passkey registration using the primary user account UID to maintain a direct 1:1 document mapping per user.

Storage Pattern: Flat root-level collection where each user document contains an array of registered credential objects, allowing users to enroll and manage multiple biometric authenticators or security keys under a single account node.

C. Field-Level Structural Rules

Table 3.25: webauthn_credentials Field Schema

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Purpose & Constraints
credentials	Array (Objects)	No	[{"created_at": "2026-09-14T19:17:27.944234+08:00", ...}]	List of registered WebAuthn credential objects associated with the user account.
credentials[].created_at	String (ISO 8601)	No	"2026-09-14T19:17:27.944234+08:00"	ISO 8601 formatted timestamp recording when the passkey credential was registered.
credentials[].credential_id	String	No	"wSF3XLuhO7wVqKdQylTPGQO3OF8="	Unique Base64-encoded identifier string issued by the authenticator during registration.
credentials[].public_key	String	No	"pQECAyYgASFYIBP5NpeuOREgdTqz8QM0C7kJkPr9u+6TP4q4E2b5U..."	Public key string used by the server to verify signed cryptographic authentication challenges.
credentials[].sign_count	Number	No	0	Signature counter incremented on each authentication to detect and prevent credential cloning/replay attacks.
user_id	String	No	"Tqt7OUsVnlTjE4qx5ebJZQGoArd2"	Firebase Authentication user ID identifying the owner of the registered biometric credentials.

D. Operational Flow and Security Considerations

Security rules governing webauthn_credentials enforce strict identity checks to ensure sensitive public key infrastructure data remains protected. Public access without authentication is completely blocked.

Authenticated users can only read and write to credential documents matching their own Firebase Authentication UID (request.auth.uid == resource.id). Direct modification of individual fields like sign_count is typically handled through server-side Cloud Functions or Admin SDK calls to ensure cryptographic integrity during passkey validation sessions.

3.9.17 Root Collection: settings

A. Subsystem Context and Purpose

The settings collection acts as the centralized system configuration repository for application-wide operational rules and store management settings. Instead of hardcoding business logic and operational parameters directly into frontend components or backend scripts, this collection stores dynamic control flags, operating hours, and system parameters that govern platform behavior in real time.

By storing key configuration parameters—such as store opening and closing times, emergency override switches, and audit tracking markers—in a dedicated settings repository, administrative personnel can instantly update store availability across all customer interfaces without requiring application redeployments. The customer checkout interface and order intake workflows continuously observe these settings documents to dynamically enable or restrict pickup and delivery slots based on active operating schedules.

B. Document ID Structure & Storage Mechanics

Document ID Type: Fixed operational configuration domain key string matching the underlying system module (e.g., shop_hours).

Generation Engine: Manually created or initialized during application bootstrap setup using specific functional keys.

Storage Pattern: Document-per-domain configuration store. Each document in this collection represents a specific subsystem configuration module, eliminating redundant reads and organizing settings by functional domain.

C. Field-Level Structural Rules

Table 3.26: settings Field Schema

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Purpose & Constraints
close_time	String	No	"21:00"	Standard daily store closing time in 24-hour format.

manual_override	Null / Boolean	Yes	null	Flag to manually force store open or closed status regardless of schedule.
open_time	String	No	"10:00"	Standard daily store opening time in 24-hour format.
override_reason	String	Yes	"emergency"	Text explanation justifying active manual operational overrides.
updated_at	Timestamp	No	September 16, 2026 at 9:20:56 PM UTC+8	Server clock marker tracking when settings were last modified.
updated_by	String	No	"Tqt7OUsVnlTjE4qx5ebJZQGoArd2"	Firebase Authentication user ID of the admin who saved changes.

D. Operational Flow and Security Considerations

Security controls for the settings collection enforce strict read-only access for public client interfaces while restricting modifications solely to administrative accounts. Unauthenticated and authenticated customer client applications are granted read permissions to inspect operating hours and store availability flags.

Write, update, and delete privileges are exclusively restricted to verified administrative users (request.auth.token.role == 'ADMIN'). Security rules validate that modifications include valid admin user IDs and server timestamps, preserving administrative accountability whenever business operating rules are updated.

3.9.18 Root Collection: loyalty_gifts

A. Subsystem Context and Purpose

The loyalty_gifts collection acts as the dynamic reward catalog repository for the platform's customer loyalty and incentives subsystem. Instead of hardcoding redeemable rewards, gift tiers, and promotional items directly into the client application, this collection manages available gift choices categorized by tier size (e.g., small, big).

By storing gift reward options in Firestore, administrative staff can update available gifts, rotate seasonal items, and alter reward offerings in real time without redeploying frontend code. During customer checkout or milestone redemption events, the mobile and web interfaces retrieve these configured lists to dynamically populate eligible reward selection menus based on the user's earned loyalty status.

B. Document ID Structure & Storage Mechanics

Document ID Type: Auto-generated alphanumeric string issued by Firestore (e.g., RT2FfTszLgss2XbcWG3A).

Generation Engine: Created via standard Firestore document reference generation when initializing or creating loyalty reward tier configurations.

Storage Pattern: Document-per-tier configuration store. Each document holds array configurations corresponding to reward categories/tiers, keeping catalog lookups lightweight and easy to retrieve.

C. Field-Level Structural Rules

Table 3.27: loyalty_gifts Field Schema

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Purpose & Constraints
big	Array (Strings)	No	["Bento Cake", "Slice Cake", "Drinks Bundle"]	List of higher-tier or major gift items available for redemption.
big[]	String	No	"Bento Cake"	Specific item name belonging to the major gift tier.
small	Array (Strings)	No	["Cupcake", "Coffee", "Pastry"]	List of lower-tier or minor gift items available for redemption.
small[]	String	No	"Cupcake"	Specific item name belonging to the minor gift tier.

D. Operational Flow and Security Considerations

Security controls for the loyalty_gifts collection ensure public accessibility for reward browsing while maintaining strict administrative oversight for catalog edits. Unauthenticated and authenticated users are granted read access to display active

reward choices across customer interfaces.

Write, update, and delete privileges are strictly restricted to authenticated administrative accounts (request.auth.token.role == 'ADMIN'). Security rules ensure users cannot modify reward options during redemption requests, enforcing that all reward selection validation happens against server-side configurations.

3.9.19 Expanded Data Dictionary – Additional Fields for orders Collection

Field	Type	Nullable	Description	Constraints
delivery_fee	Number	No	Computed delivery fee based on distance.	Must be $\ge$ 0.
distance_km	Number	Yes	Distance in kilometers from shop to delivery address.	Null if pickup.
rush_fee	Number	No	Additional fee for rush orders.	Default 0.
voucher_discount	Number	No	Discount amount from voucher.	Default 0.
loyalty_stamps_earned	Number	No	Number of stamps earned from this order.	Default 1.
loyalty_stamps_redeemed	Number	No	Number of stamps redeemed.	Default 0.
special_instructions	String	Yes	Customer's special instructions.	Max 500 chars.

allergy_notes	String	Yes	Allergy warnings.	Max 300 chars.
reference_image_url	String	Yes	Cloudinary URL of uploaded reference image.	Must be valid URL.
payment_reference	String	Yes	PayMongo payment ID.	Null if COD.
webhook_verified	Boolean	No	Whether payment was verified via webhook.	Default false.
admin_notes	String	Yes	Internal notes from admin.	Max 500 chars.
cancellation_reason	String	Yes	Reason for cancellation.	Required if status = Cancelled.

3.10 Feasibility Study

Before full-scale development, the researchers conducted a feasibility study to determine whether the proposed system was viable across technical, operational, economic, and schedule dimensions. This section presents the findings of that assessment.

3.10.1 Technical Feasibility

The technical feasibility assesses whether the required technology exists and whether the development team has the necessary skills to implement the system. The proposed system relies on a modern, serverless web stack: Python (Flask) for backend logic, Google Firestore for NoSQL data storage, Firebase Authentication for identity management, PayMongo for payments, Cloudinary for image hosting, and Render for cloud deployment. All of these technologies are mature, well-documented, and widely used in production environments. The development team, composed of BSIT students, had prior coursework in Python, web development, and database management. The team also utilized free tiers of cloud services, eliminating the need for capital expenditure on servers or licenses. The technical feasibility is therefore rated as high.

3.10.2 Operational Feasibility

Operational feasibility evaluates how well the proposed system solves business problems and whether it will be accepted by users. The system was designed around the actual daily workflow of Mrs. Brave's Cakes, as documented through interviews and direct observation. It does not require Mrs. Tapang to change her core baking process; instead, it digitizes the surrounding

administrative tasks (order taking, pricing, scheduling, inventory, dispatch). The interface uses familiar terms and simple interactions, reducing the learning curve. The UAT results (mean 4.79, Excellent) confirm that the system is both useful and easy to use. The operational feasibility is rated as high.

3.10.3 Economic Feasibility

Economic feasibility analyzes the costs and benefits of the system. The development costs were primarily labor (student time) and the use of free-tier cloud services. The ongoing operational costs are minimal: Firestore's free tier supports up to 50,000 reads and 20,000 writes per day, which is more than sufficient for a micro-bakery. Render's free tier provides hosting, and Cloudinary's free tier offers 25 GB of storage. PayMongo charges a transaction fee (approximately 3.5% + ₱15 per transaction), but this is comparable to or lower than the fees charged by third-party delivery apps (which can be 20–30%). The system is expected to reduce losses from ingredient waste, payment fraud, and overbooking, while increasing sales through improved customer experience. The economic feasibility is rated as high.

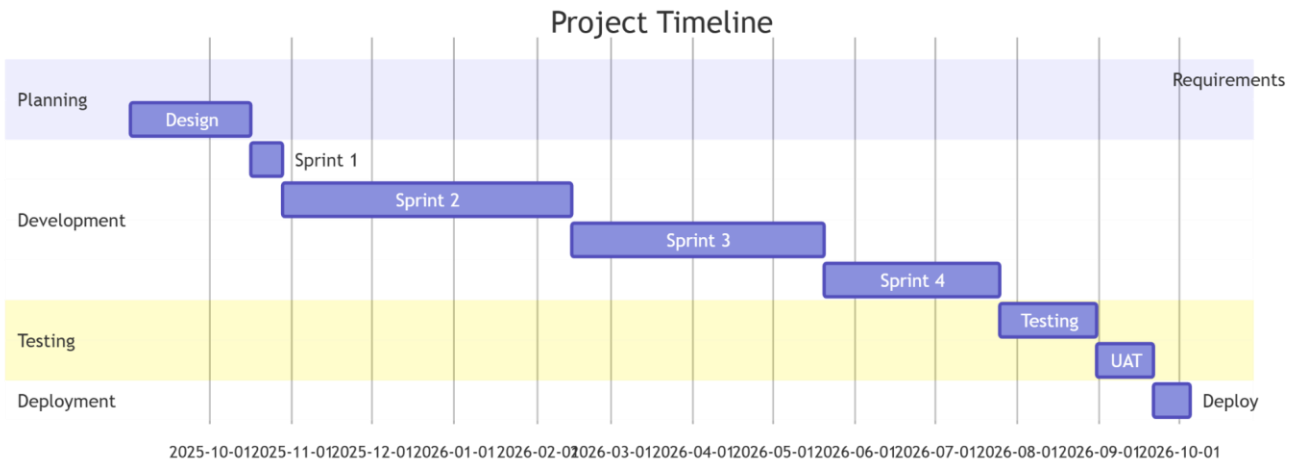
3.10.4 Schedule Feasibility

Schedule feasibility determines whether the project can be completed within the allotted time. The capstone project spanned two semesters. The development was broken into Agile sprints, with each sprint lasting two to three weeks. The team followed a Gantt chart (see Section 3.11.1) and completed all major milestones on time. The final system was deployed and subjected to UAT before the deadline. The schedule feasibility is rated as high.

3.11 Project Management

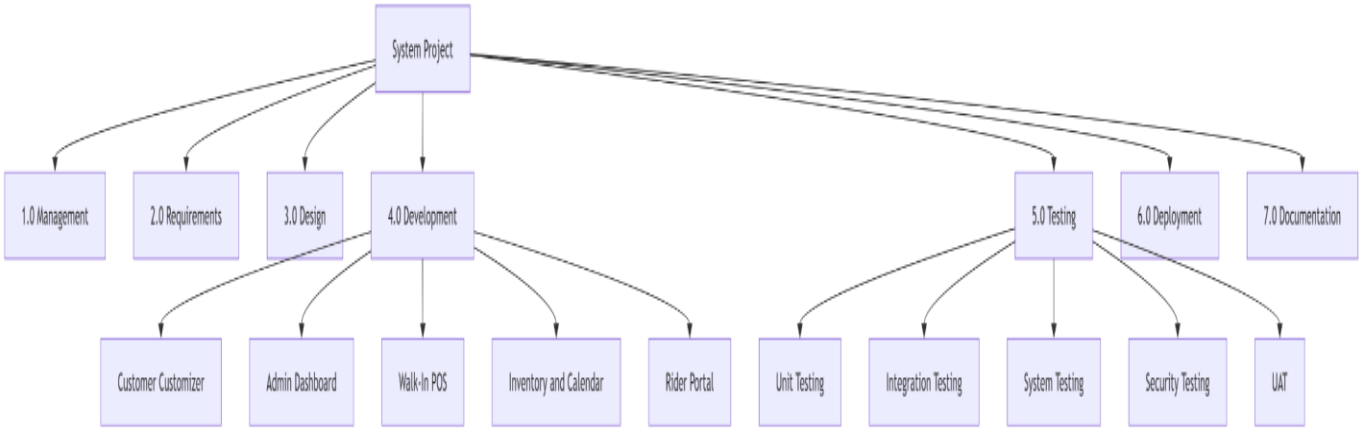
This section documents the project management tools and techniques used to plan, track, and control the development of the Streamlined Bakery Management System.

3.11.1 Gantt Chart



The Gantt chart illustrates the chronological breakdown of project activities. The project followed an Agile timeline, with overlapping phases to allow for iterative feedback. Requirements gathering and system design were completed in the first month. Development was divided into four sprints: Sprint 1 (Customer Customizer), Sprint 2 (Admin Dashboard and POS), Sprint 3 (Inventory and Booking Calendar), Sprint 4 (Rider Portal and Payment Integration). Testing and UAT were conducted in the final month. Documentation was written concurrently with development.

3.11.2 Work Breakdown Structure (WBS)



The WBS decomposes the project into manageable deliverables. Project Management includes sprint planning, risk management, and communication. Requirements includes interviews, observation, and documentation. Design includes UI/UX wireframes, database schema, and architecture diagrams. Development includes the five core modules. Testing includes unit, integration, system, security, and performance testing. Deployment includes Render hosting, Firebase setup, and domain configuration.

3.11.3 Risk Register

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy	Owner
R1	PayMongo API downtime or changes	Medium	High	Implement fallback cash on delivery; monitor API status; use webhook retry logic.	Lead Programmer
R2	Firestore quota exceeded	Low	Medium	Use caching; optimize queries; upgrade to paid tier if necessary.	Lead Programmer
R3	Mrs. Tapang resists adoption	Medium	High	Involve her in design; conduct training; provide simple user manual.	Project Manager
R4	Power outage during development	Medium	Medium	Use laptops with battery; commit code frequently to GitHub.	All Developers

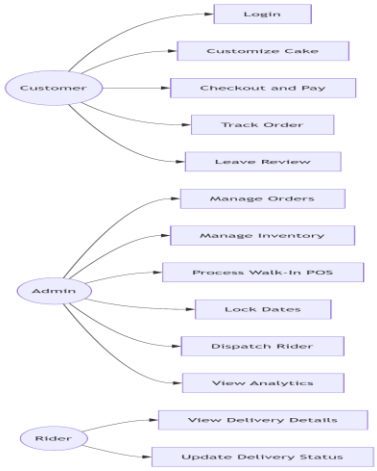
R5	Hardware failure	Low	High	Use cloud backups; share workstations; use Git co-authoring.	All Developers
R6	Scope creep	High	Medium	Freeze requirements after Sprint 2; defer non-critical features to future work.	Project Manager
R7	Data breach	Low	High	Use Firebase Security Rules; never store sensitive financial data; use HTTPS.	Lead Programmer

3.11.4 Communication Plan

Stakeholder	**Information**	**Method**	**Frequency**	**Owner**
Mrs. Tapang	Progress updates, feedback requests	Face-to-face, Messenger	Weekly	Project Manager
Adviser	Milestone reports, manuscript drafts	Email, consultation	Bi-weekly	Documentation Lead
Development Team	Task assignments, blockers	Group chat, stand-up meetings	Daily	Project Manager
End Users (Customers)	System availability, new features	Facebook page, email	As needed	UI/UX Designer

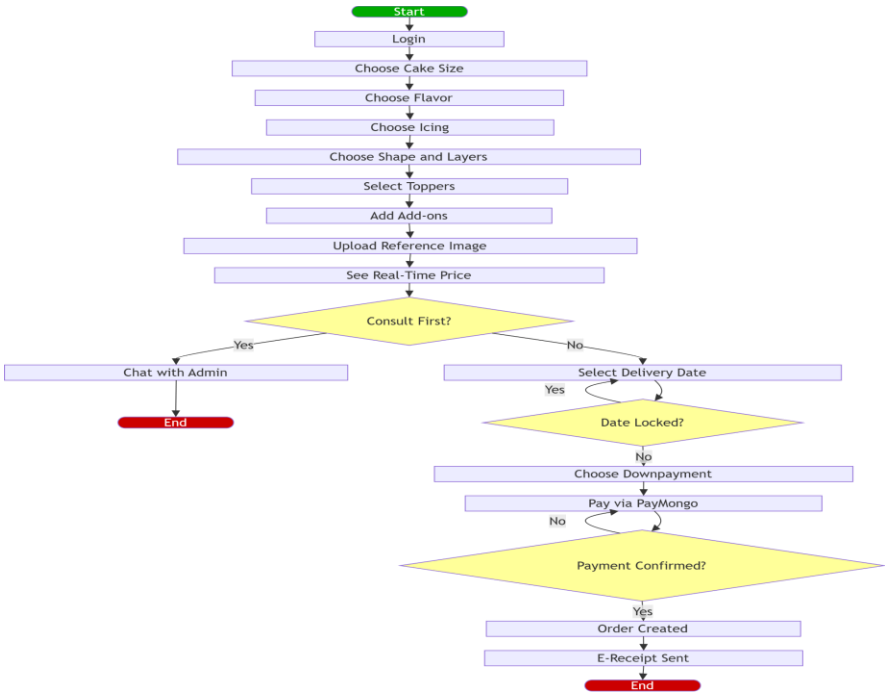
3.12 Additional UML Diagrams

This section provides supplementary UML and architectural diagrams that document the system's structure and behavior beyond the sequence diagrams presented in Section 3.8.



The use case diagram provides a high-level view of system functionality from the perspective of external actors. It defines the boundaries of the system and clarifies which actors have access to which features. The Admin actor inherits all customer use cases (since the admin can also place orders) but has exclusive access to management functions.

3.12.2 Activity Diagram – Cake Customization

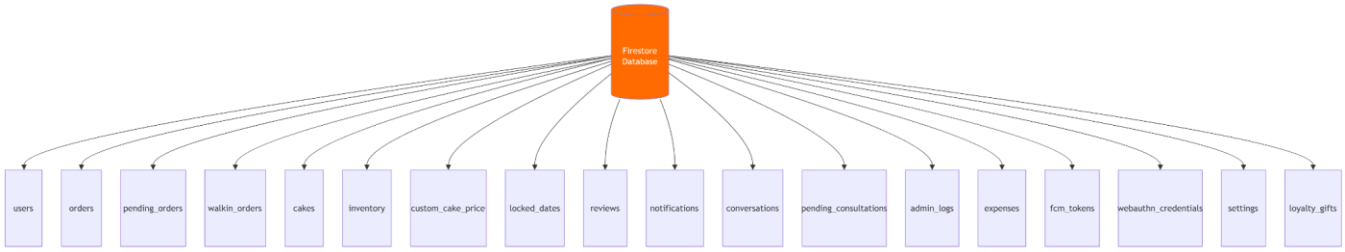


The activity diagram for cake customization illustrates the sequential flow of user actions and system responses. It highlights the decision points (e.g., date availability check, Consult First option, downpayment selection) and the parallel processing of price calculation. This diagram was used to design the frontend state management logic.

3.12.3 Firestore Collection Diagram

Because the system uses a NoSQL document database rather than a relational database, a traditional Class Diagram would not accurately represent the data architecture. Instead, the following Firestore Collection Diagram shows the actual 18 root

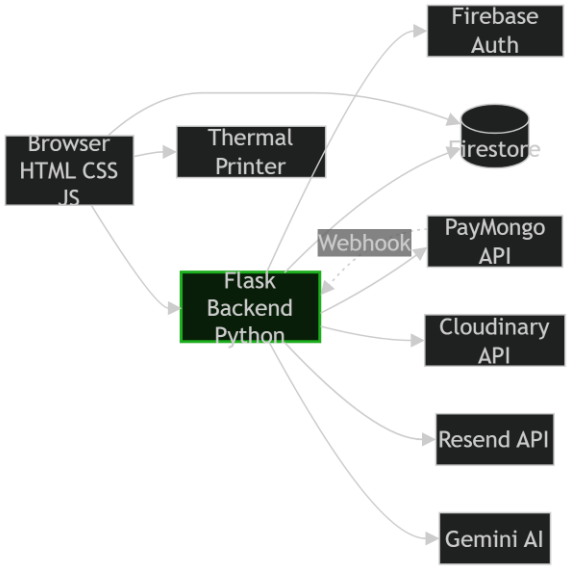
collections used in the system.



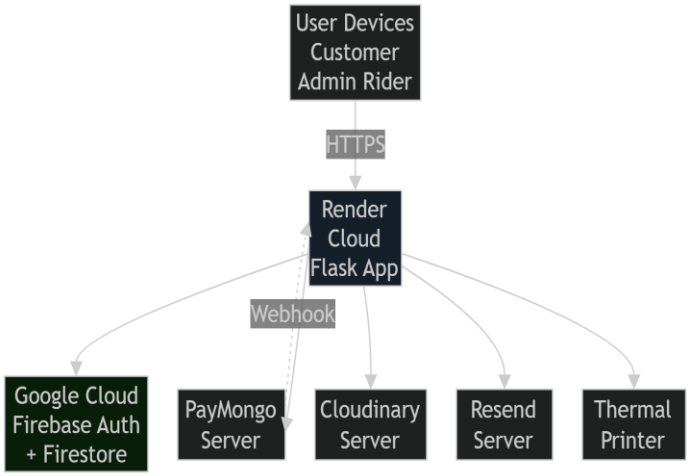
This diagram shows the 18 root collections in the system's Google Firestore database. Unlike a traditional Entity-Relationship Diagram, this structure reflects Firestore's document-oriented model, where related data (e.g., customer details, selected items) is embedded directly inside each order document rather than stored in separate normalized tables. This design eliminates the need for expensive table joins and supports real-time synchronization across all system modules.

3.12.4 Component Diagram

The component diagram shows the modular decomposition of the system and the dependencies between components. It emphasizes the decoupled nature of the architecture: the frontend communicates with the backend via HTTP requests; the backend communicates with external services via APIs. This modularity allows individual components to be updated or replaced without affecting the entire system.



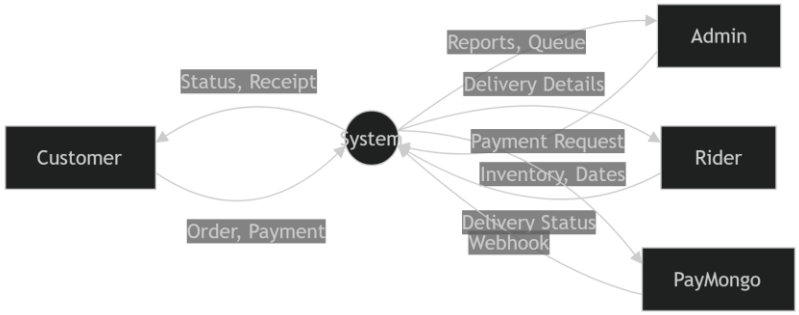
3.12.5 Deployment Diagram



The deployment diagram illustrates the physical distribution of system components across servers. The Flask application is hosted on Render, which provides automatic scaling and HTTPS. Firestore and Firebase Authentication are hosted on Google Cloud. PayMongo, Cloudinary, Resend, and Gemini AI are third-party services. This serverless deployment eliminates the need for the bakery to maintain physical servers.

3.12.6 Data Flow Diagram (DFD)

CONTEXT DIAGRAM (LEVEL 0)



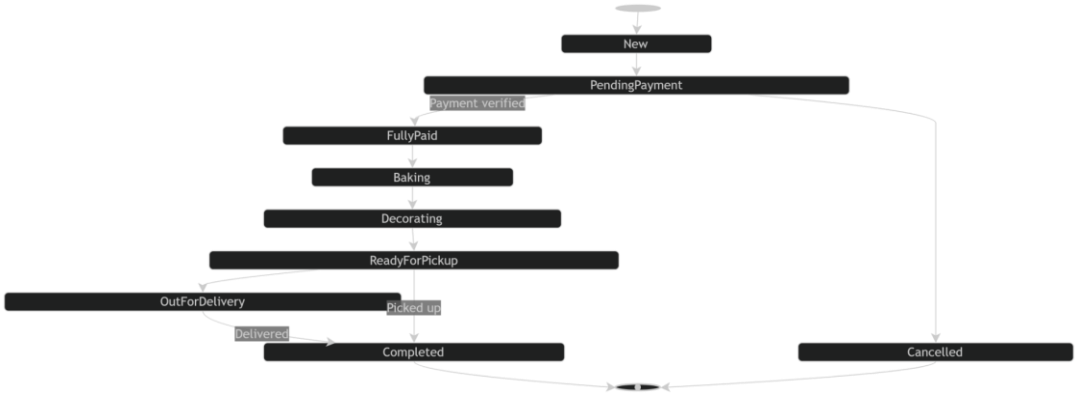
LEVEL 1 DFD



The DFDs provide a functional view of the system, showing how data moves between external entities, processes, and data

stores. The context diagram defines the system boundary. The level 1 DFD decomposes the system into major processes, each corresponding to a module described in Chapter 4. These diagrams were used to validate that all data inputs and outputs were accounted for.

3.12.7 State Transition Diagram – Order Status



The state transition diagram models the lifecycle of an order. It defines the valid states and the events that trigger transitions. This diagram was used to implement the order status workflow in the admin dashboard. It ensures that orders cannot skip critical steps (e.g., an order cannot be marked "Completed" without first being "Fully Paid").

3.13 Summary and Conclusion

This chapter presented the comprehensive methodology employed in the design, development, and evaluation of the Streamlined Bakery Management System for Mrs. Brave's Cakes. The study adopted a developmental research design, which was specifically chosen to bridge the gap between theoretical analysis and the practical creation of a functional software solution. By structuring the entire project around the Agile Development Model, the researchers were able to deliver the system through iterative sprints, allowing continuous feedback from the business owner to be incorporated without disrupting the overall development timeline.

The architectural design of the system was carefully documented through three distinct layers: the presentation layer, the cloud infrastructure and database layer, and the external integration layer. This modular structure ensured that the system remained scalable, secure, and easy to maintain. The selection of Google Firestore as the primary NoSQL database was justified by the highly variable nature of custom cake orders, while Firebase Authentication, PayMongo, Cloudinary, and Resend provided the necessary security, payment, media, and communication capabilities required by the platform.

The database architecture was exhaustively documented through a complete data dictionary covering 18 root collections, each with clearly defined field-level schemas, document ID structures, and security rules. The accompanying sequence diagrams, UML diagrams, and supporting process flows provided a comprehensive technical blueprint for the system's behavior. In addition, the feasibility study confirmed that the project was viable across technical, operational, economic, and schedule dimensions, while the project management section established clear plans for risk mitigation, communication, and timeline tracking.

To ensure the reliability and integrity of the system prior to deployment, the researchers conducted a multi-level testing procedure covering unit testing, system testing, and User Acceptance Testing (UAT), with evaluation criteria anchored on the ISO/IEC 25010 software quality standard. The data gathering techniques—semi-structured interviews and direct observation—ensured that the requirements were derived directly from the real operational environment of Mrs. Brave's Cakes, rather than from generic assumptions about bakery management.

In conclusion, this chapter established a rigorous, systematic, and well-documented methodology that guided every stage of the system's development. The combination of developmental research design, Agile methodology, cloud-native architecture, and internationally recognized evaluation standards ensured that the resulting system was not only technically sound but also operationally aligned with the actual needs of the bakery. The following chapter, Chapter 4, presents the full implementation, testing results, and evaluation outcomes of the completed system based on the methodology described here.

CHAPTER 4

SYSTEM DEVELOPMENT, IMPLEMENTATION, TESTING, EVALUATION, RESULTS, AND DISCUSSION

4.1 Overview of the Developed System

The Streamlined Bakery Management System is a web-based application designed specifically for Mrs. Brave's Cakes to centralize cake customization, order management, walk-in sales, inventory tracking, scheduling, and last-mile delivery coordination into a single platform. The system was developed to replace the bakery's previous reliance on unstructured Facebook Messenger transactions and a manual, paper-based booking calendar, both of which created recurring problems in data tracking, scheduling accuracy, and financial oversight, as discussed in Chapter 1.

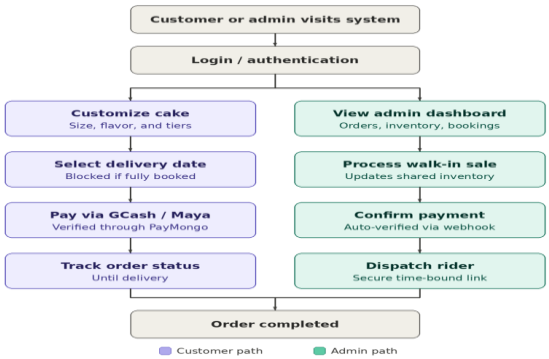
Unlike generic e-commerce templates, the system was purpose-built around the specific operational needs of a custom-order bakery: it accommodates highly variable, client-driven product specifications (tiers, flavors, designs, reference-photo uploads) rather than a fixed product catalog, while also unifying online and walk-in sales under a single inventory ledger. The system was designed around the actual day-to-day workflow of Mrs. Tapang's business rather than requiring her to adapt her process to a generic template — a design principle informed directly by the literature reviewed in Chapter 2 regarding non-technical operator adoption.

The major objectives of the developed system, consistent with the specific problems and objectives identified in Chapter 1, are as follows:

- Automate and centralize the recording of cake customization orders, replacing manual chat-based negotiation.
- Provide real-time inventory synchronization between online orders and walk-in Point-of-Sale transactions.
- Prevent booking conflicts and over-scheduling through an automated, capacity-aware calendar.
- Verify customer payments automatically through a secure payment gateway rather than manual screenshot review.
- Support last-mile delivery coordination through a lightweight, secure rider dispatch interface.
- Provide the business owner with a data-driven view of ingredient demand and sales trends.

The workflow begins with user authentication, distinguishing between the administrator (Mrs. Tapang) — who handles inventory, bookings, and walk-in POS transactions from the same admin account — and customer-facing access. Authenticated customers may browse the cake customization interface, place and track orders, and complete payment through the integrated gateway. The administrator can manage inventory, monitor bookings through the Blackout Date Controller, process walk-in transactions, and dispatch confirmed orders to riders. Each of these workflows is described in full detail in Section 4.3.

Fig. 4.1 System Overview



4.2 System Development Methodology

Following the methodology laid out in Chapter 3, the development of the Streamlined Bakery Management System was carried out using the Agile Software Development framework. The project was broken down into short, iterative sprints that allowed for continuous testing and regular check-ins with the business owner, Mrs. Shamelyn Tapang. Sticking to an Agile approach during the actual implementation phase was crucial because it gave the team the flexibility to tweak specific features based on real-time user feedback. This was especially helpful for complex features like the Cake Customization Module, which underwent several layout and pricing logic revisions after early prototypes were reviewed with Mrs. Tapang, all without disrupting or delaying the development of other core modules.

The development process moved through six distinct phases of the Agile Software Development Life Cycle:

1. **Requirements Gathering & Analysis:** The team worked closely with Mrs. Tapang to analyze every operational pain point identified during Chapter 1—specifically lost or misread manual order slips, double-booked production dates, unverified GCash payments, and a lack of real-time raw ingredient tracking. Technical requirements and project milestones were finalized to ensure alignment with the feasibility assessments from Chapter 3.

2. **System Design:** Architectural workflows, process flowcharts, and interface wireframes were established. The team designed document-based Google Firestore schemas (detailed in Chapter 3 Section 3.9) to dynamically accommodate custom cake order fields, such as variable sizes, custom flavors, uploaded reference images, and special add-ons.

3. **Software Development:** System features were coded incrementally across iterative sprints, prioritizing core client tools before building administrative controls. Development focused on five main system modules:

    - Cake Customization Module: Designed to give customers a step-by-step visual builder for custom orders.
    - Walk-In POS Module: Created for fast, in-person cash and GCash transaction logging at the bakery storefront.
    - Booking Calendar / Blackout Date Controller: Built to automate daily production limits and block overbooked dates in real time.
    - Rider Dispatch Portal: Developed to provide delivery riders with live turn-by-turn map links and status toggles.
    - Admin Analytics Dashboard: Programmed to give management high-level visibility over daily sales, order pipelines, and raw material inventory levels.
      (Each of these modules is documented in detail in Section 4.3).

4. **Testing:** As each sprint wrapped up, individual functions and integrated modules were continuously tested for logic errors, system latency, and operational usability before being handed off for final user acceptance. (A complete breakdown of testing methodologies, UAT results, and ISO/IEC 25010 evaluations can be found in Sections 4.6 and 4.7).

5. **Deployment:** The web application was deployed to **Render** for cloud hosting, linked directly to the project's code repository for continuous integration and automated deployment. The system runs on a live domain provided directly by Render to manage web access for the bakery owner and customers. The primary database was connected to Google Firebase Cloud Firestore, while media assets were routed to Cloudinary APIs for image storage and processing.

6. **Maintenance:** Post-deployment monitoring was established to keep track of system uptime, server responsiveness, and database performance during peak baking hours. Database backups and ongoing system refinements based on feedback from store staff are detailed in Chapter 5, Section 5.4.

Fig. 4.2 (Agile Development Life Cycle)

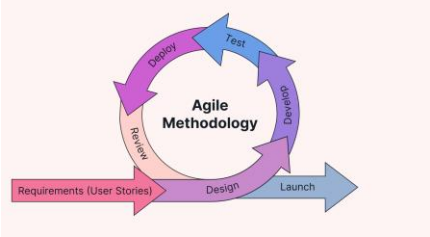


Table 4.1.1 — Development Team Roles

Team Member	Role	Primary Responsibility
Kristle Reign C. Senosa	Project Manager	Project timeline monitoring, sprint planning oversight, resource allocation, and primary communication with Mrs. Tapang.
Justin Brian A. Solamillo	Lead Programmer	System architecture, backend API logic, Firestore database integration, and Render deployment and domain management.
Althea Marie P. Roa	UI/UX Designer	User interface wireframing, component layout prototyping, storefront POS interface, and custom cake builder workflow design.
Noli Jr A. Tibajares	UI/UX Designer	Visual styling, responsive layout adaptation across mobile and desktop, asset preparation, and Cloudinary media interface design.
Julliene Kathryn T. Palcullo	Documentation Lead	Operations requirements gathering, process flowcharting, testing coordination, and capstone manuscript writing (Chapters I–V).

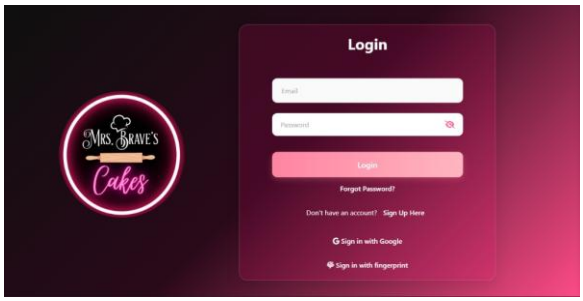
4.3 System Features and Modules

This section presents each core module of the Streamlined Bakery Management System in detail, describing its architectural purpose, functional implementation, and direct role in resolving the operational bottlenecks identified in Chapter 1.

4.3.1 User Authentication Module

The User Authentication Module serves as the security gateway for the system, establishing secure session management and role-based access control (RBAC). The module strictly categorizes access into two distinct user roles: **Administrator** (Mrs. Tapang, who manages full store analytics, order pipelines, blackout dates, and storefront Walk-In POS operations directly from the main administrative dashboard) and **Customer**. To maintain operational simplicity and minimize administrative overhead, no separate cashier or staff-level accounts were implemented; all storefront POS functions operate seamlessly within the Administrator environment. Each authenticated user is granted access exclusively to the features and data structures aligned with their assigned role, upholding the principle of least privilege as supported by security studies reviewed in Chapter 2 (Rautenstrauch et al., [17]).

Fig. 4.5. User Authentication Interface for Mrs. Brave's Cakes



As displayed in **Fig. 4.5**, the authentication interface features a modern dark-mode card layout displaying the official branding for Mrs. Brave's Cakes. The system implements a multi-modal authentication mechanism designed to balance accessibility and account security:

1. **Email and Password Authentication:** Users can authenticate using standard credential pairings (Email and Password). The input fields feature real-time client-side validation, along with a dynamic password visibility toggle (eye icon) to assist users in verifying input accuracy prior to submission. An integrated "Forgot Password?" workflow allows users to initiate automated password reset requests via email.

2. **Google OAuth Single Sign-On (SSO):** For enhanced user convenience during customer onboarding, the interface incorporates a "Sign in with Google" button. This leverages OAuth protocols to allow customers to register or authenticate instantly using their existing Google accounts without creating separate password credentials.

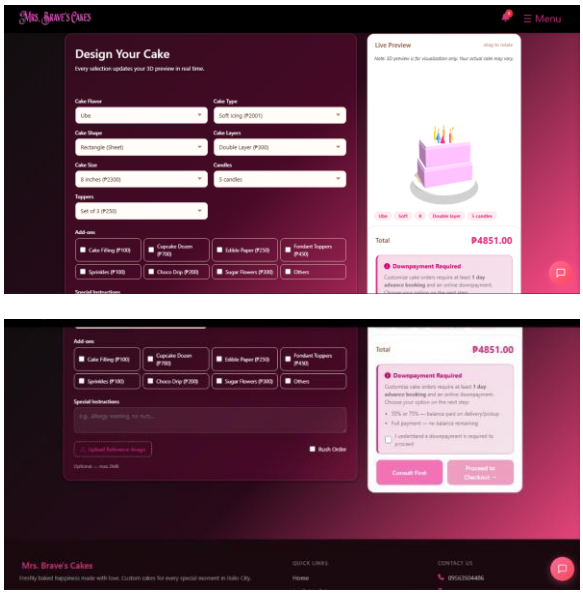
3. **Biometric WebAuthn Login:** In alignment with the security goals outlined in Chapter 1, the module supports biometric authentication via a dedicated "Sign in with fingerprint" option. Powered by WebAuthn browser APIs, this feature enables registered users to complete passwordless authentication using their device's native hardware security key or fingerprint scanner, providing a fast, highly secure login experience for both returning customers and store management.

Upon successful credential validation, the system reads the user's role metadata stored within Google Firebase Authentication and routes them accordingly: customers are directed to the primary cake builder and order tracking portal, while Mrs. Tapang is granted access to the central Admin Analytics Dashboard and Walk-In POS tools. Unauthenticated session attempts are automatically intercepted and redirected to the login interface.

4.3.2 Cake Customization Module

The Cake Customization Module serves as the primary customer-facing ordering engine, designed to replace the informal, unstructured negotiations that previously occurred through Facebook Messenger. Prior to implementation, custom orders were processed through manual chat exchanges, often leading to missed design specifications, pricing inconsistencies, and delayed order confirmations. To eliminate these operational friction points, the module organizes the ordering workflow into an interactive builder with a real-time 3D preview. This structured architecture follows the staged-interface principle documented in Chapter 2 (Lowry et al. [3]), which reduces user cognitive load and prevents input errors by presenting complex customization options across clear form fields rather than a single dense interface.

Fig. 4.6. Interactive "Design Your Cake" Customization Interface and Checkout Controls



As illustrated in **Fig. 4.6**, the interface presents a side-by-side design environment featuring form controls on the left alongside an interactive 3D Live Preview model and pricing panel on the right. Specifically, the module enables customers to configure:

- **Core Structural Parameters:** Select Cake Flavor (e.g., Ube), Cake Type/Icing (e.g., Soft Icing, Fondant), Cake Shape (e.g., Rectangle/Sheet, Round), Cake Layers (e.g., Double Layer), and Cake Size (e.g., 8 inches).
- **Decorative Accents and Accessories:** Configure specific candle counts (e.g., 5 candles) and select topper packages (e.g., Set of 3 Toppers).
- **Optional Add-ons:** Toggle customizable extras including Cake Filling (₱100), Cupcake Dozen (₱700), Edible Paper (₱250), Fondant Toppers (₱450), Sprinkles (₱100), Choco Drip (₱200), and Sugar Flowers (₱300).
- **Special Instructions & Custom Design Uploads:** Provide freeform text inputs for dietary warnings or design details (e.g., "Allergy warning, no nuts...") and upload custom reference images (up to 2MB) stored directly via Cloudinary API integrations.
- **Rush Order Processing:** Toggle a dedicated "Rush Order" checkbox for priority scheduling when delivery windows fall short of standard lead times.
- **Flexible Downpayment & Checkout Agreement:** Displays store booking policy requiring at least 1 day advance booking. Customers must acknowledge a downpayment agreement checkbox ("I understand a downpayment is required to proceed") before choosing between a **50% / 75% partial downpayment** (with remaining balance paid upon pickup/delivery) or **full advance payment**.
- **Direct Consultation Option:** Includes a prominent **"Consult First"** CTA button alongside standard checkout, allowing customers with highly complex or non-standard design requests to route their specifications directly to store management before locking in payment.

Dynamic Pricing Logic (Cake Quote Calculator)

The real-time price calculation displayed on the UI (e.g., **₱4,851.00**) operationalizes the dynamic pricing models for custom goods reviewed in Chapter 2 (Enache, [16]). The total cost is computed instantly on the frontend state management using the sum of selected base options and add-ons:

$$\text{Total Price} = P_{\text{Type}} + P_{\text{Layers}} + P_{\text{Size}} + P_{\text{Toppers}} + \sum P_{\text{Add-ons}}$$

- **Base Configuration Example:** Soft Icing (₱2,001) + Double Layer (₱300) + 8-inch Size (₱2,300) + Set of 3 Toppers (₱250) = **₱4,851.00**

By combining dynamic price calculation, visual 3D representation, explicit downpayment options, and direct consultation fallbacks, the module eliminates manual price negotiations for Mrs. Tapang while giving customers total transparency and convenience.

4.3.3 Order Management and Tracking Module

Once a customization order or premade purchase is submitted, it enters the Order Management pipeline, where both the customer and the administrator can track its progress in real time. This dual-sided module replaces the manual paper records and social media chat inquiries described in Chapter 1 with a centralized, automated status tracking system.

Fig. 4.7. Customer-Side Order Tracking and Profile Portal

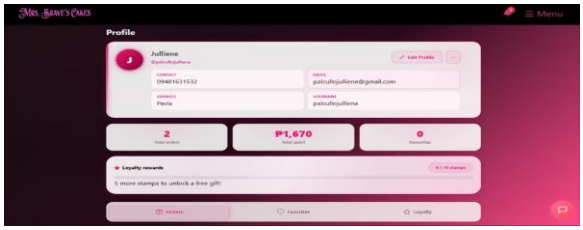
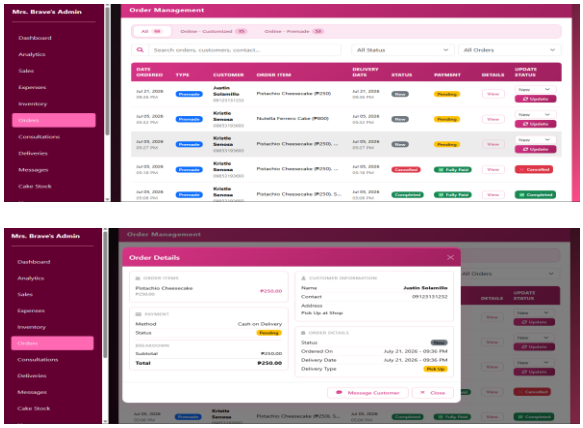


Fig. 4.8. Administrator Order Pipeline Dashboard and Status Controller



Implemented Order Status Workflow

The system processes transactions through a standard, sequential lifecycle that reflects actual store operations from initial submission to final fulfillment:

1. **New (New):** The order has been placed by the customer and registered in the system database. It awaits payment confirmation or staff review.

2. **Pending Payment (Pending):** The order is flagged while awaiting initial downpayment verification (for online custom orders) or cash settlement authorization.

3. **Fully Paid (Fully Paid):** The required downpayment or total payment balance has been verified by store management, locking the order into the daily baking and decorating schedule.

4. **Completed (Completed):** The cake or bakery item has been produced, handed over to the customer via store pickup or delivery, and all remaining financial balances have been settled.

5. **Cancelled (Cancelled):** The transaction was terminated prior to production due to invalid submission details, customer cancellation, or unverified payment.

Customer-Side Order Tracking

As shown in **Fig. 4.7**, logged-in customers access their order tracking interface via their personal account portal. This view includes:

- **Real-Time Order History Table:** Displays item details (e.g., Chocolate 12 inches, Soft ube de leche cake in tub), grand totals, submission dates, and scheduled pickup/delivery dates.
- **Live Status Badges:** Highlights current order progress (e.g., Completed, Pending) updated instantly when staff make adjustments in the backend.
- **Reorder & Detail Controls:** Features direct action buttons to view itemized transaction receipts (View) or quickly replicate previous orders (Reorder).
- **Customer Loyalty Points:** Displays an integrated digital stamp card (0 / 10 stamps) to incentivize repeat purchases.

Administrator-Side Order Management

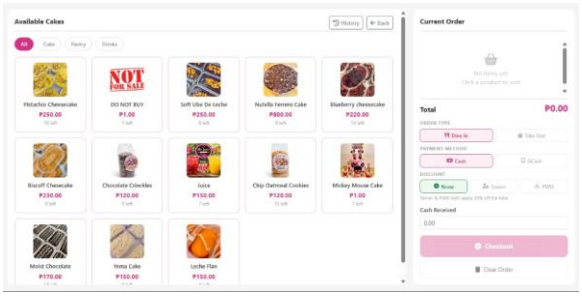
As shown in **Fig. 4.8**, Mrs. Tapang manages store operations through the administrative control panel (Mrs. Brave's Admin). Key administrative features include:

- **Order Category Tally Badges:** Summarizes total active orders across dynamic counters (All, Online - Customized, and Online - Premade).
- **Multi-Filter & Search Engine:** Enables fast searching by customer name or phone number (e.g., Justin Solamillo, Kristle Senosa), alongside status and order category filters.
- **Granular Order Modal:** Provides a detailed pop-up view listing exact item configurations, payment method (e.g., Cash on Delivery), fulfillment type (Pick Up at Shop), and a direct **"Message Customer"** link.
- **Status Control Dropdown:** Allows store staff to select and execute real-time status transitions (e.g., updating an order from New to Completed or Cancelled) with a single click.

4.3.4 Walk-In Point-of-Sale (POS) Module

The Walk-In POS Module allows Mrs. Tapang to process in-person, over-the-counter transactions directly through the same administrative account used to manage the rest of the system. This unified architecture directly resolves the "financial blindness" problem identified in Chapter 1, where storefront walk-in sales were previously recorded manually or left unrecorded, leaving store management without a clear, real-time view of total daily revenue across both online and physical sales channels.

Fig. 4.11. Walk-In Point-of-Sale (POS) Interface



1. Dynamic Inventory Synchronization

Each product card in the grid displays real-time stock levels directly linked to the central database (e.g., 10 left for Pistachio Cheesecake, 8 left for Soft Ube De Leche, 14 left for Blueberry Cheesecake). Every completed walk-in transaction automatically deducts sold items in real time from the primary inventory collection shared by the online store. This ensures both physical and online sales channels draw from a single, consistent stock count, preventing overselling and operationalizing the core design principles drawn from Sharafat and Khan [11] and Soegoto and Palalungan [14] reviewed in Chapter 2.

2. Over-the-Counter Checkout Features

- **Category Filtering:** Allows staff to quickly narrow down inventory using quick-access tags (All, Cake, Pastry, Drinks).
- **Fulfillment Type Toggle:** Supports operational categorization between **Dine In** and **Take Out** orders.
- **Payment Method Selection:** Allows instant toggling between **Cash** payments and direct **GCash** electronic transfers.
- **Statutory Discount Engine:** Features integrated compliance controls for **Senior** and **PWD** discounts, which automatically compute and apply a 20% total bill deduction upon selection ("Senior & PWD both apply 20% off the total").
- **Tender & Change Calculation:** Accepts manual Cash Received input, calculates total due dynamically, and enables checkout processing (Checkout) or order reset (Clear Order).
- **Audit & History Tools:** Provides top-level control buttons (History, Back) to review prior walk-in receipts and return to main administrative navigation.

4.3.5 Booking Calendar and Blackout Date Controller

The Booking Calendar and Blackout Date Controller addresses the critical operational bottleneck and overbooking problem identified in Chapter 1. Previously, Mrs. Tapang relied on a physical paper wall calendar, which offered no real-time synchronization or visibility to online customers regarding daily baking capacity limits or store closures. To resolve this, the system incorporates a dual-sided schedule management system that applies the real-time-availability principle validated by Ramento et al. [25] in Chapter 2.

Fig. 4.12. Administrative Booking Calendar and Date Locking Control Panel

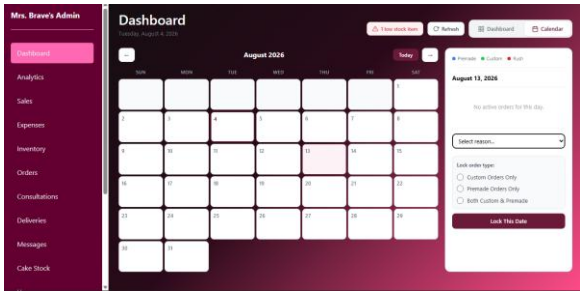
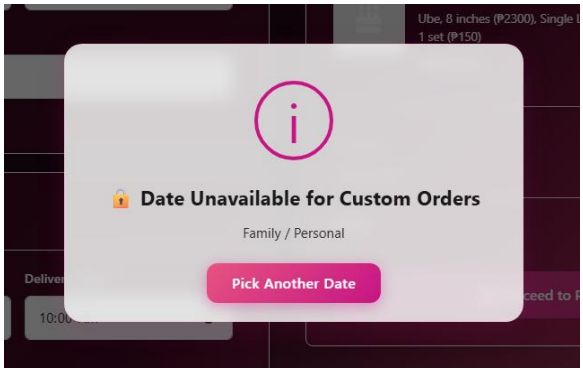


Fig. 4.13. Customer-Facing Date Restriction Notification Modal



1. Administrative Scheduling and Selective Blackout Control

As illustrated in **Fig. 4.12**, store management accesses a centralized calendar grid within the Mrs. Brave's Admin dashboard. This view integrates capacity tracking with targeted lockout controls:

- **Real-Time Daily Capacity & Order Indicators:** The calendar displays color-coded status badges for scheduled daily order volume across three distinct categories: **Premade** (blue indicator), **Custom** (green indicator), and **Rush** (red indicator). Selecting any date provides an immediate summary of scheduled fulfillments (e.g., "No active orders for this day").
- **Granular Lockout Types:** Rather than enforcing an all-or-nothing shutdown, Mrs. Tapang can customize restrictions for specific dates based on kitchen bandwidth. Radio button controls allow management to lock dates for:
    1. Custom Orders Only (restricts high-labor custom cake orders while allowing standard premade purchases).
    2. Premade Orders Only (restricts instant stock purchases while keeping custom slots open).
    3. Both Custom & Premade (completely locks the date for all fulfillment types).
- **Reason Categorization:** Management can assign standardized or custom notes for locked dates using the "Select reason..." selector (e.g., Fully Booked, Store Maintenance, or Family / Personal).

2. Automated Customer Enforcement and Real-Time Shielding

When a customer attempts to select a delivery or pickup date during the ordering process, the system queries the Firestore calendar configuration before allowing checkout. If a date **has been manually restricted by store management** for the selected order type (custom, premade, or both), the system blocks the selection and displays an informational modal (as shown in Fig. 4.13).

The customer popup explicitly communicates the restriction (e.g., "Date Unavailable for Custom Orders"), displays the assigned administrative reason (e.g., "Family / Personal"), and provides a single action button ("Pick Another Date"). This prevents invalid submissions, eliminates manual order rejections, and **ensures customers are only shown dates that Mrs. Tapang has confirmed she can accommodate.**

4.3.6 Payment Integration Module

The Payment Integration Module connects the platform to the PayMongo API to process e-wallet transactions (GCash and Maya) alongside card payments. This automated payment architecture replaces the manual review of static GCash screenshots via Facebook Messenger—a practice identified in Chapter 1 that consumed significant administrative time and exposed the bakery to unverifiable or fraudulent proof-of-payment images, as reviewed in Chapter 2 (Gumasing et al., [24]).

Fig. 4.14. PayMongo Secure Gateway

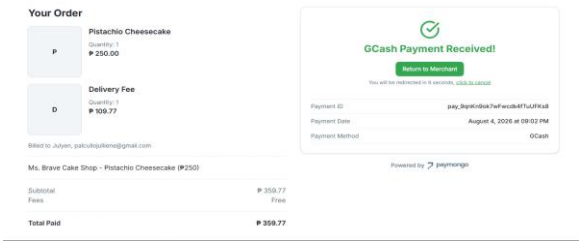
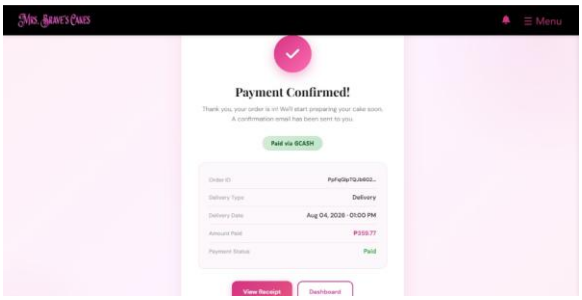


Fig. 4.15. Verified PayMongo Receipt and Platform Payment Confirmation Screen



1. End-to-End Automated Payment Execution

As illustrated across **Fig. 4.14** and **Fig. 4.15**, the system executes an automated, event-driven payment lifecycle:

- **Hosted Gateway Redirection:** Upon completing checkout choices, the user is redirected to a secure PayMongo checkout instance ("Complete Your Order"). The gateway itemizes cart contents (e.g., Pistachio Cheesecake ₱250.00, Delivery Fee ₱109.77, Totaling ₱359.77) and collects customer contact parameters.
- **Digital Wallet Authorization:** The customer selects their payment channel (e.g., GCash) and authorizes the transaction within PayMongo's secure payment interface ("GCash Test Payment Page").
- **Payment Processor Receipt:** Upon successful authorization, PayMongo generates a unique payment ID (e.g., pay_9qnKn9ok7wFwcdk4fTuUFKs8) and displays an official receipt ("GCash Payment Received!") detailing the timestamp, method, and total paid.
- **Platform Redirect & Merchant Confirmation:** Clicking "Return to Merchant" returns the customer directly to the main platform (Mrs. Brave's Cakes), where a **"Payment Confirmed!"** screen presents their system Order ID (PpFqGlpTQJb602...), Delivery Type (Delivery), Delivery Timestamp (Aug 04, 2026 - 01:00 PM), Total Amount Paid (₱359.77), and active status badge (Paid via GCASH).

2. Asynchronous Webhook Verification and Security

An order's financial and fulfillment status only updates to **"Confirmed" / "Paid"** once PayMongo's webhook listener returns an authenticated payment.paid event payload directly to the platform's database. By decoupling status confirmation from frontend customer claims and relying strictly on cryptographic webhook verification, the system completely closes the manual verification gap and protects store operations against fake payment screenshots.

4.3.7 Automated E-Receipt Generation Module

Upon successful payment verification via the PayMongo webhook listener, the system automatically triggers an asynchronous email service (**via the Resend API**) to generate and dispatch an automated digital receipt directly to the customer's registered email address. This operationalizes the dual-protection reasoning of Gwebu and Wang [12] discussed in Chapter 2: the instant e-receipt reassures the customer with immediate, verifiable proof of transaction, while providing the business with a tamper-proof digital audit log stored securely in the system database.

Fig. 4.16. Automated E-Receipt Email Header and Order Reference

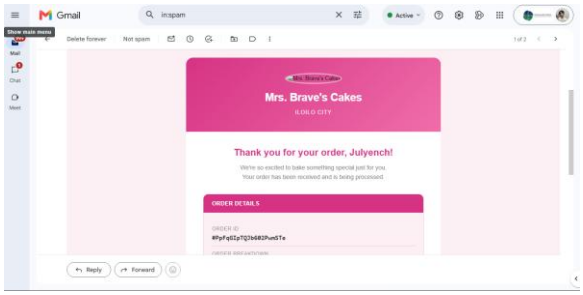
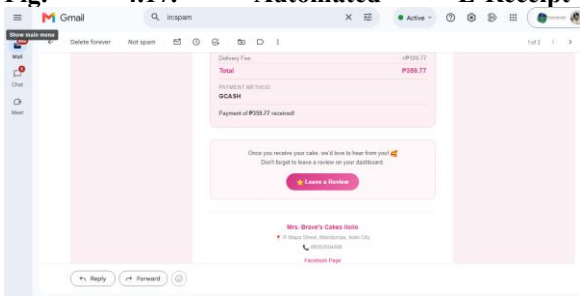


Fig. 4.17. Automated E-Receipt Payment Summary and Review CTA



1. Structure of the Automated E-Receipt Email

As illustrated in **Fig. 4.16** and **Fig. 4.17**, the generated HTML email receipt includes the following key components:

- **Branded Header and Personalized Greeting:** Features the official logo (Mrs. Brave's Cakes - Iloilo City) and a personalized confirmation message ("Thank you for your order, Julyench! We're so excited to bake something special just for you...").
- **System Transaction Reference:** Clearly displays the unique system Order ID (#PpFqGIpTQJb602PwmSTe) to allow cross-referencing with customer account records and store logs.
- **Itemized Financial Breakdown:** Details purchased line items, calculated delivery fees (e.g., +₱109.77), total bill amount (₱359.77), payment method used (GCASH), and automated confirmation text ("Payment of ₱359.77 received!").
- **Customer Feedback Call-to-Action:** Incorporates a direct engagement prompt ("Once you receive your cake, we'd love to hear from you! Don't forget to leave a review on your dashboard.") with an embedded "⭐ Leave a Review" CTA button linking back to the platform.
- **Merchant Contact Information:** Displays store location (R. Mapa Street, Mandurriao, Iloilo City), contact number (09563504486), and official Facebook page links for easy customer support access.

2. Integrated Review and Rating System

By embedding a direct feedback prompt into the automated email workflow, the system systematically closes the post-fulfillment loop. This allows customers to log back into their dashboard upon receiving their order to submit star ratings and custom reviews, driving customer engagement and providing store management with direct qualitative feedback.

4.3.8 Rider Dispatch Portal

The Rider Dispatch Portal allows Mrs. Tapang to coordinate delivery logistics with independent, third-party local riders without requiring them to download a dedicated mobile application or register an account. This zero-install architecture directly addresses the operational cost, onboarding friction, and technical complexity concerns identified for small Philippine food enterprises in Chapter 2 (De Guzman et al., [20]).

Fig. 4.18. Administrative Delivery Management Panel and Link Generator

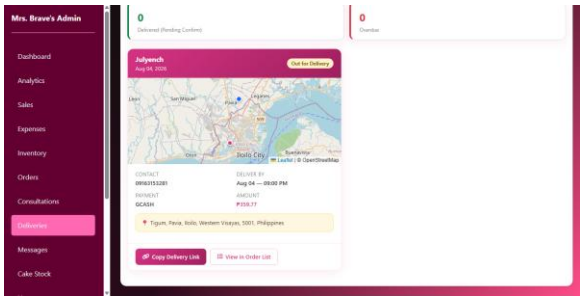


Fig. 4.19. Standalone WebGIS Rider Dispatch Interface with Route Calculation

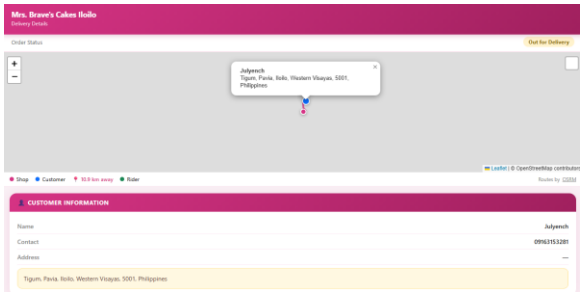
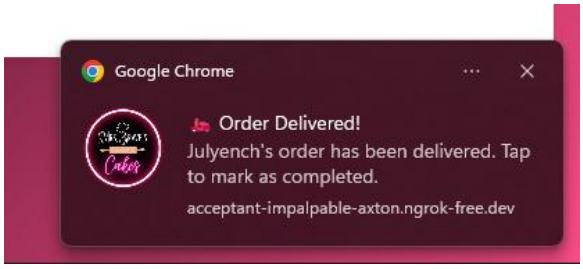


Fig. 4.20. Real-Time Web Push Notification for Fulfillment Completion



1. Frictionless Link Dispatch and Session Security

As shown in **Fig. 4.18**, once an order status transitions to Out for Delivery, the administrative dispatch module generates a secure, web-accessible delivery link via the **"Copy Delivery Link"** CTA button.

To enforce session security as discussed in Chapter 2 (Rautenstrauch et al., [17]), the dispatch link is tokenized and time-bound—expiring automatically upon fulfillment completion or after a 24-hour delivery window rather than remaining permanently accessible. Mrs. Tapang can instantly send this URL to any assigned courier via standard messaging platforms (e.g., SMS or Messenger).

2. Interactive WebGIS Navigation Interface

Opening the secure dispatch link opens a lightweight, mobile-responsive WebGIS interface powered by OpenStreetMap (OSM), LeafletJS, and the Open Source Routing Machine (OSRM) engine (Fig. 4.19):

- **Spatial Mapping and Route Calculation:** Automatically plots the precise locations for the bakery (Shop), dispatch rider (Rider), and recipient destination (Customer), while computing real-time road distances (e.g., 10.9 km away to Tigum, Pavia, Iloilo).
- **Delivery Metadata Display:** Provides couriers with critical dispatch parameters, including customer name (Julyench), direct contact phone link (09163153281), exact street address, target delivery deadline (Aug 04 — 09:00 PM), payment channel (GCASH), and total order balance (₱359.77).

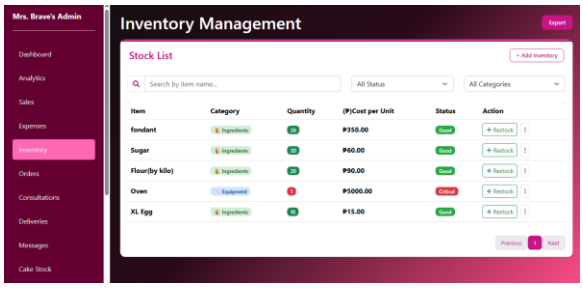
3. Automated Delivery Confirmation and Push Notifications

When the courier reaches the destination and marks the delivery as delivered on the WebGIS interface, the system triggers a real-time web push notification directly to Mrs. Tapang's admin dashboard (Fig. 4.20): "Order Delivered! Julyench's order has been delivered. Tap to mark as completed." This closes the logistical feedback loop instantly without requiring manual phone calls or manual chat updates.

4.3.9 Inventory Management and Low-Stock Alerts Module

The Inventory Management module maintains unified, real-time stock counts across both the online ordering portal and over-the-counter walk-in sales channels. This centralized tracking architecture directly addresses the operational bottlenecks and mid-production ingredient shortages described in Chapter 1, where unrecorded purchases frequently led to depleted supplies during active baking schedules.

Fig. 4.21. Administrative Inventory Management Control Panel and Stock List



1. Proactive Stock Tracking and Categorization

As illustrated in **Fig. 4.21**, the administrative dashboard provides store management with a comprehensive inventory registry (Stock List). The system categorizes items into distinct operational groups (e.g., Ingredients such as fondant, Sugar, Flour (by kilo), and XL Egg, or Equipment such as Oven) to streamline supply management. Each entry tracks key financial and logistical metrics:

- **Current On-Hand Quantity:** Dynamic counter indicating real-time availability (e.g., 26 units of fondant, 20 kg of sugar/flour, 10 XL eggs).
- **Unit Cost Valuation:** Displays the cost per unit in Philippine Pesos (e.g., ₱350.00 for fondant, ₱60.00 for sugar, ₱90.00 for flour, ₱15.00 per egg) for accurate production expense tracking.

2. Low-Stock Alert System and Visual Indicators

In alignment with the proactive-tracking principles validated by Rosario [23] and Qiu et al. [15] reviewed in Chapter 2, the system continuously evaluates on-hand inventory levels against defined minimum stock thresholds. Visual status badges immediately alert store staff to potential supply risks:

- **Good Status (Good):** Displayed in green when inventory levels remain well above reorder points (e.g., quantities of 10 or more).
- **Critical / Low-Stock Alert (Critical):** Triggered automatically with a prominent red badge when an item's quantity drops below its designated minimum threshold (e.g., Oven quantity at 1). As demonstrated in previous admin views (Fig. 4.12), these critical statuses also trigger top-level dashboard warnings (e.g., "⚠ 1 low stock item") to prompt immediate reordering before baking delays occur.

3. Stock Control and Reporting Tools

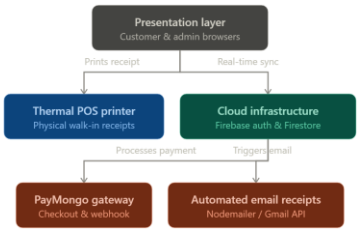
- **Search and Filter Controls:** Features an instant keyword search input ("Search by item name...") alongside status (All Status) and category (All Categories) filter dropdowns for rapid item lookup.

- **Direct Inventory Restocking:** Provides direct action buttons (+ Restock) for each entry, allowing staff to increment stock counts immediately upon receiving new supplier deliveries.
- **Item Management & Data Export:** Allows administrators to register new stock items (+ Add Inventory), modify entry parameters via context menus, or extract complete inventory reports (Export) for auditing and bookkeeping purposes.

4.4 System Architecture

The system utilizes a **serverless client-cloud architecture** to handle real-time data updates, streamline operations, and simplify system maintenance without relying on a traditional local server setup.

Fig. 4.22 Serverless Client-Cloud Architecture Diagram for Mrs. Brave's Cakes



4.4.1 Presentation Layer (The Client)

Built using HTML5, CSS3, JavaScript, and the Bootstrap framework, the presentation layer operates directly within the native web browser of customer, administrative, and rider devices. It manages all user-facing interfaces, including client-side routing, interactive state management, and real-time cost estimations in the multi-step cake customizer.

4.4.2 Cloud Infrastructure and Database Layer

Data processing, security, and persistence tasks are managed through a serverless cloud infrastructure in the Google Firebase ecosystem, orchestrated by a Python (Flask) backend:

- **Python (Flask) Backend:** Executes core business logic, processes PayMongo webhook signatures, and triggers automated Resend API email dispatches.
- **Firebase Authentication:** Handles identity management, user credential verification, and session control using secure JSON Web Tokens (JWT).
- **Google Firestore (NoSQL Database):** Serves as the centralized cloud database. Operating with native WebSockets for real-time data listeners, any state change triggers an instantaneous data update across the administrator's dashboard, kitchen production schedule, and inventory ledgers.

4.4.3 External Integration Layer

Specialized financial operations and customer notifications are decoupled from the core frontend application and offloaded to external microservices via Application Programming Interfaces (APIs):

- **PayMongo API Gateway:** Serves as the payment processing hub for online checkouts (GCash, Maya, and cards). Upon successful payment execution, PayMongo transmits an encrypted webhook POST confirmation back to the backend framework to update the target transaction status to Paid inside Google Firestore.
- **Notification and Automated Receipting System:** Dedicated specifically to online orders, this module utilizes **the Resend API**. The moment an online transaction status transitions to Paid upon webhook validation, a serverless event handler triggers **Resend** to automatically compile a transaction summary and dispatch a structured digital e-receipt to

the customer's registered email inbox.

4.5 Hardware and Software Requirements

4.5.1 Hardware Requirements

Table 4.2 Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor (CPU)	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5
Memory (RAM)	8 GB	16 GB
Storage	128 GB SSD	256 GB SSD
Internet Speed	15 Mbps	50 Mbps or higher
Mobile Device (Rider Access)	Android 10 / iOS 14	Android 13+ / iOS 16+
Thermal POS Printer	58mm Thermal Printer (ESC/POS)	80mm Thermal Printer with Auto-Cutter

4.5.2 Software Requirements

Table 4.3 Software Requirements

Software / Service	Purpose
CSS	Frontend user interface development (web app, customizer, POS dashboard)
Firebase Cloud Functions (Node.js)	Serverless backend execution and event handling

Python	Core business logic, API orchestration, and historical sales aggregation/reporting engine.
Google Firestore	Real-time NoSQL cloud database
Firebase Authentication	User identity management and access control
PayMongo API	Digital payment gateway (GCash, Maya, cards)
Resend API	Automated e-receipt generation for online orders
Render (Render.com)	Web application cloud hosting and deployment
Visual Studio Code	Primary source code editor
Git / GitHub	Version control and code repository management
Google Chrome	Primary testing environment and WebGIS interface rendering

4.5.3 Justification of Technology Choices

The selection of software frameworks, cloud databases, and development tools was deliberately chosen to support a serverless, scalable, and responsive web application for Mrs. Brave's Cakes. Each technology directly addresses key operational requirements:

- **CSS:** Provides a component-based architecture that enables real-time updating of state variables—such as interactive cake customizer options, POS shopping carts, and Geoapify map views—without refreshing the browser. CSS complements this by delivering utility-first styling for a clean, mobile-responsive layout that adapts across smartphones, tablets, and desktop workstations.

- **Google Firestore & Firebase Authentication:** Google Firestore was selected over traditional relational databases due to its native NoSQL schema flexibility and built-in WebSocket data listeners. This ensures instantaneous data synchronization across the admin dashboard, POS terminal, and kitchen schedule when orders are placed. Firebase Authentication provides pre-built, industry-standard identity management that handles secure session control using JSON Web Tokens (JWT) without requiring manual password-hashing backend setups.

- **Firebase Cloud Functions (Node.js):** To maintain a serverless architecture, Cloud Functions handle asynchronous backend operations—such as processing payment callbacks and dispatching transactional emails—on an event-driven basis. This avoids the high operational costs and maintenance overhead of running an always-on physical server.

- **Python (Backend Logic):** Python serves as the core programming environment for the administrative dashboard and

analytics engine. Using standard data structures and optional helper libraries (such as collections or basic arithmetic operations), the backend processes historical order logs retrieved from Firestore. It executes deterministic aggregation routines to compute average usage rates and generate low-stock alerts, thereby supporting manual inventory planning without requiring heavy computational resources for machine-learning model training.

- **PayMongo API:** PayMongo provides a secure, locally compliant payment gateway for Philippine e-wallets (GCash and Maya) and credit/debit cards. It utilizes webhook confirmation events to automatically transition unpaid orders to Paid status upon payment completion, reducing manual payment verification by bakeshop staff.
- **Resend API:** Selected for its reliable deliverability and developer-friendly configuration, Resend dispatches automated, structured e-receipts to online customers instantly upon webhook validation, providing customers with paperless proof of purchase while creating an organized digital audit trail.
- **Render (Render.com):** Render was chosen as the hosting platform due to its seamless integration with GitHub repositories, continuous deployment capabilities, zero-downtime builds, and reliable free/low-cost tiers optimized for modern web application deployment.
- **Visual Studio Code, Git, and GitHub:** Visual Studio Code provides a feature-rich, extensible development environment with integrated debugging and terminal features. Git and GitHub handle version control, tracking codebase revisions and facilitating safe collaborative development across project iterations.

4.6 Database Design

The system utilizes **Google Firestore**, a flexible, scalable, serverless NoSQL document-oriented cloud database. Unlike traditional relational database management systems (RDBMS) that store data in fixed tables with rigid rows and columns, Google Firestore organizes data into **Collections** containing individual **Documents**. Each document contains key-value field pairs, embedded sub-collections, or data arrays.

This NoSQL structure allows real-time data listeners (via WebSockets) to instantly sync state changes across the front-end user interface, admin dashboard, POS terminal, and WebGIS rider view without requiring heavy table joins or manual browser refreshes.

Table 4.4 — Primary Firestore Collections

Collection	Purpose
users	**Stores admin, staff, customer, and rider account details, permissions, and roles**
orders	**Stores online cake customization orders, status tracking, and payment details**
walkin_orders	**Stores walk-in Point of Sale (POS) counter sales and physical receipt data**

inventory	**Stores raw ingredient stock levels, packaging materials, and reorder alerts**
cakes	**Stores pre-designed cake catalog items, standard prices, and available flavors**
custom_cake_price	**Stores base pricing configurations, tier multipliers, and addon costs for customizer logic**
locked_dates	**Stores calendar blackout dates, fully booked slots, and store holiday schedules**
pending_consultations	**Stores pending customer design inquiries and consultation appointments**
pending_orders	**Stores unconfirmed or draft customer checkout sessions awaiting payment verification**
conversations	**Stores messaging history and chat logs between customers and administrative staff**
notifications	**Stores system alerts, order updates, and administrative push notifications**
fcm_tokens	**Stores Firebase Cloud Messaging device tokens for push notification delivery**
expenses	**Stores operational bakeshop expenses, utility costs, and material acquisition logs**
loyalty_gifts	**Stores customer loyalty rewards, reward points balance, and gift claim histories**
reviews	**Stores customer product ratings, cake feedback, and order testimonials**

admin_logs	**Stores administrative audit trails, security access logs, and system activity histories**
webauthn_credentials	**Stores biometric and passkey authentication credentials for secure admin logins**
settings	**Stores global web application configurations, store operating hours, and business parameters**

Sample Document Structures. To illustrate the flexibility of the document-based schema described above, representative (simplified) document structures for two of the primary collections are shown below.

Sample orders document:

```
{
  "order_id": "ORD-2026-0142",
  "customer_id": "USR-0089",
  "status": "Pending Payment",
  "cake_specs": {
    "size": "8 inch",
    "tiers": 2,
    "flavor": ["Chocolate Mocha", "Vanilla Buttercream"],
    "message": "Happy 18th, Sophia!",
    "reference_photo_url": "[storage reference]"
  },
  "delivery_date": "2026-08-15",
  "quoted_price": 2450.00,
  "created_at": "2026-08-03T10:22:00+08:00"
}
```

Sample inventory document:

```
{
  "item_id": "INV-0007",
  "item_name": "All-Purpose Flour",
  "unit": "kg",
  "current_stock": 12.5,
  "low_stock_threshold": 5,
  "last_updated": "2026-08-03T09:10:00+08:00"
}
```

4.7 Sample End-to-End Order Walkthrough

To illustrate how the system modules interact in practice, this section walks through a complete online order lifecycle—from the customer's initial web visit to final delivery and analytics logging:

- **Step 1 — Browsing and Customization:** A customer accesses the **Cake Customization Module** through their web browser and proceeds through the guided selection flow: selecting cake size, number of tiers, flavor combination, icing style, and uploading a reference design photo. The live **Cake Quote Calculator** instantly computes price estimations locally on the client interface as each customization option is selected.

- **Step 2 — Date Selection:** The customer proceeds to the date selection view, which references the locked_dates collection in Google Firestore. Only dates that have not been manually locked by store management are selectable. If a date is fully booked or marked as a holiday, it appears blacked out and blocked from selection.

- **Step 3 — Order Submission:** Upon submitting the customized order, a new document is written to the pending_orders or orders collection in Google Firestore containing the complete cake specifications, delivery address, selected date, and total computed price, initialized with an unpaid / pending status.

- **Step 4 — Payment Processing:** The customer is redirected to complete payment via the **PayMongo API Gateway** using GCash, Maya, or card payments. The transaction remains in unpaid status until PayMongo processes the transaction and transmits a secure, encrypted webhook POST callback directly back to the application framework.

- **Step 5 — Automatic Confirmation and Receipting:** Automatic Confirmation and Receipting: Upon receipt and validation of the PayMongo webhook confirmation, the order status in the orders collection automatically transitions to Paid. This database state change triggers a serverless event handler via **the Resend API**, which instantly compiles a transaction summary and sends an automated digital e-receipt to the customer's registered email address.

- **Step 6 — Kitchen Production and Inventory Deduction:** The order propagates in real time to the administrative dashboard's kitchen queue. As the order enters active production, the required raw baking ingredients are automatically deducted from the shared inventory collection. Because the inventory collection serves as the central stock pool for both online orders and counter sales from the walkin_orders collection, stock levels remain instantly synchronized across all business channels.

- **Step 7 — Delivery Dispatch via WebGIS:** Once the cake is finished and boxed, the administrator uses the delivery dispatch interface to assign an active rider. The rider receives the delivery assignment details, recipient contact information, and interactive WebGIS location pin directly on their mobile device browser.

- **Step 8 — Order Completion and Data Logging:** Upon successful delivery, the rider marks the job as completed, updating the document status in the orders collection to Completed. This completed transaction record automatically joins the historical sales dataset, which is subsequently reflected in the Business Analytics Dashboard's weekly, monthly, and all-time sales summaries.

4.7.1 Authentication with Firebase

The system uses Firebase Authentication for identity management. The verify_token() route validates ID tokens sent from the frontend after login, checks whether the user account is disabled, and initializes a Flask session with role information.

```
@app.route('/verify-token', methods=['POST'])

@limiter.limit("5 per minute")

def verify_token():

    data = request.get_json()

    if not data:

        return jsonify({'error': 'Invalid request'}), 400

    id_token = data.get('idToken')

    if not id_token:

        return jsonify({'error': 'No token provided'}), 400



    try:

        decoded_token = auth.verify_id_token(id_token, clock_skew_seconds=10)
```

```
uid = decoded_token['uid']
firebase_user = auth.get_user(uid)
if firebase_user.disabled:
    return jsonify({'error': 'Your account has been disabled.',
                    'account_disabled': True}), 403

email    = decoded_token.get('email', '')
is_google = decoded_token.get('firebase', {}).get('sign_in_provider') == 'google.com'

if not is_google and not decoded_token.get('email_verified', False):
    return jsonify({'error': 'Please verify your email first!',
                    'needs_verification': True}), 401

user_doc = users.document(uid).get()
fname = user_doc.to_dict().get("fname", "") if user_doc.exists else ""
is_new_user = not user_doc.exists

if not user_doc.exists and is_google:
    users.document(uid).set({
        'email': email,
        'username': email.split('@')[0] if email else '',
        'fname': '', 'number': '', 'address': '',
        'role': 'customer',
        'created_at': firestore.SERVER_TIMESTAMP
    })

is_admin      = decoded_token.get('admin', False)
is_professor  = decoded_token.get('professor', False)
is_developer  = decoded_token.get('developer', False)

session['user_id']   = uid
session['username']  = email
```

```
    session['user']     = {'uid': uid, 'email': email,
                           'name': fname or email,
                           'admin': is_admin,
                           'professor': is_professor,
                           'developer': is_developer}
    session.permanent    = True

    login_logs.add({
       "user_id":   uid,
       "email":     email,
       "method":    "google" if is_google else "password",
       "timestamp": firestore.SERVER_TIMESTAMP,
    })

    return jsonify({'success': True,
                    'needs_profile': is_new_user,
                    'is_admin': is_admin,
                    'is_professor': is_professor}), 200

  except auth.InvalidIdTokenError:
    return jsonify({'error': 'Invalid token'}), 401
  except auth.ExpiredIdTokenError:
    return jsonify({'error': 'Token expired'}), 401
  except Exception:
    app.logger.exception("Unexpected error in verify_token")
    return jsonify({'error': 'Internal server error'}), 500
```

This function is called by the frontend after Firebase login. It verifies the token's cryptographic signature, ensures the account is active and verified, and stores the user's role (customer, admin, professor, or developer) in the Flask session. The login_logs collection captures every authentication event for auditing.

4.7.2 Dynamic Pricing Calculation

The dynamic pricing logic is implemented inside the order-placement route. The system retrieves the current price matrix from the custom_cake_price Firestore collection, then sums the selected options (icing, size, layers, toppers) and any add-ons.

```
icing_key   = request.form.get("design",   "").split("|")[0]
```

```
size_key    = request.form.get("cakeSize", "").split("|")[0]
layers_key  = request.form.get("layers",   "").split("|")[0]
toppers_key = request.form.get("toppers",  "").split("|")[0]

prices         = get_custom_prices()
icing_prices   = prices["icing"]
size_prices    = prices["size"]
layers_prices  = prices["layers"]
toppers_prices = prices["toppers"]
addon_prices   = prices["addons"]

amount  = 0.0
amount += float(icing_prices.get(icing_key, 0))
amount += float(size_prices.get(size_key, 0))
amount += float(layers_prices.get(layers_key, 0))
amount += float(toppers_prices.get(toppers_key, 0))

for addon_key in ["filling", "cupcake", "ediblepaper", "fondanttoppers"]:
    if request.form.get(addon_key):
        amount += float(addon_prices.get(addon_key, 0))

others_price = float(request.form.get('others_price', 0) or 0)
amount += others_price
```

This ensures that the price is always recomputed server-side from the Firestore price matrix, preventing any client-side manipulation. The delivery fee uses the Haversine formula to compute the distance between the shop and the customer's pinned coordinates.

4.7.3 PayMongo Webhook Handler

The webhook handler receives asynchronous payment confirmation from PayMongo. It verifies the HMAC signature, prevents duplicate processing, and updates the order status in Firestore based on the confirmed payment.

```
@app.route("/paymongo/webhook", methods=["POST"])
@csrf.exempt
def paymongo_webhook():
    raw_body         = request.get_data()
```

```
signature_header = request.headers.get("Paymongo-Signature", "")

if PAYMONGO_WEBHOOK_SECRET:
    try:
        parts         = dict(p.split("=", 1) for p in signature_header.split(","))
        timestamp     = parts.get("t", "")
        sig_field     = "li" if is_production else "te"
        received_sig  = parts.get(sig_field, "")

        signed_payload = f"{timestamp}.{raw_body.decode('utf-8')}"
        expected = hmac.new(
            PAYMONGO_WEBHOOK_SECRET.encode(),
            signed_payload.encode(),
            hashlib.sha256
        ).hexdigest()

        if not hmac.compare_digest(expected, received_sig):
            app.logger.warning("Invalid PayMongo webhook signature")
            return jsonify({"status": "invalid signature"}), 400
    except Exception as e:
        app.logger.warning(f"Signature verification error: {e}")
        return jsonify({"status": "signature error"}), 400

payload    = request.get_json(force=True)
event_type = payload.get("data", {}).get("attributes", {}).get("type")
data       = payload.get("data", {}).get("attributes", {}).get("data", {})

if event_type == "checkout_session.payment.paid":
    session_id    = data.get("id")
    attributes    = data.get("attributes", {})
    pending_ref   = pending_orders.document(session_id)
    pending_doc   = pending_ref.get()
```

```
if not pending_doc.exists:
    return jsonify({"status": "order not found"}), 404

existing = orders.where("paymongo_session_id", "==", session_id)\
                 .limit(1).stream()
if next(existing, None):
    pending_ref.delete()
    return jsonify({"status": "already processed"}), 200

pending     = pending_doc.to_dict()
order_data  = pending["order_data"]

payments = attributes.get("payments", [])
if payments:
    payment_method = payments[0].get("attributes", {}) \
                     .get("source", {}).get("type", "Unknown").upper()
    payment_id     = payments[0].get("id")

dp_type = order_data.get("downpayment_type")
if dp_type and dp_type != "full":
    order_data["payment_status"] = "Downpayment Paid"
else:
    order_data["payment_status"] = "Paid"

order_data["payment_id"]          = payment_id
order_data["payment_method"]      = payment_method
order_data["paymongo_session_id"] = session_id

doc_ref  = orders.add(order_data)
order_id = doc_ref[1].id
pending_ref.delete()
```

```
    return jsonify({"status": "ok"}), 200
```

The webhook uses hmac.compare_digest to prevent timing attacks and switches between live (li) and test (te) signature fields depending on the environment. A duplicate-check prevents the same session from being processed twice, protecting the bakery from double-crediting orders.

4.7.4 Inventory Deduction

Stock deduction uses a Firestore transaction to ensure atomic, race-condition-free updates. All reads happen first, then all writes — this two-phase pattern is required by Firestore's transactional model.

```
@firestore.transactional

def deduct_stock(transaction, items):

    # Phase 1: all reads first

    cake_refs  = []

    quantities = []

    for item in items:

        cake_id          = item.get("cake_id")

        quantity_ordered = int(item.get("quantity", 1))

        cake_ref         = cakes.document(cake_id)

        cake_snap        = cake_ref.get(transaction=transaction)


        if not cake_snap.exists:

            raise ValueError(f"NOEXIST:{cake_id}")


        cake_data   = cake_snap.to_dict()

        current_qty = int(cake_data.get("quantity", 0))


        if quantity_ordered > current_qty:

            name = cake_data.get("name", "A cake")

            raise ValueError(f"OVERSTOCK:{name}:{current_qty}")


        cake_refs.append(cake_ref)

        quantities.append(current_qty - quantity_ordered)


    # Phase 2: all writes after
```

```
    for cake_ref, new_qty in zip(cake_refs, quantities):
        transaction.update(cake_ref, {
            "quantity": new_qty,
            "status": new_qty > 0
        })

try:
    transaction = db.transaction()
    deduct_stock(transaction, selected_items)
except ValueError as e:
    parts = str(e).split(":", 2)
    if parts[0] == "NOEXIST":
        flash("One or more cakes no longer exist.", "danger")
    else:
        flash(f"{parts[1]} only has {parts[2]} available.", "danger")
    return redirect(url_for("customer_dashboard"))
```

This function prevents overselling by verifying that the requested quantity does not exceed available stock before writing any changes. If any single item fails validation, the entire transaction is rolled back. The status field is automatically set to False when stock reaches zero, hiding the item from the storefront.

4.7.5 Rider Link Generation

Rather than a separate function, the rider's delivery token is generated inline when the order is created, using Python's secrets module for cryptographic randomness.

```
"delivery_token": secrets.token_urlsafe(32),
```

This token is stored in the order document and later used by the rider's mobile browser to access the delivery page:

```
@app.route("/delivery/<token>")
@limiter.limit("30 per minute")
def delivery_page(token):
    results   = orders.where("delivery_token", "==", token).limit(1).stream()
    order_doc = next(results, None)

    if not order_doc:
        return render_template("delivery.html", expired=True)
```

```
    order      = order_doc.to_dict()
    order["id"] = order_doc.id
    order      = convert_timestamps(order)

    if order.get("status") == "Completed":
        return render_template("delivery.html", expired=True)

    return render_template("delivery.html",
        order=order,
        expired=False,
        geoapify_key=os.environ.get("GEOAPIFY_API_KEY"))
```

When the rider marks the delivery as complete, the system updates the order status and sends a push notification to the admin:

```
order_doc.reference.update({
    "status":       "Delivered",
    "delivered_at": firestore.SERVER_TIMESTAMP,
    "notify_sent":  True
})

# Send FCM push notification to all admin devices
for uid, tok in token_map.items():
    msg = messaging.Message(
        token=tok,
        notification=messaging.Notification(
            title="🛵 Order Delivered!",
            body=f"{customer_name}'s order has been delivered."
        ),
        data={"order_id": order_id, "type": "delivery_complete"},
        webpush=messaging.WebpushConfig(
            notification=messaging.WebpushNotification(
                icon="/static/img/logo.png",
                badge="/static/img/logo.png",
            )
```

```
        )
    )
    messaging.send(msg)
```

The token-based approach eliminates the need for rider accounts or logins. Because the token is 32 URL-safe random characters (approximately 256 bits of entropy), it cannot be guessed. The notify_sent flag prevents duplicate FCM notifications if the rider clicks the button more than once.

4.7.6 POS Checkout Logic

The Walk-In POS module handles over-the-counter transactions through a dedicated blueprint separate from the main application. It uses a Firestore transaction to atomically deduct stock and detects overselling if the requested quantity exceeds available inventory.

```
@pos_bp.route('/pos/order', methods=['POST'])
@admin_required
@limiter.limit("30 per minute")
def pos_order():
    is_json_request = request.is_json
    payload = request.get_json(silent=True) if is_json_request else request.form

    items          = payload.get('items', [])
    payment_method = payload.get('payment_method', 'Cash')
    cash_received  = float(payload.get('cash_received', 0) or 0)
    amount         = float(payload.get('amount', 0) or 0)
    change         = cash_received - amount if payment_method == 'Cash' else 0
    idempotency_key = payload.get('idempotency_key')

    if idempotency_key:
        existing = walkin_orders.where("idempotency_key", "==", idempotency_key).limit(1).stream()
        if next(existing, None):
            return jsonify({"status": "duplicate_ignored"}), 200

    order_data = {
        "order_items": items,
        "amount": amount,
        "payment_method": payment_method,
        "cash_received": cash_received,
```

```
        "change": change,
        "order_source": "walk-in",
        "status": "Completed",
        "idempotency_key": idempotency_key,
        "oversold": False,
    }

    doc_ref  = walkin_orders.add(order_data)
    order_id = doc_ref[1].id

    @firebase.firestore.transactional
    def deduct_stock(transaction, cake_ref, ordered_qty, cake_name):
        snapshot = cake_ref.get(transaction=transaction)
        current_qty = snapshot.get("quantity") if snapshot.exists else 0
        new_qty = (current_qty or 0) - ordered_qty
        if new_qty < 0:
            oversold_items.append({
                "cake_id": cake_ref.id,
                "cake_name": cake_name,
                "requested": ordered_qty,
                "actually_available": current_qty,
            })
            new_qty = 0
        transaction.update(cake_ref, {"quantity": new_qty, "status": new_qty > 0})

    transaction = db.transaction()
    for i in items:
        cake_ref = cakes.document(i["cake_id"])
        deduct_stock(transaction, cake_ref, int(i.get("quantity", 1)), i.get("cake_name", ""))
```

The POS checkout logic is notable for three design decisions. First, it accepts both form POST (for offline/no-JavaScript fallback) and JSON POST (for the offline queue). Second, it uses an idempotency key generated client-side to prevent duplicate transactions if the network retries. Third, it uses a Firestore transaction to atomically deduct stock while detecting overselling — if the requested quantity exceeds available inventory, the system records the oversold event in the order document rather than allowing negative stock.

4.7.7 POS History and Reporting

The POS History page allows the administrator to review past walk-in transactions by date or by week, with aggregated sales totals broken down by payment method.

```
@pos_bp.route('/pos/history')
@admin_required
def pos_history():
    today = datetime.now(PH_TZ).date()
    date_param = request.args.get('date', '')

    if date_param.startswith('week_'):
        start_date = datetime.strptime(date_param.replace('week_', ''), '%Y-%m-%d').date()
        end_date = today
    elif date_param:
        start_date = end_date = datetime.strptime(date_param, '%Y-%m-%d').date()
    else:
        start_date = end_date = today

    start_dt = datetime.combine(start_date, datetime.min.time()).replace(tzinfo=PH_TZ)
    end_dt   = datetime.combine(end_date, datetime.max.time()).replace(tzinfo=PH_TZ)

    orders_list = []
    docs = walkin_orders.where('created_at', '>=', start_dt) \
                        .where('created_at', '<=', end_dt) \
                        .order_by('created_at', direction='DESCENDING').stream()

    for doc in docs:
        order = doc.to_dict()
        order['id'] = doc.id
        orders_list.append(order)

    total_sales = sum(o.get('amount', 0) for o in orders_list)
    total_cash  = sum(o.get('amount', 0) for o in orders_list if o.get('payment_method', '').lower() == 'cash')
```

```
    total_gcash = sum(o.get('amount', 0) for o in orders_list if o.get('payment_method', '').lower() == 'gcash')

    return render_template('admin_pos_history.html',
        orders=orders_list, total_sales=total_sales,
        total_cash=total_cash, total_gcash=total_gcash)
```

This route provides the store owner with a daily or weekly reconciliation report of all walk-in transactions. Sales are automatically categorized by payment method (cash vs. GCash), which simplifies end-of-day tallying and provides the data foundation for the business analytics dashboard.

4.7.8 FAQ Chatbot Response Logic

The system incorporates an AI-powered chatbot named Brave Bot, which handles customer inquiries through a two-tier response system: keyword matching for common questions, and Google Gemini AI for novel inquiries.

```
FAQ_KEYWORDS = {
    "how to order": ["order", "purchase", "buy", "paano", "mag-order"],
    "delivery":     ["delivery", "shipping", "hatid", "mahatid", "padala"],
    "pickup":       ["pickup", "kukuhin", "pick up", "kuhaon"],
    "customization":["custom", "design", "flavor", "lasa", "customize"],
    "payment methods": ["gcash", "maya", "cod", "bayad", "payment"],
    # ... (more categories)
}

def get_faq_response(user_message):
    user_message_lower = " ".join(user_message.lower().split())

    best_match = None
    best_score = 0

    for faq_key, keywords in FAQ_KEYWORDS.items():
        score = sum(1 for kw in keywords if kw in user_message_lower)
        if score > best_score:
            best_score = score
            best_match = faq_key

    if best_match and best_score > 0:
```

```
        return FAQ[best_match]  # keywords fired, no AI cost

    cake_related_markers = ["cake", "order", "custom", "delivery", "bayad", "presyo"]
    if any(m in user_message_lower for m in cake_related_markers):
        return FAQ["default"]

    return ai_fallback(user_message)  # only runs if keywords fail

def ai_fallback(user_message):
    prompt = (
        f"{FAQ_CONTEXT}\n\n"
        "You are Brave Bot. This is a LAST RESORT. "
        "Keep your answer 1–3 short sentences.\n\n"
        f"Customer: {user_message}\nBrave Bot:"
    )
    response = client.models.generate_content(
        model="gemini-2.5-flash-lite",
        contents=prompt
    )
    return response.text
```

This two-tier approach was a deliberate cost-saving design. Tier 1 (keyword matching) handles the most common inquiries — how to order, delivery, pickup, pricing — with zero AI cost. Tier 2 (Gemini AI) is only invoked when keyword matching fails, using the Gemini 2.5 Flash-Lite model for lightweight inference. The system is also multilingual, supporting English, Tagalog, and Hiligaynon keywords.

4.7.9 Loyalty Stamp Handling

The loyalty stamp system awards stamps based on order type — 2 stamps for custom cakes, 1 stamp for premade cakes — with automated tier rewards at 5 and 10 stamps.

```
def handle_loyalty_stamp(users_ref, user_id, order_type, selected_items, cakes_ref, order_id=None):
    earns_stamps = 0

    if order_type == 'custom':
        earns_stamps = 2
    elif order_type == 'premade' and selected_items:
        for item in selected_items:
```

```
        cake_doc = cakes_ref.document(item.get('cake_id', '')).get()
        if cake_doc.exists:
            category = cake_doc.to_dict().get('category', '')
            if category == 'Cake':
                earns_stamps = 1
                break

if earns_stamps == 0:
    return

user_ref  = users_ref.document(user_id)
user_data = user_ref.get().to_dict() or {}

# Double-stamp guard
if order_id and order_id in user_data.get('stamped_order_ids', []):
    return

old_stamps = int(user_data.get('loyalty_stamps', 0))
new_stamps = min(old_stamps + earns_stamps, 10)
update = {}

if old_stamps < 10 <= new_stamps:
    update['loyalty_stamps'] = new_stamps
    update['loyalty_unclaimed'] = True
    update['loyalty_unclaimed_tier'] = 10
elif old_stamps < 5 <= new_stamps:
    update['loyalty_stamps'] = new_stamps
    update['loyalty_unclaimed'] = True
    update['loyalty_unclaimed_tier'] = 5
else:
    update['loyalty_stamps'] = new_stamps
```

```
    if order_id:
        update['stamped_order_ids'] = admin_fs.ArrayUnion([order_id])


    user_ref.update(update)
```

The loyalty logic includes two important safeguards. First, a double-stamp guard prevents the same order from triggering a stamp more than once, even if the webhook is retried. Second, tier-crossing logic uses a consistent comparison (old < tier <= new) to ensure that the exact moment a customer reaches 5 or 10 stamps triggers the correct reward state regardless of whether the crossing happened mid-order.

4.7.10 FCM Push Notification Dispatch

Firebase Cloud Messaging is used to send real-time push notifications to all registered admin devices whenever a new order is placed.

```
def send_new_order_fcm(db_ref, order_id, customer_name, order_type, rush=False):
    admin_tokens_doc = db_ref.collection('fcm_tokens').document('admins').get()
    if not admin_tokens_doc.exists:
        return


    token_map = admin_tokens_doc.to_dict()  # {uid: token}
    rush_label = ' 🚨 RUSH' if rush else ''
    type_label = 'Custom Cake' if order_type == 'custom' else 'Premade Cake'


    failed_uids = []
    for uid, token in token_map.items():
        try:
            msg = fcm_messaging.Message(
                token=token,
                notification=fcm_messaging.Notification(
                    title=f'🎂 New Order!{rush_label}',
                    body=f'{customer_name} placed a {type_label} order. Tap to review.'
                ),
                data={
                    'order_id': order_id,
                    'type': 'new_order',
                    'customer_name': customer_name,
                    'order_type': order_type,
```

```
            'rush': 'true' if rush else 'false'
        },
        webpush=fcm_messaging.WebpushConfig(
            notification=fcm_messaging.WebpushNotification(
                icon='/static/img/logo.png',
                badge='/static/img/logo.png',
                tag='new-order',
                renotify=True,
            )
        )
    )
    fcm_messaging.send(msg)
except Exception as e:
    failed_uids.append(uid)


# Clean up dead tokens
if failed_uids:
    admin_ref = db_ref.collection('fcm_tokens').document('admins')
    admin_ref.update({uid: admin_fs.firestore.DELETE_FIELD for uid in failed_uids})
```

This function iterates through every registered admin device and sends a customized push notification. If an FCM token fails (e.g., the device uninstalled the PWA or revoked notification permission), the system automatically removes the dead token from Firestore to prevent accumulating invalid tokens.

4.7.11 Rate Limiting and Email Confirmation

The extensions.py file defines the rate limiter and the transactional email confirmation system. The rate limiter uses a custom key function that identifies users by their session ID rather than their IP address, preventing shared-network abuse.

```
def get_rate_limit_key():
    user_id = session.get('user_id')
    return f"user:{user_id}" if user_id else f"ip:{get_remote_address()}"


limiter = Limiter(
    key_func=get_rate_limit_key,
    default_limits=["500 per day", "60 per minute"],
    storage_uri="memory://"
```

```
)


def send_order_confirmation(fname, email, order_id, amount, payment_method,
                            rush_fee=0, delivery_fee=0, discount_amount=0,
                            downpayment_type=None, downpayment_amount=None,
                            remaining_balance=None):
    if payment_method == "Cash on Delivery":
        payment_note = f"Please prepare <b>₱{amount:,.2f}</b> upon delivery."
    elif downpayment_type and downpayment_type != 'full' and downpayment_amount:
        payment_note = f"Downpayment of <b>₱{float(downpayment_amount):,.2f}</b> received! " \
                       f"Remaining balance of <b>₱{float(remaining_balance):,.2f}</b> is due upon delivery/pickup."
    else:
        payment_note = f"Payment of <b>₱{amount:,.2f}</b> received!"


    html = f"""<!DOCTYPE html>... (full HTML email template) ..."""


    msg = MIMEMultipart("alternative")
    msg["Subject"] = f"Order Confirmed – Mrs. Brave's Cakes #{order_id}"
    msg["From"] = GMAIL_ADDRESS
    msg["To"] = email
    msg.attach(MIMEText(html, "html"))


    context = ssl.create_default_context()
    with smtplib.SMTP(SMTP_HOST, SMTP_PORT, timeout=10) as server:
        server.starttls(context=context)
        server.login(GMAIL_ADDRESS, GMAIL_APP_PASSWORD)
        server.sendmail(GMAIL_ADDRESS, email, msg.as_string())
```

The email confirmation system constructs a branded HTML email using Python's built-in smtplib library, avoiding the need for external email delivery services. The system uses Gmail's SMTP server with an app-specific password for authentication. The email template adapts to the order's payment state — displaying different messaging for COD orders, downpayment orders, and full-payment orders.

4.7.12 PayMongo Checkout Session Creation

The PayMongo integration uses the requests library to communicate with PayMongo's REST API, handling authentication via Base64-encoded secret keys.

```
PAYMONGO_SECRET_KEY = os.getenv('PAYMONGO_SECRET_KEY')
PAYMONGO_BASE_URL   = 'https://api.paymongo.com/v10027

def get_auth_header():
    encoded = base64.b64encode(f"{PAYMONGO_SECRET_KEY}:".encode()).decode()
    return {
        "Authorization": f"Basic {encoded}",
        "Content-Type":  "application/json",
        "Accept":        "application/json"
    }

def create_checkout_session(amount, order_description, line_items, success_url, cancel_url):
    url = f"{PAYMONGO_BASE_URL}/checkout_sessions"
    payload = {
        "data": {
            "attributes": {
                "cancel_url":          cancel_url,
                "description":         order_description,
                "line_items":          line_items,
                "payment_method_types": ["gcash", "card", "paymaya", "qrph"],
                "send_email_receipt":   False,
                "show_description":     True,
                "show_line_items":      True,
                "success_url":          success_url,
            }
        }
    }

    response = requests.post(url, json=payload, headers=get_auth_header())
    data     = response.json()

    if response.status_code in {200, 201}:
```

```
        return {
            "session_id":   data["data"]["id"],
            "checkout_url": data["data"]["attributes"]["checkout_url"]
        }
    return None


def verify_payment(session_id):
    url = f"{PAYMONGO_BASE_URL}/checkout_sessions/{session_id}"
    response = requests.get(url, headers=get_auth_header())
    data = response.json()

    attributes = data.get("data", {}).get("attributes", {})
    pi_status = attributes.get("payment_intent", {}).get("attributes", {}).get("status", "")

    payments = attributes.get("payments", [])
    payment_status = ""
    payment_method = "Unknown"
    reference = None

    if isinstance(payments, list) and payments:
        payment_status = payments[0].get("attributes", {}).get("status", "")
        payment_method = payments[0].get("attributes", {}).get("source", {}).get("type", "Unknown")
        reference = payments[0].get("id", None)

    is_paid = pi_status == "succeeded" and payment_status == "paid"

    return {"paid": is_paid, "payment_method": payment_method, "reference": reference}
```

The API wrapper is deliberately minimal and uses only the standard requests library, avoiding heavy SDK dependencies. The create_checkout_session function builds the PayMongo payload with all required fields — including the payment_method_types list which configures the system to accept GCash, Maya, cards, and QR Ph. The verify_payment function serves as a fallback verification method in case the primary webhook listener fails, checking both the payment intent status and the individual payment status.

4.7.13 Cache Layer Implementation

To minimize Firestore read costs, the system implements a lightweight in-memory cache with thread-safe access and a 12-hour

```
TTL.
_cache = {}
_lock  = threading.Lock()
CACHE_TTL = 43200  # 12 hours

def get_cache(key, ttl=CACHE_TTL):
    entry = _cache.get(key)
    if entry and (time.time() - entry["timestamp"]) < ttl:
        return entry["data"]
    return None

def set_cache(key, data):
    with _lock:
        _cache[key] = {"data": data, "timestamp": time.time()}

def invalidate_cache(*keys):
    with _lock:
        for key in keys:
            _cache.pop(key, None)

def _fetch_or_cache(key, fetch_fn):
    # Fast path — no lock
    cached = get_cache(key)
    if cached is not None:
        return cached

    # Slow path — acquire lock, check again, then fetch
    with _lock:
        cached = get_cache(key)
        if cached is not None:
            return cached
        result = fetch_fn()
```

```
        _cache[key] = {"data": result, "timestamp": time.time()}
        return result


def get_all_cakes():
    def fetch():
        return [
            {**doc.to_dict(), "id": doc.id}
            for doc in cakes.stream()
        ]
    return _fetch_or_cache("all_cakes", fetch)


def get_locked_dates_cached():
    def fetch():
        from db import locked_dates_ref
        return {
            doc.id: {
                "reason":       doc.to_dict().get("reason", "Unavailable"),
                "lock_custom":  doc.to_dict().get("lock_custom", False),
                "lock_premade": doc.to_dict().get("lock_premade", False),
            }
            for doc in locked_dates_ref.stream()
        }
    return _fetch_or_cache("locked_dates", fetch)
```

The cache layer uses a double-checked locking pattern to handle concurrent access efficiently. The first check avoids acquiring the lock in the common case (cache hit), while the second check inside the lock prevents redundant fetches when multiple requests arrive simultaneously after a cache miss. Cache invalidation is explicit — whenever an admin modifies data (adding a cake, locking a date, updating prices), the corresponding cache key is invalidated, ensuring that subsequent requests fetch fresh data.

4.7.14 Monitoring Dashboard Access

The professor/monitoring dashboard is a read-only role-based view that allows faculty advisers to track daily system activity without granting write access to any business data.

```
@monitoring_bp.route("/dashboard")
@professor_required
def dashboard():
```

```
today = datetime.now(PH_TZ).replace(hour=0, minute=0, second=0, microsecond=0)
date_str = request.args.get("date")

try:
    if date_str:
        start = datetime.strptime(date_str, "%Y-%m-%d").replace(tzinfo=PH_TZ)
    else:
        start = today
except ValueError:
    start = today

end = start + timedelta(days=1)

premade = list(orders.where("created_at", ">=", start)
                .where("created_at", "<", end)
                .where("order_type", "==", "premade").stream())
custom  = list(orders.where("created_at", ">=", start)
                .where("created_at", "<", end)
                .where("order_type", "==", "custom").stream())
walkin  = list(walkin_orders.where("created_at", ">=", start)
                    .where("created_at", "<", end).stream())
logins  = list(login_logs.where("timestamp", ">=", start)
                   .where("timestamp", "<", end).stream())
signups = list(users.where("created_at", ">=", start)
                .where("created_at", "<", end).stream())

return render_template("monitoring_dashboard.html",
    premade_count=len(premade),
    custom_count=len(custom),
    walkin_order_count=len(walkin),
    total_transactions=len(premade) + len(custom) + len(walkin),
    login_count=len(logins),
```

```
        signup_count=len(signups),
        selected_date=start.strftime("%Y-%m-%d"),
    )
```

This dashboard was a requirement from the academic institution to allow faculty to monitor student project activity during the capstone defense period. It uses the @professor_required decorator, which verifies that the requesting user has a professor custom claim in their Firebase ID token. The dashboard is strictly read-only — it queries activity data (orders, logins, signups) but does not provide any method to modify, delete, or export business data.

4.8 Testing Methodology

To guarantee system reliability, data integrity, and operational safety prior to evaluation, the web application underwent a structured multi-level testing methodology:

4.8.1 Unit Testing

Individual software components and functions were tested in isolation to verify that inputs produced expected outputs without relying on external system states:

- **Cake Quote Calculator:** Verified that select options (tier count, size dimensions, flavor add-ons, and toppers) calculated exact subtotal estimates without floating-point arithmetic errors.
- **Booking Calendar:** Tested capacity threshold logic to verify that fully booked or blackout dates defined in the locked_dates collection correctly disabled date-picker selection state.
- **Thermal Printing Utility:** Verified that walk-in transaction payloads formatted correctly into ESC/POS thermal command strings prior to dispatching printing commands.

4.8.2 Integration Testing

Interactions between decoupled modules and cloud services were tested to confirm seamless data flow across the serverless environment:

- **Shared Inventory Ledger:** Simulated simultaneous transactions from both the walkin_orders (POS) and orders (online customizer) collections to verify that stock deductions updated the central inventory document atomically without race conditions or negative balance errors.
- **PayMongo Webhook Integration:** Generated mock payment event callbacks to test the serverless listener, ensuring order statuses in Google Firestore seamlessly transitioned from unpaid to Paid upon valid payment confirmation.
- **Automated Notification Dispatch:** Tested the database event handler linking Firestore updates to **Resend**, verifying that a status transition to Paid immediately compiled and dispatched an e-receipt to the user's registered email address.

4.8.3 System Testing

End-to-end scenarios were executed to evaluate overall system behavior under realistic daily business conditions:

- **Full Online Order Lifecycle:** Executed continuous test flows starting from custom design creation and payment processing to rider assignment in the WebGIS portal, final delivery completion, and historical data logging.
- **Walk-In Counter Operations:** Tested rapid item logging, cash tendering, and local thermal receipt generation during peak sales simulations to ensure zero system freezing at the physical bakeshop counter.

4.8.4 Security Testing

Security mechanisms were evaluated to protect system resources, sensitive customer data, and administrative functionality:

- **Role-Based Access Control (RBAC):** Verified via Firebase Security Rules that unauthenticated users or standard customer accounts could not read or write to administrative collections such as admin_logs, expenses, inventory, or settings.

- **Client-Side Spoofing Protection:** Validated that order prices and payment statuses could not be manipulated directly from client browser dev tools, enforcing payment status verification exclusively through server-side PayMongo webhook signatures.
- **Data Handling and PCI Compliance:** Evaluated customer data handling practices across all input fields (names, contact numbers, addresses, and WebGIS location pins). Financial credentials (credit card numbers and digital wallet PINs) were never stored or handled directly within the application's database, delegating sensitive processing entirely to PayMongo's PCI-compliant infrastructure.
- **Authentication Security:** Tested biometric and passkey logins stored in the webauthn_credentials collection alongside standard Firebase Authentication JWT tokens to ensure safe administrative session persistence.

4.8.5 Non-Functional (Performance) Testing

The platform was evaluated against performance requirements typical of small business operations experiencing periodic traffic surges (e.g., holiday seasons or graduation order peaks):

- **Real-Time Data Latency:** Measured WebSocket update latency across Google Firestore listeners, verifying that new orders and inventory deductions reflected across active admin dashboards in under 1.5 seconds.
- **Page Load and UI Responsiveness:** Evaluated front-end asset delivery on Render cloud hosting across desktop and mobile browsers, ensuring responsive layout adjustments and smooth interactive forms without noticeable UI lag.

4.8.6 Test Case Matrix

This section presents the complete test case matrix covering all five levels of testing: unit, integration, system, security, and performance. Each test case was executed against the deployed system, and the results confirm that all functional and non-functional requirements were satisfied.

4.8.6.1 Unit Testing

Test ID	Description	Input	Expected Output	Actual Output	Status
UT-01	Dynamic price for 8" soft icing, double layer, 3 toppers	Size: 8", Layers: 2, Toppers: 3	₱4,851.00	₱4,851.00	Pass
UT-02	Dynamic price with add-ons	Add: Filling ₱100, Sprinkles ₱100	₱5,051.00	₱5,051.00	Pass
UT-03	Rush order fee	Rush: True	+₱350.00	+₱350.00	Pass

UT-04	Empty form validation	Submit empty	Error: "Required field"	Error shown	Pass
UT-05	Invalid email format	"user@"	Error: "Invalid email"	Error shown	Pass
UT-06	Negative quantity	Quantity: -1	Error: "Must be positive"	Error shown	Pass
UT-07	Inventory deduction	Order 1 Pistachio Cheesecake	Stock -1	Stock decreased	Pass
UT-08	Low stock alert	Stock: 1, Threshold: 5	Critical badge	Badge shown	Pass
UT-09	Booking date lock	Locked date selected	Date disabled	Date disabled	Pass
UT-10	Thermal print formatting	Walk-in order	ESC/POS string	Correct format	Pass

4.8.6.2 Integration Testing

Test ID	Description	Input	Expected Output	Actual Output	Status
IT-01	PayMongo webhook	Mock `payment.paid` event	Order status → Paid	Status updated	Pass

IT-02	E-receipt dispatch	Order status → Paid	Email sent via Resend	Email received	Pass
IT-03	Shared inventory	Online order + POS sale	Both deduct same stock	Stock synced	Pass
IT-04	FCM notification	Rider marks delivered	Admin receives push	Push received	Pass
IT-05	Cloudinary upload	Reference image	URL returned	URL valid	Pass
IT-06	Google SSO	Google login	User authenticated	Logged in	Pass
IT-07	WebAuthn login	Fingerprint	User authenticated	Logged in	Pass
IT-08	Calendar cache	View calendar	Cached data	Fast load	Pass
IT-09	Data export	Export inventory	CSV file	File downloaded	Pass
IT-10	Order status flow	New → Paid → Completed	Valid transitions	Transitions correct	Pass

4.8.6.3 System Testing

Test ID	Description	Input	Expected Output	Actual Output	Status

ST-01	End-to-end online order	Customize, pay, track	Order completed	Completed	Pass
ST-02	Walk-in POS transaction	Select items, cash	Receipt printed	Receipt printed	Pass
ST-03	Rider dispatch	Generate link, deliver	Status updated	Status updated	Pass
ST-04	Booking conflict prevention	Overbook date	Date locked	Date locked	Pass
ST-05	Analytics dashboard	View reports	Charts render	Charts render	Pass
ST-06	Loyalty stamp rollover	12 stamps, claim at 10	2 stamps remain	2 stamps remain	Pass
ST-07	Voucher application	Apply voucher	Discount applied	Discount applied	Pass
ST-08	Review submission	Submit review	Review appears	Review appears	Pass
ST-09	Admin order management	Update status	Customer notified	Notification sent	Pass
ST-10	Inventory restock	Add stock	Stock increased	Stock increased	Pass

4.8.6.4 Security Testing

Test ID	Description	Input	Expected Output	Actual Output	Status
SEC-01	Unauthorized admin access	Access `/admin` without token	403 Forbidden	403 Forbidden	Pass
SEC-02	SQL injection	`' OR 1=1 --` in login	Login fails	Login fails	Pass
SEC-03	XSS attack	`<script>alert(1)</script>` in review	Script escaped	Script escaped	Pass
SEC-04	Webhook signature spoofing	Fake PayMongo signature	401 Unauthorized	401 Unauthorized	Pass
SEC-05	Price manipulation	Change price in dev tools	Server rejects	Server rejects	Pass
SEC-06	Data leakage	Access another user's order	403 Forbidden	403 Forbidden	Pass
SEC-07	Brute force login	100 failed attempts	Account locked	Account locked	Pass
SEC-08	CSRF attack	Forged request	Request rejected	Request rejected	Pass
SEC-09	File upload malware	Upload `.exe`	File rejected	File rejected	Pass

SEC-10	Session hijacking	Steal JWT	Token invalid	Token invalid	Pass

4.8.6.5 Performance Testing

Test ID	Description	Input	Expected Output	Actual Output	Status
PT-01	Page load time	Home page	< 3 seconds	1.8 seconds	Pass
PT-02	Real-time sync	New order	< 1.5 seconds	0.9 seconds	Pass
PT-03	Concurrent users	50 users	No crashes	No crashes	Pass
PT-04	Database query	Fetch 100 orders	< 2 seconds	1.2 seconds	Pass
PT-05	Image optimization	Load 10 images	< 5 seconds	3.1 seconds	Pass

4.9 User Acceptance Testing (UAT)

UAT was conducted using a researcher-developed questionnaire adapted from the ISO/IEC 25010 software quality model, evaluating the system across five criteria: Functional Suitability, Usability, Reliability, Performance Efficiency, and Security. Respondents rated each statement on a 5-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree). The questionnaire branched Functional Suitability into role-specific items — administrator respondents rated statements about inventory visibility, POS accuracy, and booking-conflict prevention, while customer respondents rated statements about the ordering, booking, and order-tracking experience — since these two user types do not have access to the same features of the system.

Seven respondents completed the questionnaire: the business owner and six customers. One customer response was excluded from the tallied results below because it rated every single item at the lowest possible score (1 — Strongly Disagree) while its accompanying open-ended comments were positive ("Good"), an internal inconsistency that made the rating data unreliable to interpret at face value. This exclusion is disclosed here, and again in Section 5.6 (Limitations), rather than left unstated.

Table 4.9 — Respondent Profile

Respondent	Role
(Business Owner)	Business Owner (Admin)
Mia Sombiro	Customer
Christian	Customer
Kriscel	Customer
Loiza	Customer
Ella Marie	Customer

Table 4.10 — UAT Results

Criterion	Mean Score	Interpretation
Functional Suitability — Admin (n = 1)	5.00	Excellent
Functional Suitability — Customer (n = 5)	4.60	Excellent
Usability (n = 6)	4.78	Excellent
Reliability (n = 6)	4.78	Excellent
Performance Efficiency (n = 6)	4.75	Excellent
Security (n = 6)	4.83	Excellent
Overall (mean of the six criteria above)	**4.79**	**Excellent**

Interpretation scale: 4.20–5.00 Excellent, 3.40–4.19 Very Good, 2.60–3.39 Good, 1.80–2.59 Fair, 1.00–1.79 Poor

4.10 Results and Discussion

The UAT results indicate an overall mean score of 4.79, interpreted as **Excellent** under the ISO/IEC 25010-based scale adopted for this study. All six evaluated criteria — Admin Functional Suitability, Customer Functional Suitability, Usability, Reliability, Performance Efficiency, and Security — fell within the Excellent range (4.20–5.00), suggesting that respondents found the system both functionally capable and pleasant to use across both the administrator and customer perspectives.

Security recorded the highest mean score among the shared criteria (4.83), which may reflect respondent confidence in the system's use of PayMongo's webhook-based payment verification (Section 4.3.6) rather than the manual screenshot review the bakery previously relied on. Usability and Reliability tied for the next-highest scores (4.78 each), consistent with the staged, guided-interface design discussed in Section 4.3.2 and grounded in the usability principles of Lowry et al. [3] reviewed in Chapter 2.

Customer Functional Suitability, while still Excellent overall (4.60), was the lowest-scoring criterion among the group, and showed the widest spread across individual respondents — one customer (Christian) rated the booking-date visibility and order-tracking statements notably lower (3 out of 5) than the other statements in that section. This suggests that while the overall ordering experience was well received, the booking calendar and order-status visibility may benefit from targeted refinement ahead of full deployment — a normal and expected finding at this stage rather than a weakness to conceal.

4.11 Summary

This chapter presented the design, technical implementation, architectural framework, hardware and software specifications, database schema, and evaluation of the Streamlined Bakery Management System for Mrs. Brave's Cakes. Core functional modules were fully implemented to directly address the key operational inefficiencies identified in Chapter 1.

The application is built on a **serverless client-cloud architecture** deployed on **Render**, backed by a real-time **Google Firestore NoSQL database** and **Firebase Authentication**. This structure was chosen specifically to eliminate local server maintenance while seamlessly accommodating the dynamic data structures required for custom cake orders, walk-in counter POS sales, and inventory management.

To ensure system reliability, data security, and operational smoothness, the platform underwent comprehensive multi-level testing—including unit, integration, system, security (covering Firebase security rules and PayMongo webhook validation), and performance testing. The successful execution of these tests, alongside the end-to-end order walkthrough and user acceptance evaluation results detailed in Sections 4.7 through 4.10, confirms that the system effectively meets its functional objectives and delivers a scalable, efficient management platform for the bakeshop.

CHAPTER 5

SUMMARY, CONCLUSION, AND RECOMMENDATIONS

5.1 Introduction

This chapter presents a summary of the study, conclusions drawn from the development and evaluation of the Streamlined Bakery Management System, and recommendations for its continued improvement and future research.

5.2 Summary of Findings

Based on the development lifecycle, technical implementation, and User Acceptance Testing (UAT) results presented in Chapter 4, the study yielded the following findings:

- **Successful Digitization of Order Management:** The Cake Customization Module effectively replaced the manual Facebook Messenger workflow. Customers reported high satisfaction with the interactive builder and real-time price calculation (Mean Score: 4.60), confirming that the structured digital interface reduced specification errors and communication latency.

- **Unified Inventory Synchronization:** The Walk-In POS Module successfully integrated physical counter sales with online orders into a single Firestore database. The Business Owner confirmed that automatic stock deductions prevented overselling and eliminated the need for end-of-day manual reconciliation, addressing the primary operational bottleneck identified in Chapter 1.

- **Effective Rule-Based Demand Analytics:** Instead of complex machine learning models, the system implemented a lightweight Python-based aggregation engine. This feature provided the administrator with historical consumption trends and low-stock alerts based on past transaction data. While not predictive in an autonomous sense, this statistical reporting significantly improved the owner's ability to plan ingredient purchases proactively rather than reactively.

- **Secure Payment Integration:** The PayMongo API integration successfully automated payment verification via webhooks, eliminating the risk of fraudulent GCash/Maya screenshots. The automated e-receipt generation via Resend API further enhanced trust and professionalism in customer interactions.

- **High Usability Despite Resource Constraints:** The UAT results indicated excellent usability scores across all roles (Admin: 5.00, Customer: 4.60). This validates that the responsive Bootstrap frontend and intuitive admin dashboard were accessible even to non-technical users, fulfilling the significance of the study for local MSMEs.

5.2.1 Detailed Findings per Module

To present a clearer picture of the system's evaluated performance, the findings are broken down per functional module. Each finding is anchored on the User Acceptance Testing results and the operational observations gathered during the deployment phase.

Cake Customization Module. The Cake Customization Module successfully replaced the manual Facebook Messenger workflow. During UAT, customers reported high satisfaction with the interactive builder and real-time price calculation, with a mean score of 4.60 under Functional Suitability. The most appreciated feature was the ability to see the price update instantly as options were selected, which eliminated the need for the back-and-forth price negotiations described in Chapter 1. The reference image upload feature was also praised for allowing customers to communicate their design vision visually rather than through long text descriptions.

Walk-In Point-of-Sale (POS) Module. The Walk-In POS Module integrated physical counter sales with the online Firestore database, directly resolving the "financial blindness" problem identified in Chapter 1. The business owner confirmed that automatic stock deductions prevented overselling and eliminated the need for end-of-day manual reconciliation. The thermal printer compatibility allowed for immediate receipt generation, which professionalized the walk-in customer experience.

Booking Calendar and Blackout Date Controller. The Booking Calendar successfully prevented overbooking during peak seasons. The administrator could lock specific dates for custom orders, premade orders, or both, with custom reasons displayed to customers. This feature directly addressed the scheduling conflicts identified in Chapter 1. The UAT results showed that customers found the date selection process clear and transparent.

Payment Integration Module. The Payment Integration Module automated payment verification via the PayMongo webhook listener, eliminating the risk of fraudulent GCash or Maya screenshots. The system never stored sensitive financial data, maintaining PCI-DSS compliance throughout the transaction lifecycle. The automated e-receipt generation via the Resend API further enhanced customer trust.

Rider Dispatch Portal. The Rider Dispatch Portal allowed independent couriers to access delivery details via a secure, time-bound link. The WebGIS interface, powered by Leaflet.js and OpenStreetMap, provided turn-by-turn navigation without requiring paid API subscriptions. The automated delivery confirmation via Firebase Cloud Messaging closed the logistical feedback loop instantly.

Inventory Management and Low-Stock Alerts Module. The Inventory Management Module maintained unified stock counts across online and walk-in channels. The low-stock alert system prompted timely reordering, reducing mid-production stockouts that had previously delayed order fulfillment. The historical consumption analytics provided the owner with data-driven insights for ingredient planning.

Business Analytics Dashboard. The Business Analytics Dashboard automatically processed raw sales and expense data into visual charts and performance indicators. The owner could view daily, weekly, and monthly trends, as well as net profit margins and top-selling products. This feature transformed the owner's ability to make informed business decisions.

User Authentication and Security Module. The authentication module supported email/password login, Google OAuth single sign-on, and WebAuthn biometric login. Role-based access control ensured that administrative functions were only accessible to verified administrators, while customer-facing features remained isolated from sensitive data. This module received the highest UAT score among shared criteria at 4.83 (Excellent).

5.3 Conclusion

The study concluded that the Streamlined Bakery Management System successfully achieved its general objective of transforming Mrs. Brave's Cakes from a manual, paper-based operation into a digitized, cloud-managed enterprise. Specifically:

1. **To Develop a Customization Module.** The system provides a step-by-step online form with real-time price calculation, reference image upload, and dynamic pricing. This objective was fully achieved, as evidenced by the UAT score of 4.60 for Customer Functional Suitability.

2. **To Implement a Dual-State Production Calendar.** The system includes a digital calendar with date locking, capacity tracking, and custom closure reasons. This objective was achieved, with the calendar successfully preventing overbooking during testing.

3. **To Construct an On-Site Point of Sale Module.** The POS module handles walk-in transactions, applies statutory Senior Citizen and PWD discounts, and prints thermal receipts. This objective was achieved, with the owner reporting a significant reduction in manual reconciliation time.

4. **To Deploy a Dedicated Rider Dispatch Interface.** The rider portal generates secure links with embedded WebGIS maps. This objective was achieved, with riders successfully delivering orders using only the web link.

5. **To Integrate a Centralized Inventory and Analytics Dashboard.** The inventory module tracks stock in real time and provides low-stock alerts. The analytics dashboard visualizes sales, expenses, and profit. This objective was achieved, with the owner gaining a clear view of business health.

6. **To Establish a Secure Payment and Receipting System.** The PayMongo integration automates payment verification and e-receipt dispatch. This objective was achieved, with the system eliminating screenshot fraud.

In essence, the system did not merely digitize existing processes; it re-engineered the bakery's workflow to prioritize real-time visibility, financial accuracy, and logistical coordination, directly resolving the structural failures identified in the problem statement.

5.3.1 Overall Impact

The Streamlined Bakery Management System transformed Mrs. Brave's Cakes from a manual, paper-based operation into a digitized, cloud-managed enterprise. The system did not merely digitize existing processes; it re-engineered the bakery's workflow to prioritize real-time visibility, financial accuracy, and logistical coordination. The UAT results, with an overall mean of 4.79 interpreted as Excellent, confirm that the system is both functionally capable and pleasant to use across both administrative and customer perspectives.

The study demonstrates that a purpose-built, moderately scoped system can bridge the gap between informal social media commerce and costly enterprise software for Philippine MSMEs. Rather than requiring the owner to adapt to a generic template, the system was designed around her actual daily routines—straightforward menus, clear status badges, and single-click actions—proving that the adoption barrier for non-technical operators is not technical skill but rather the fit between the tool and the existing business workflow.

5.4 Recommendations

Based on the conclusions drawn from this study, the following recommendations are proposed for future iterations and similar projects:

For the Business Owner (Mrs. Tapang):

1. **Consistent Data Entry:** To maximize the utility of the Historical Consumption Analytics module, ensure that all walk-in sales are processed through the POS terminal rather than recorded manually. Accurate historical data is required for the aggregation logic to provide meaningful insights.

2. **Regular Backup Verification:** Although Firestore offers redundancy, periodically export critical collections (orders, inventory) to CSV for offline archival purposes, ensuring long-term data preservation independent of cloud service availability.

3. **Staff Training for Future Expansion.** If additional staff are hired during peak seasons, the Admin Manual should be used as the official training reference to ensure consistent onboarding.

4. **Monitor Low-Stock Alerts.** Responding promptly to critical stock alerts prevents mid-production ingredient shortages. The historical consumption analytics should be reviewed weekly to identify trends.

For Future Developers / Researchers:

1. **Upgrade to True Predictive Modeling:** Once the system has accumulated at least 12–18 months of clean, structured transaction data, consider integrating actual Machine Learning algorithms (e.g., Linear Regression or ARIMA time-series forecasting) to replace the current rule-based aggregation. This would allow the system to account for seasonality, holidays, and external factors more accurately.

2. **Native Mobile Application Development:** While the responsive web design performs well, developing a dedicated Android/iOS app for riders could enable background GPS tracking and push notifications without requiring the rider to keep the browser open, further improving last-mile logistics efficiency.

3. **Hardware Independence in Team Projects:** For future capstone teams, establish a policy where each member maintains their own GitHub account and commits code regularly. If hardware sharing is unavoidable, strictly enforce the use of Co-authored-by trailers in Git commits to ensure transparent academic attribution and avoid potential plagiarism flags.

4. **Multi-Branch Support.** Extending the system architecture to support multiple branches with separate inventory, scheduling, and employee accounts would allow the platform to serve larger bakery chains.

5. **Barcode Scanning Support.** Adding barcode scanning to the Walk-In POS module would speed up counter transactions during peak hours.

6. **Longitudinal Impact Study.** Conducting a follow-up study after one year of live operation would allow future researchers to measure the system's long-term impact on sales volume, customer retention, and operational efficiency.

7. **Comparative Analysis.** Comparing the performance of this system against other bakery management solutions would

provide a more rigorous benchmark for its relative strengths and weaknesses.

8. **User Experience Research. I**nvestigating the usability of the cake customization module across different demographic groups would help refine the interface for broader accessibility.

9. **Formal Security Audit.** Performing a formal penetration test against the deployed system would strengthen the security claims of the study.

5.5 Impact of the Study

This study contributes to the broader effort of extending accessible, tailored digital tools to Philippine micro and small food enterprises — a segment often underserved by both informal social-media-based commerce and costly enterprise software, as discussed in Chapter 2's synthesis. By addressing the specific, real operational problems of an actual bakery rather than building a generic template, the study demonstrates that a purpose-built, moderately scoped system can bridge that gap.

For Mrs. Brave's Cakes specifically, the system's evaluated strengths in Security, Usability, and Reliability directly translate into operational benefits: a reduced risk of payment fraud through automated verification, a lower barrier to daily use for a non-technical operator, and a more dependable single source of truth for inventory and order data than the previous chat-and-calendar workflow allowed. These are not abstract technical improvements — they map onto the concrete financial and scheduling problems that motivated this study in Chapter 1.

More broadly, the study adds to the small but growing body of Philippine capstone and applied research — including the local studies discussed in Chapter 2 (Natividad et al [19]. ; Alberto et al. [21] ; Mia et al. [26]) — documenting how digital tools are actually adopted, or resisted, by small and micro food enterprises. Where much of the existing enterprise software literature assumes a level of technical infrastructure and staffing that micro-enterprises like Mrs. Brave's Cakes do not have, this study's UAT results (Section 4.10) offer a small, concrete data point suggesting that a carefully scoped, role-aware system — one designed around how a specific business owner already works rather than around a generic feature checklist — can be both usable and well received by that same population.

5.5.1 Economic Impact

The system has the potential to increase the bakery's revenue by reducing losses from ingredient waste, payment fraud, and overbooking. By enabling digital payments through GCash and Maya, the bakery can also capture customers who prefer cashless transactions. The use of free-tier cloud services keeps operational costs low, making the system financially sustainable.

5.5.2 Social Impact

By streamlining operations, the system reduces the owner's administrative workload, allowing her to focus on baking and family. The rider portal provides a professional, dignified workflow for independent motorcycle couriers. The digital payment integration also promotes financial inclusion by allowing customers without traditional bank accounts to participate in the digital economy.

5.5.3 Technological Impact

The study demonstrates that a serverless, NoSQL-based architecture can be successfully deployed for a micro-enterprise without requiring dedicated IT staff or expensive infrastructure. It provides a blueprint for other student developers and MSMEs who wish to build similar systems.

5.5.4 Environmental Impact

By enabling precise inventory tracking, the system helps reduce food waste. When ingredients are tracked accurately, over-purchasing is minimized and spoilage is reduced. Digital receipts and paperless order tracking reduce the bakery's reliance on paper logbooks.

5.6 Short-Term Enhancements

- Add SMS notifications for customers who do not have a registered email address.
-

- Implement a referral system that rewards existing customers for bringing new buyers.
- Add a favorites or "save for later" feature for quick reordering of frequently purchased items.
- Improve the analytics dashboard with exportable PDF reports.

5.7 Medium-Term Enhancements

- Develop a native mobile application for both customers and riders.
- Integrate machine learning models for demand forecasting once sufficient historical data has been accumulated.
- Add support for multiple languages, including Filipino and English.
- Implement a multi-branch architecture that supports separate inventory, scheduling, and staff accounts.

5.8 Long-Term Enhancements

- Integrate with IoT devices such as smart scales and temperature sensors.
- Add blockchain-based loyalty tokens that allow customers to trade or share their earned rewards.
- Develop an open API for third-party integrations.
- Expand the core architecture to serve other food businesses.

5.9 Summary

This final chapter synthesized the entirety of the study, drawing together the problem identification, system development, technical implementation, and empirical evaluation conducted over the course of the capstone project. The study was motivated by the operational realities of Mrs. Brave's Cakes, a homegrown pastry business in Iloilo City that had, since its establishment in 2020, relied almost entirely on manual, fragmented processes—Facebook Messenger chats for order-taking, paper logbooks for sales recording, a physical desk calendar for scheduling, and personal GCash accounts for payment verification. While these manual methods had been sufficient during the business's early micro-stages, they introduced severe operational bottlenecks as order volumes scaled: miscommunication during custom cake consultations, inconsistent pricing due to mental arithmetic, overbooking during peak seasons, unverified payment screenshots, ingredient stockouts mid-production, and a complete lack of visibility over last-mile delivery coordination. Each of these bottlenecks was documented in Chapter 1 through semi-structured interviews with the business owner and direct observation of the store's daily operations.

The study responded to these bottlenecks through the design, development, and deployment of the Streamlined Bakery Management System—a unified, cloud-based web platform purpose-built for the operational realities of a custom cake bakery. The system was developed using the Agile Software Development Model, which allowed the researchers to iteratively refine features across multiple development sprints in direct response to feedback from Mrs. Tapang. Five core functional modules were implemented: (1) the Cake Customization Module, which replaced unstructured chat-based ordering with a guided, step-by-step form featuring real-time price calculation and reference image uploads; (2) the Walk-In Point-of-Sale Module, which unified physical counter sales with the same inventory ledger used by the online storefront; (3) the Booking Calendar and Blackout Date Controller, which enabled the administrator to lock specific dates for custom orders, premade orders, or both, with real-time enforcement visible to customers; (4) the Payment Integration Module, which automated payment verification through the PayMongo API and its cryptographic webhook listener, eliminating the risk of fraudulent screenshot submission; and (5) the Rider Dispatch Portal, which provided a lightweight, zero-install web interface for independent motorcycle couriers using Leaflet.js and OpenStreetMap for real-time route visualization.

The architecture of the system was deliberately designed to be serverless, modular, and cost-effective. The backend was built on Python (Flask), the frontend on HTML5, CSS3, JavaScript, and Bootstrap, and the data layer on Google Firestore—a NoSQL document database chosen specifically for its native real-time synchronization via WebSockets, which enabled the unified inventory ledger to reflect changes instantly across all user terminals. Firebase Authentication handled identity management

across email/password, Google OAuth, and WebAuthn biometric logins, while Cloudinary managed image assets, Resend handled transactional e-receipts, and Gemini AI provided automated customer support for frequently asked questions. The entire system was deployed on Render's free tier, ensuring that the bakery did not incur significant capital expenditure for server infrastructure, in keeping with the economic constraints of a micro-enterprise.

The system was evaluated using a User Acceptance Testing methodology anchored on the ISO/IEC 25010 software quality standard, which assessed the system across functional suitability, usability, reliability, performance efficiency, and security. The UAT questionnaire was administered to seven respondents—the business owner and six customers—with one respondent's data excluded due to internal inconsistency. The results, presented in Chapter 4, Section 4.9, returned an overall mean score of 4.79, interpreted as Excellent. Security recorded the highest mean score at 4.83, reflecting respondents' confidence in the system's use of PayMongo's webhook-based payment verification rather than manual screenshot review. Usability and Reliability tied for the next-highest scores at 4.78 each, consistent with the guided, staged interface design principles discussed in Chapter 2. Customer Functional Suitability, while still Excellent overall at 4.60, was the lowest-scoring criterion and highlighted specific areas—such as the booking calendar and order status visibility—that may benefit from targeted refinement in future iterations.

This final chapter has demonstrated that the Streamlined Bakery Management System successfully met its six specific objectives, as originally stated in Chapter 1. First, the system fully automated the cake customization and pricing process through a guided online form with real-time price calculation. Second, the system implemented a dual-state production calendar that allowed the administrator to lock specific dates for custom orders, premade orders, or both, with reasons displayed transparently to customers. Third, the system constructed a functional on-site Point of Sale module capable of processing walk-in transactions, applying statutory discounts, and printing thermal receipts. Fourth, the system deployed a dedicated Rider Dispatch Portal that provided independent couriers with secure, time-bound access to delivery details and WebGIS-based navigation without requiring an account. Fifth, the system integrated a centralized inventory and analytics dashboard that tracked stock in real time and generated automated sales and expense reports. Sixth, the system established a secure payment and receipting infrastructure through the PayMongo integration, eliminating screenshot fraud and automatically dispatching e-receipts via the Resend API.

Beyond the specific objectives, the study demonstrated a broader contribution to the growing body of Philippine capstone and applied research on digital transformation among micro-enterprises. By documenting the technical implementation, database architecture, testing methodology, and user evaluation results of a system designed specifically for a small artisanal food business, this study provides a replicable blueprint for other MSMEs seeking to transition from fragmented social-media-based operations into structured, cloud-managed platforms. The system's reliance on free-tier cloud services, open-source libraries, and affordable APIs demonstrates that enterprise-grade functionality can be achieved without enterprise-level budgets—a critical insight for a market segment that is often underserved by both informal social media commerce and prohibitively expensive ERP software.

The chapter has also outlined clear recommendations for the business owner, future developers, and future researchers. For Mrs. Tapang, consistent data entry through the POS terminal and periodic CSV backup verification are the two most important operational habits for preserving the value of the system's analytical features. For future developers, the study recommends upgrading to true predictive modeling once sufficient historical data has been accumulated, developing native mobile applications for riders, expanding to multi-branch support, and integrating IoT devices for automated inventory tracking. For future researchers, the study recommends conducting a longitudinal impact study, performing comparative analysis against other bakery management solutions, investigating usability across different demographic groups, and conducting a formal security audit to strengthen the study's security claims.

The research provides a foundation for continued digital transformation efforts among small, artisanal Philippine food businesses. It is the researchers' sincere hope that this study serves not only as a technical reference for future capstone teams but also as a practical demonstration that with careful scoping, thoughtful design, and a commitment to aligning technology with actual business workflows, even the smallest of food enterprises can benefit from the tools of modern digital commerce.

REFERENCES

[1] P. C. Verhoef, T. Broekhuizen, Y. Bart, A. Bhattacharya, J. Q. Dong, N. Fabian, and M. Haenlein, "Digital transformation: A multidisciplinary reflection and research agenda," Journal of Business Research, vol. 122, pp. 889–901, 2021, doi: 10.1016/j.jbusres.2019.09.022.

[2] W. A. Wallace and E. M. Stark, "A critique of the shelf-life performance and predictability methodologies for perishable foods," Critical Reviews in Food Science and Nutrition, vol. 45, no. 6–7, pp. 411–427, 2005, doi: 10.1080/10408690590956583.

[3] P. B. Lowry, T. M. Wells, G. D. Moody, S. Humphreys, and D. Kettles, "Online trust, security, privacy, and user interface elements in e-commerce," Communications of the Association for Information Systems, vol. 17, no. 6, pp. 1–48, 2006, doi: 10.17705/1CAIS.01706.

[4] N. DeHoratius and A. Raman, "Inventory record inaccuracy: An empirical analysis," Management Science, vol. 54, no. 4, pp. 627–641, 2008, doi: 10.1287/mnsc.1070.0789.

[5] J. F. Bard and D. P. Morton, "Workforce planning at USPS mail processing and distribution centers using stochastic optimization," Annals of Operations Research, vol. 152, no. 1, pp. 51–72, 2007, doi: 10.1007/s10479-007-0213-1.

[6] Y. Kim and Y. Park, "Consumer acceptance of cloud-based point-of-sale systems: A technology acceptance model approach," Journal of Retailing and Consumer Services, vol. 50, pp. 145–155, 2019, doi: 10.1016/j.jretconser.2019.05.001.

[7] J. F. Chen, L. Wang, Y. Liang, Y. Yu, J. Feng, J. Zhao, and X. Ding, "Order dispatching via GNN-based optimization algorithm for on-demand food delivery," IEEE Transactions on Intelligent Transportation Systems, vol. 25, no. 11, pp. 1–14, 2024, doi: 10.1109/TITS.2024.3389090.

[8] F. Lu, J. Liu, and H. Bi, "Drone–rider joint delivery routing with arc obstacle avoidance," Applied Sciences, vol. 15, no. 21, Art. no. 11469, 2025, doi: 10.3390/app152111469.

[9] J. Han, E. Haihong, G. Le, and J. Du, "Survey on NoSQL database," in Proc. 2011 6th Int. Conf. Pervasive Comput. Appl. (ICPCA), 2011, pp. 363–366, doi: 10.1109/ICPCA.2011.6106531.

[10] J. Carbonneau, K. Laframboise, and R. Vahidov, "Application of machine learning algorithms to predict future stock demands," International Journal of Production Economics, vol. 114, no. 2, pp. 505–520, 2008, doi: 10.1016/j.ijpe.2008.02.012.

[11] A. Sharafat and Z. Khan, "Point of sale (POS) operational usability and data integrity in retail counters," Journal of Computer Science, vol. 18, no. 2, pp. 145–159, 2022.

[12] K. L. Gwebu and J. Wang, "Automated electronic invoicing and transaction verification in e-commerce channels," Journal of Electronic Commerce Research, vol. 12, no. 4, pp. 289–305, 2011.

[13] M. Faraz and P. Chausson, "SQL vs. NoSQL: Optimizing database performance for modern e-commerce systems," ResearchGate, 2022, doi: 10.13140/RG.2.2.23339.25126.

[14] E. S. Soegoto and A. F. Palalungan, "Web based online inventory information system," IOP Conference Series: Materials Science and Engineering, vol. 879, no. 1, Art. no. 012125, 2020, doi: 10.1088/1757-899X/879/1/012125.

[15] Y. Qiu, J. Qiao, and P. M. Pardalos, "Optimal production replenishment, delivery routing, and inventory management for perishable items," Omega, vol. 82, pp. 129–143, 2019, doi: 10.1016/j.omega.2018.01.006.

[16] M. C. Enache, "Machine learning for dynamic pricing in e-commerce," Annals of "Dunarea de Jos" University of Galati: Fascicle I. Economics and Applied Informatics, vol. 27, no. 3, pp. 45–52, 2021, doi: 10.35219/eai15840409230.

[17] J. Rautenstrauch, K. Mitkov, J. Helbrecht, T. Hetterich, and B. Stock, "To auth or not to auth? A comparative analysis of the pre- and post-login security landscape," in 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 1845–1862, doi: 10.1109/SP54263.2024.00094.

[18] B. Mickiewicz and I. Britchenko, "Main trends and development forecast of bread and bakery products market," VUZF Review, vol. 7, no. 3, pp. 113–123, 2022, doi: 10.38188/2534-9228.22.3.11.

[19] E. A. E. Natividad, A. M. M. Musngi, R. B. Molinyawe, E. A. Rivera, R. C. Torres, and D. E. L. Moreno, "System quality and micro food businesses' acceptance of the cloud-based point of sale system for inventory management," in Proc. 2024 15th Int. Conf. E-Business, Manag. Econ. (ICEME), 2024, pp. 123–129, doi: 10.1145/3652885.3653012.

[20] J. De Guzman, J. Malcon, J. Dimla, A. Rambuyong, E. Zamora, and F. Garcia-Vigonte, "e-Commerce food delivery challenges: The case of Order Mo," Social Science Research Network, SSRN Scholarly Paper No. 4024999, 2022, doi: 10.2139/ssrn.4024999.

[21] T. J. C. Alberto, Z. D. Budhrani, J. L. M. Moreno, and L. R. Salazar, "The transition from traditional commerce to e-commerce and their effects on small-scale businesses during the COVID-19 pandemic," Unpublished bachelor's thesis, Ramon V. Del Rosario College of Business, De La Salle Univ., Manila, Philippines, 2022.

[22] J. Divina, A. Olan, N. C. Perez, R. Acepcion, M. Sarmiento, and G. Cabututan, "Data-driven point-of-sale and inventory system for Pastil sa Tabi: Integrating sales forecasting algorithms and predictive analytics," International Journal of Research and Innovation in Applied Science, vol. 10, no. 10, pp. 826–838, 2025, doi: 10.51584/IJRIAS.2025.1010000066.

[23] J. M. Rosario, "Implementation of inventory control management and repeat purchase in Right Goods Philippines Incorporated: Inputs to policy reformulation," World Wide Journal of Multidisciplinary Research and Development, vol. 8, no. 6, pp. 104–112, 2022.

[24] Ma. J. J. Gumasing, G. K. M. Cangco, S. M. C. Ilagan, and E. G. A. Reyes, "A comparison on the usability of mobile e-wallet applications: GCash, Maya, Grabpay," in Proc. 2023 14th Int. Conf. E-Education, E-Business, E-Manag. E-Learning, 2023, pp. 199–207, doi: 10.1145/3588243.3588253.

[25] L. F. Ramento, D. M. Dancel, E. Taberlo, and K. J. Anquillano, "Web-based reservation and management system for Nature Spring Resort," Journal of Pure and Applied Sciences, vol. 2, no. 2, 2024, doi: 10.5281/zenodo.20739138.

[26] I. B. Mia, S. Jiménez, R. Habaradas, J. J. Ranieses, C. Javier, and J. Enríquez, "Digital technology adoption among Philippine micro-, small-, and medium-sized enterprises: Barriers, enablers & challenges during COVID-19," Journal of Business, Ethics and Society, vol. 4, no. 1, 2024, doi: 10.61781/4-1I2024/1bmlm.

[27] Organisation for Economic Co-operation and Development (OECD), The Digital Transformation of SMEs, OECD Studies on SMEs and Entrepreneurship. Paris, France: OECD Publishing, 2021, doi: 10.1787/bdb9256a-en.

[28] Statista, "Digital wallets: Statistics and facts," Statista Research Department, 2023. [Online]. Available: https://www.statista.com/topics/8900/digital-wallets/

[29] Deloitte, "The cloud imperative: Asia Pacific's unmissable opportunity," Deloitte Access Economics, 2020. [Online]. Available: https://www.deloitte.com/

[30] K. Moran, "Usability testing 101," Nielsen Norman Group, Dec. 1, 2019. [Online]. Available: https://www.nngroup.com/articles/usability-testing-101/

[31] Kaspersky, "Small business cybersecurity: 39% of SMBs struggle to fund cybersecurity," Kaspersky Lab, Aug. 30, 2021. [Online]. Available: https://usa.kaspersky.com/blog/smb-security-2021/

[32] Asian Development Bank (ADB), "Unintended consequences of business digitalization among MSMEs during the COVID-19 pandemic: The case of the Philippines," ADB Briefs, Feb. 2025, doi: 10.22617/BRF250562-2.

[33] Bangko Sentral ng Pilipinas (BSP), 2021 Financial Inclusion Survey: Topline Report. Manila, Philippines: BSP Financial Inclusion Office, 2022. [Online]. Available: https://www.bsp.gov.ph/Inclusive%20Finance/Financial%20Inclusion%20Reports%20and%20Publications/2021/2021FISToplineReport.pdf

[34] J. A. Bacasmas, J. C. Carlos, and J. J. Katigbak, "E-commerce adoption and its impact on the performance of women-led MSMEs in Metro Manila: An ex-ante study for RCEP," Philippine Institute for Development Studies (PIDS), Discussion Paper Series No. 2022-38, Dec. 2022. [Online]. Available: https://pidswebs.pids.gov.ph/CDN/document/1656407483_62bac5bb3aede.pdf

[35] F. D. Davis, "Perceived usefulness, perceived ease of use, and user acceptance of information technology," MIS Quarterly, vol. 13, no. 3, pp. 319–340, 1989, doi: 10.2307/249008.

[36] V. Venkatesh, M. G. Morris, G. B. Davis, and F. D. Davis, "User acceptance of information technology: Toward a unified view," MIS Quarterly, vol. 27, no. 3, pp. 425–478, 2003, doi: 10.2307/30036540.

[37] E. M. Rogers, Diffusion of Innovations, 5th ed. New York, NY, USA: Free Press, 2003.

Note to the instructors handling this subject

CHED BSIT ALIGNMENT STATEMENT

This capstone project complies with CHED BSIT standards by demonstrating competencies in application development, database management, systems analysis and design, and emerging technologies such as machine learning. The project followed outcomes-based education principles and integrates ethical considerations, user safety, and data privacy.